

Bachelorarbeit
im Studiengang Medieninformatik

Performance- und Ressourcenanalyse von React und Svelte bei steigender UI-Dichte und Update-Frequenz

vorgelegt von

Jonas Öhler
Matrikelnummer: 44338

an der Hochschule der Medien Stuttgart am
20.07.2026
zur Erlangung des akademischen Grades eines Bachelor of Science

Erstprüfer: Prof. Dr. Simon Wiest
Zweitprüfer: Prof. Dr. Tobias Jordine

Ehrenwörtliche Erklärung

Hiermit versichere ich, Jonas Öhler, ehrenwörtlich, dass ich die vorliegende Bachelorarbeit mit dem Titel: „Performance- und Ressourcenanalyse von React und Svelte bei steigender UI-Dichte und Update-Frequenz“ selbstständig und ohne fremde Hilfe verfasst und keine anderen als die angegebenen Hilfsmittel benutzt habe. Die Stellen der Arbeit, die dem Wortlaut oder dem Sinn nach anderen Werken entnommen wurden, sind in jedem Fall unter Angabe der Quelle kenntlich gemacht. Ebenso sind alle Stellen, die mit Hilfe eines KI-basierten Schreibwerkzeugs erstellt oder überarbeitet wurden, kenntlich gemacht. Die Arbeit ist noch nicht veröffentlicht oder in anderer Form als Prüfungsleistung vorgelegt worden.

Ich habe die Bedeutung der ehrenwörtlichen Versicherung und die prüfungsrechtlichen Folgen (§ 24 Abs. 2 Bachelor-SPO, § 23 Abs. 2 Master-SPO (Vollzeit)) einer unrichtigen oder Mai 2023 Prüfungsverwaltung 6 unvollständigen ehrenwörtlichen Versicherung zur Kenntnis genommen

Ort, Datum

Unterschrift

Zusammenfassung

Reaktionsfähige Benutzeroberflächen sind in datenintensiven Echtzeit-Anwendungen wie Monitoring-Dashboards ein zentrales Qualitätsmerkmal. React 19 und Svelte 5 verfolgen dabei grundlegend unterschiedliche Architekturansätze: React 19 kombiniert ein Virtual-DOM-basiertes Laufzeitmodell mit einem Compiler zur automatischen Memoisierung, während Svelte 5 compilerbasierte feingranulare Reaktivität einsetzt. Bestehende Vergleichsstudien basieren jedoch überwiegend auf älteren Framework-Versionen und untersuchen vorwiegend isolierte Operationen; empirische Daten unter kombinierter Dauerlast aus steigender UI-Dichte und hoher Update-Frequenz fehlen bislang. Diese Arbeit untersucht, wie sich React 19 und Svelte 5 hinsichtlich Interaktionslatenz (INP), Scripting-Aufwand und Speicherverbrauch unter kombiniert steigender UI-Dichte und Update-Frequenz unterscheiden und ab welcher Last-Kombination *Long Tasks* den Main Thread dauerhaft blockieren. Dazu wurden in einem automatisierten Benchmark (Playwright, Chrome DevTools Protocol) 1920 Messläufe über acht UI-Dichtestufen ($n \in \{100 \dots 10\,000\}$) und vier Update-Frequenzen (10 bis 62 Hz) durchgeführt. Als Metriken dienten INP, Scripting-Aufwand pro Tick, JS-Heap-Delta sowie Long-Task-Zähler. Die Ergebnisse zeigen, dass kein Framework uneingeschränkt überlegen ist. Bei hoher UI-Dichte liefert Svelte 5 in der Mehrzahl der Szenarien geringeren Scripting-Aufwand und niedrigere INP-Werte; zudem liegt Sveltes *Tipping Point* bei höheren n -Werten. Über alle Dichtestufen weist Svelte 5 zudem einen 2,8- bis 6,4-fach geringeren Heap-Verbrauch auf. Allerdings erzielt React 19 bei moderater UI-Dichte und hoher Update-Frequenz durch kooperatives Yielding der Fiber-Engine niedrigere INP-Werte. Unter Extremelast ($n = 10\,000$, 62 Hz) kehrt sich das Long-Task-Verhältnis um: Svelte erzeugt dort mehr *Long Tasks* als React. Für die Praxis ergibt sich: Svelte 5 ist bei hoher UI-Dichte und in speicherkritischen Umgebungen vorzuziehen; React 19 kann bei moderater Dichte und hoher Update-Frequenz die bessere Wahl sein.

Abstract

Responsive user interfaces are a central quality criterion in data-intensive real-time applications such as monitoring dashboards. React 19 and Svelte 5 pursue fundamentally different architectural approaches: React 19 combines a virtual DOM-based runtime model with a compiler for automatic memoization, while Svelte 5 employs compiler-based fine-grained reactivity. However, existing comparison studies are predominantly based on older framework versions and examine primarily isolated operations; empirical data under combined sustained load from increasing UI density and high update frequency are lacking. This work investigates how React 19 and Svelte 5 differ with respect to interaction latency (INP), scripting effort, and memory consumption under combined increasing UI density and update frequency, and at which load combination *Long Tasks* persistently block the main thread. To this end, 1920 measurement runs were conducted in an automated benchmark (Playwright, Chrome DevTools Protocol) across eight UI density levels ($n \in \{100 \dots 10\,000\}$) and four update frequencies (10 to 62 Hz). The metrics used were INP, scripting effort per tick, JS heap delta, and long task count. The results show that neither framework is universally superior. At high UI density, Svelte 5 delivers lower scripting effort and lower INP values in the majority of scenarios; Svelte's *tipping point* additionally lies at higher n values. Across all density levels, Svelte 5 additionally exhibits a 2.8- to 6.4-fold lower heap consumption. However, React 19 achieves lower INP values at moderate UI density and high update frequency through cooperative yielding of the Fiber engine. Under extreme load ($n = 10\,000$, 62 Hz), the long task ratio reverses: Svelte produces more *long tasks* than React. In practice: Svelte 5 is preferable at high UI density and in memory-constrained environments; React 19 may be the better choice at moderate density and high update frequency.

Inhaltsverzeichnis

1. Einleitung	1
1.1. Motivation	1
1.2. Ausgangssituation und Forschungslücke	1
1.3. Zielsetzung und Forschungsfrage	2
1.4. Aufbau der Arbeit	2
2. Theoretische Grundlagen	3
2.1. Rendering-Konzepte im Browser	3
2.1.1. Der Critical Rendering Path und das DOM	3
2.1.2. Herausforderungen durch hochfrequente Updates (Main-Thread-Blocking)	6
2.2. Architektur von React	7
2.2.1. Virtual DOM und Reconciliation	7
2.2.2. Der React Compiler (ab React 19): Automatisierte Memoisierung vs. manu- elles Opt-in	7
2.2.3. Fiber-Engine: Scheduling und Rendering	8
2.3. Architektur von Svelte	9
2.3.1. Compilerbasierter Ansatz	9
2.3.2. Reaktivität und Update-Mechanismen	10
2.3.3. Effizienz der imperativen DOM-Updates	11
2.4. Vergleichbare Metriken und Performancebegriffe	11
2.4.1. Interaktionslatenz und Antwortverhalten (INP)	11
2.4.2. Analyse der Speichereffizienz (Heap-Snapshots)	12
2.4.3. Scripting- vs. Rendering-Zeit im Browser-Profil	14
2.5. Theoretische Vergleichsmodelle und Hypothesen	15
2.5.1. Synthese der Abstraktionskosten	15
2.5.2. Komplexität und Skalierung (n vs. f)	15
2.5.3. Erwartetes Skalierungsverhalten	16
2.5.4. Hypothesen	17
3. Stand der Forschung	18
3.1. Existierende Framework-Benchmarks	18
3.1.1. Methodik synthetischer Benchmarks	18
3.1.2. Empirische Performance-Profil: React 19 und Svelte 5	18
3.1.3. Kritische Würdigung: Die methodische Grenze	19
3.2. Akademische Leistungsstudien und verwandte Arbeiten	19
3.2.1. Analysen zum Virtual-DOM-Overhead	19
3.2.2. Laufzeit- vs. Compiler-Architekturen im direkten Vergleich	19

3.3.	Identifikation der Forschungslücke	20
3.3.1.	Fokus auf veraltete Framework-Generationen	20
3.3.2.	Fehlende Untersuchung von kombinierter Dauerlast	20
3.3.3.	Ableitung für diese Arbeit	20
4.	Methodik und Versuchsaufbau	21
4.1.	Definition der Test-Szenarien und Testmatrix	21
4.1.1.	Szenario A: Variierende UI-Dichte	21
4.1.2.	Szenario B: Variierende Update-Frequenz	22
4.1.3.	Kombinierte Testmatrix	22
4.1.4.	Gemessene Metriken und Hypothesenzuordnung	23
4.2.	Testumgebung und Standardisierung	24
4.2.1.	Hardware-Spezifikation	24
4.2.2.	Software-Versionierung und Build-Konfiguration	24
4.3.	Automatisierung und statistische Methodik	26
4.3.1.	Teststeuerung mittels Playwright und CDP-Trace-Analyse	26
4.3.2.	Messwiederholungen und Warmup-Lauf	28
4.3.3.	Statistische Auswertung: Median und IQR	29
4.3.4.	Normalisierung der Scripting-Zeit: <code>scriptingMsPerTick</code>	29
4.3.5.	Zweigeteilte Speicheranalyse	29
4.4.	Validität und Gütekriterien	30
4.4.1.	Interne Validität: Sicherstellung identischer Logik in beiden Implementierungen	30
4.4.2.	Externe Validität: Übertragbarkeit auf reale Anwendungen	32
4.4.3.	Messgenauigkeit und Störfaktoren	33
5.	Implementierung	35
5.1.	Architektur des Monorepos und Shared Logic	35
5.1.1.	Das Paket <code>@benchmark/shared-logic</code>	35
5.1.2.	Datenmodell: Interfaces <code>GridItem</code> , <code>TickMutation</code> und <code>TestPlan</code>	36
5.1.3.	Implementierung der <code>MockEngine</code>	36
5.1.4.	Der drift-korrigierte Web-Worker-Timer (<code>ticker.worker.ts</code>)	36
5.2.	Technische Umsetzung der React-Referenzanwendung	37
5.2.1.	Komponentenstruktur und Prop-Weitergabe	37
5.2.2.	Unveränderliche Referenzaktualisierung im Tick-Handler	37
5.2.3.	Steuerung durch den Web Worker und Benchmark-Ablauf	38
5.2.4.	Integration des React Compilers	39
5.3.	Technische Umsetzung der Svelte-Referenzanwendung	39
5.3.1.	Komponentenstruktur und Prop-Weitergabe	39
5.3.2.	Direkte Zustandsmutation im Tick-Handler	40
5.3.3.	Steuerung durch den Web Worker und Benchmark-Ablauf	40
5.4.	Technische Umsetzung des Benchmark-Pakets	41
5.4.1.	Playwright-Konfiguration: <code>playwright.config.ts</code>	41
5.4.2.	Szenariensteuerung: <code>scenarios.ts</code>	41

5.4.3.	Performance-Messung: <code>performance.spec.ts</code>	41
5.4.4.	Speichermessung: <code>memory.spec.ts</code>	42
5.4.5.	INP-Messung: <code>inp.spec.ts</code>	42
5.4.6.	CDP-Hilfsfunktionen: <code>cdp.ts</code>	43
5.4.7.	Ergebnisspeicherung und Auswertungsexport: <code>results.ts</code>	43
6.	Ergebnisse und Auswertung	45
6.1.	H2: Scripting-Aufwand pro Tick bei steigender UI-Dichte	45
6.1.1.	Scripting-Zeit, Overhead-Faktor und Frequenzeinfluss	45
6.1.2.	Bewertung von H2	48
6.2.	H3: Speichereffizienz unter Dauerlast	48
6.2.1.	Qualitative Heap-Analyse: Dominator-Struktur im Dev-Build	48
6.2.2.	JS-Heap-Delta und Speicher-Overhead-Verhältnis	50
6.2.3.	Bewertung von H3	52
6.3.	H1: Interaktionslatenz (INP) unter Last	52
6.3.1.	INP-Verlauf über die Grid-Größe	52
6.3.2.	Zerlegung der INP-Phasen	53
6.3.3.	Bewertung von H1	55
6.4.	H4: Identifikation des Tipping Points	56
6.4.1.	Frame-Budget-Auslastung und Long Tasks	56
6.4.2.	Bewertung von H4	58
6.5.	Zusammenfassung der Ergebnisse	58
6.6.	Limitationen	59
6.6.1.	Fehlende Isolation des Compiler-Einflusses	59
6.6.2.	Ausschließlich Chromium	59
6.6.3.	Verbleibende Browser-interne Störeinflüsse	60
6.6.4.	Presentationdelay als zusammengesetzte Metrik	60
7.	Fazit	61
7.1.	Beantwortung der Forschungsfrage	61
7.2.	Handlungsempfehlung für die Praxis	61
7.3.	Ausblick	62
	Literatur	63
	Abkürzungs- und Symbolverzeichnis	69
	Abbildungsverzeichnis	71
	Tabellenverzeichnis	72
	Anhang	73
	A. Quellcode-Listings	74
	B. Vollständige Messergebnisse	78

1. Einleitung

1.1. Motivation

Die Reaktionsfähigkeit einer Webanwendung ist ein zentraler Bestandteil der Nutzererfahrung: Verzögerungen bei der Eingabeverarbeitung sowie ruckelnde Animationen beeinträchtigen unmittelbar die wahrgenommene Qualität der Benutzeroberfläche [1]. Mit der Einführung der Core Web Vitals hat Google diesen Zusammenhang messbar gemacht. Die Metrik *Interaction to Next Paint* (INP) misst über die gesamte Nutzungsdauer hinweg, wie schnell eine Webseite auf Nutzereingaben reagiert [2]. Besonders herausfordernd sind datenintensive Echtzeit-Anwendungen wie Monitoring-Dashboards oder interaktive Visualisierungen. In solchen Systemen führen kontinuierliche Datenströme dazu, dass große Teile der Benutzeroberfläche in hoher Frequenz aktualisiert werden [3]. Jede dieser Operationen belastet den sogenannten *Main Thread* des Browsers, auf dem sowohl die Framework-Logik als auch die Rendering-Pipeline ausgeführt werden [4]. Überschreitet die Verarbeitungszeit pro Frame das verfügbare Zeitfenster von etwa 16,7 ms, entstehen wahrnehmbare Stockungen (*jank*) und eine erhöhte Interaktionslatenz [5]. Die Wahl des UI-Frameworks bestimmt dabei entscheidend, wie effizient dieses begrenzte Zeitbudget genutzt wird.

1.2. Ausgangssituation und Forschungslücke

React und Svelte basieren auf grundsätzlich unterschiedlichen Architekturansätzen. React setzt auf ein Virtual-DOM-basiertes Laufzeitmodell, bei dem Zustandsänderungen zur Laufzeit durch einen Differenzierungsalgorithmus (*Reconciliation*) in DOM-Operationen umgewandelt werden [6]. Svelte setzt stattdessen auf einen Compiler, der den Komponentencode bereits zur Kompilierzeit in direkten DOM-Manipulationscode übersetzt. Dadurch werden zur Laufzeit weder ein Virtual DOM noch ein Differenzierungsverfahren benötigt [7]. React 19 und Svelte 5 haben in beiden Frameworks neue Architekturkonzepte eingeführt. React 19 integriert einen Compiler, der die bisher manuelle Memoisierung von Komponenten automatisiert [8]. Svelte 5 führt mit sogenannten *Runes* ein signalbasiertes Reaktivitätsmodell ein, bei dem zur Laufzeit automatisch erkannt wird, welche DOM-Knoten bei einer Zustandsänderung aktualisiert werden müssen [9]. Da beide Neuerungen das jeweilige Reaktivitätsmodell grundlegend verändern, lassen sich frühere Leistungsmessungen nicht mehr auf die aktuellen Versionen übertragen. Diese Entwicklung wird in der bisherigen Forschungsliteratur nicht berücksichtigt. Vergleichende Leistungsstudien basieren überwiegend auf älteren Framework-Versionen wie Svelte 3 und React 16 oder 18 [10–12]. Zudem untersuchen bestehende Benchmarks nur isolierte Operationen wie das initiale Rendern oder das Einfügen von Listeneinträgen unter kontrollierten Bedingungen [13]. Ungeklärt ist bislang, wie beide Frameworks unter einer kombinierten Dauerlast aus hoher UI-Dichte und hoher Update-Frequenz reagieren

und ab welchem Punkt diese Last den Main Thread durch *Long Tasks* dauerhaft blockiert (vgl. Abschnitt 3.3).

1.3. Zielsetzung und Forschungsfrage

Ziel dieser Arbeit ist es, den *Tipping Point* beider Frameworks empirisch zu bestimmen: den Übergang, an dem die kombinierte Last aus UI-Dichte n (Anzahl gleichzeitig gerenderter Komponenten) und Update-Frequenz f den Main Thread durch Long Tasks [14] dauerhaft blockiert und die Interaktionslatenz messbar verschlechtert. Dabei wird konkret untersucht, welches der beiden Frameworks (React 19 mit aktiviertem Compiler oder Svelte 5 mit Runes) diesen Tipping Point bei welcher Last-Kombination zuerst erreicht und wie sich die Skalierungseigenschaften beider Architekturen unterhalb dieses Schwellenwerts unterscheiden. Die zentrale Forschungsfrage dieser Arbeit lautet daher:

Wie unterscheiden sich React 19 und Svelte 5 hinsichtlich Interaktionslatenz, Scripting-Aufwand und Speicherverbrauch bei kombiniert steigender UI-Dichte und Update-Frequenz, und welche Last-Kombination führt dabei zur dauerhaften Blockierung des Main Threads?

Daraus ergeben sich vier Hypothesen, die den empirischen Teil der Arbeit gliedern (vgl. Abschnitt 2.5.4):

H1 untersucht, ob Svelte 5 bei steigender UI-Dichte eine geringere Interaktionslatenz (INP) aufweist als React 19 und ob Reacts kooperatives Yielding der Fiber-Engine diesen Vorteil bei Last unterhalb des Frame-Budgets aufheben kann.

H2 fragt, ob der Scripting-Aufwand pro Tick bei React 19 mit steigendem n stärker wächst als bei Svelte 5.

H3 prüft, ob der Speicherverbrauch pro Komponente bei React 19 höher ist als bei Svelte 5.

H4 untersucht, ob React 19 den Tipping Point bei kombinierter Last früher erreicht als Svelte 5 und ob Sveltes synchrones Effect-Batching ohne Frame-Yielding diese Rangfolge bei extremer Last umkehren kann.

1.4. Aufbau der Arbeit

Kapitel 2 legt die theoretischen Grundlagen (Rendering-Pipeline moderner Browser, die internen Architekturen von React 19 und Svelte 5, die relevanten Metriken sowie das Skalierungsmodell mit den vier Hypothesen); Kapitel 3 ordnet die Arbeit in den Forschungsstand ein und begründet die identifizierte Forschungslücke. Kapitel 4 beschreibt Versuchsaufbau, Testmatrix, Testumgebung sowie die statistische Auswertung der Messreihen. Kapitel 5 dokumentiert die technische Umsetzung der Referenzanwendungen und des Benchmark-Pakets. Kapitel 6 präsentiert und diskutiert die Ergebnisse anhand der vier Hypothesen und schließt mit einer Darstellung der Limitationen.

2. Theoretische Grundlagen

2.1. Rendering-Konzepte im Browser

Dieses Kapitel beschreibt die wesentlichen Rendering-Mechanismen aktueller Webbrowser. Auf dieser Basis lässt sich nachvollziehen, wie sich interne Abläufe im Browser auf die Leistungsfähigkeit von Webanwendungen auswirken.

2.1.1. Der Critical Rendering Path und das DOM

Grundprinzip des Critical Rendering Path

Der Critical Rendering Path (CRP) beschreibt den Prozess, in dem ein Browser Rohdaten (HTML, CSS und JavaScript) in eine visuelle Darstellung konvertiert. Diese Verarbeitung erfolgt in einer fest vorgegebenen Pipeline (siehe Abb. 2.1), wobei das Parsing (Erstellung strukturierter Objektmodelle) vom eigentlichen Rendering (visuelle Ausgabe) zu unterscheiden ist. Während der Browser das HTML-Dokument bereits inkrementell in das Document Object Model (DOM) umwandelt, müssen Stilvorgaben (CSS) vollständig vorliegen, da sich Regeln gegenseitig überschreiben können. Das daraus resultierende CSS Object Model (CSSOM) ist daher *render-blocking* [15]. Aus der Kombination von DOM und CSSOM erzeugt der Browser den Render Tree, welcher ausschließlich visuell relevante Knoten inklusive ihrer berechneten Stile umfasst. Auf Basis dieses Modells folgen die Phasen Layout, Paint und Compositing, welche die geometrische Berechnung und die Rasterung der Pixel übernehmen [5, 15]. Ein wesentliches Merkmal dieser Pipeline ist, dass ihre Ausführung überwiegend auf dem Main Thread stattfindet [4]. Da dieser sowohl für die Verarbeitung von JavaScript als auch für die Rendering-Phasen zuständig ist, erfolgt die Abarbeitung sequenziell, was bei rechenintensiven Aufgaben zu Blockierungen der Benutzeroberfläche führen kann (vgl. Abb. 2.1) [4].

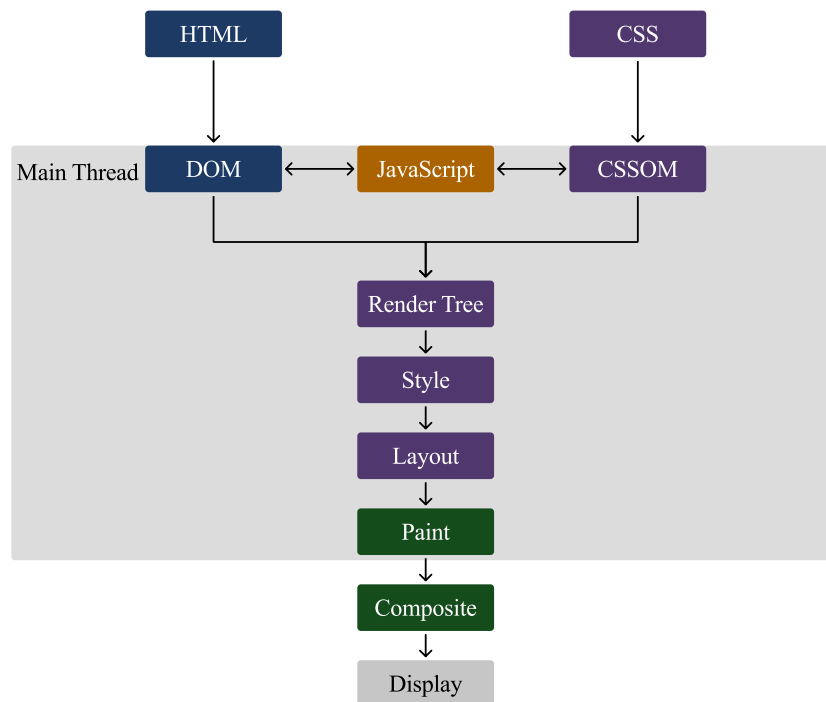


Abb. 2.1.: Phasenmodell des Critical Rendering Paths zur Verdeutlichung der Main-Thread-Auslastung (Eigene Darstellung in Anlehnung an [16]).

DOM als zentrale Repräsentation der UI

Das DOM stellt die Inhalte hierarchisch in Form eines Baumes dar. Durch diese programmierbare Schnittstelle erhalten Skriptsprachen Zugriff auf die Objekte des Dokuments und können so Struktur und Inhalte flexibel verändern. Eltern-Kind-Beziehungen im DOM-Baum übersetzen die Verschachtelung des Quelltexts und bilden so die Grundlage für die Interaktivität moderner Webanwendungen [17]. Jede dieser Änderungen am DOM kann dabei Rendering-Prozesse auslösen (vgl. Abschnitt 2.1.1).

Sichtbarkeit im Render Tree und Layout-Abhängigkeiten

Bei der Modellbildung im Render Tree ist insbesondere zwischen `display: none` (nicht im Render Tree enthalten) und `visibility: hidden` (Elemente sind im Render Tree enthalten, belegen Layoutplatz, bleiben jedoch unsichtbar) zu unterscheiden [18]. Abbildung 2.2 veranschaulicht diesen zentralen Unterschied zwischen logischer Struktur und visueller Repräsentation im Rahmen des Critical Rendering Paths. Während des Layout-Schritts wandelt der Browser die Knoten des Render Trees in konkrete Koordinaten um und bestimmt für jedes sichtbare Element innerhalb des Viewports, welche Ausmaße und Positionen es im Box-Modell einnimmt [18].

2. Theoretische Grundlagen

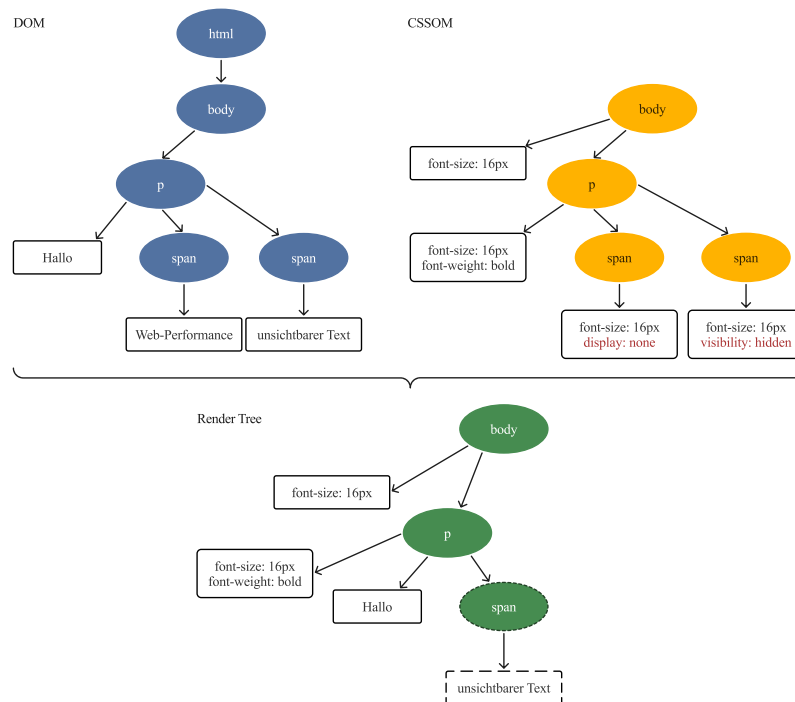


Abb. 2.2.: Strukturelle Gegenüberstellung von DOM, CSSOM und resultierendem Render Tree zur Veranschaulichung layoutrelevanter CSS-Eigenschaften. (Eigene Darstellung in Anlehnung an [18]).

Reflow, Repaint und Compositing: Performance-Kosten im Detail

Je nach Art einer visuellen Änderung durchläuft der Browser die Rendering-Pipeline laut [5] in drei unterschiedlichen Szenarien, die sich in ihrem Rechenaufwand stark unterscheiden. Eine Layout-Neuberechnung (Reflow) tritt auf, wenn sich geometrisch relevante Eigenschaften eines Elements ändern und der Browser dadurch die betroffenen und abhängigen Elemente im Layout erneut berechnen muss. Da Änderungen an einzelnen Elementen Auswirkungen auf die gesamte Hierarchie haben können, ist dieser Schritt besonders rechenintensiv. Handelt es sich um rein visuelle Anpassungen (z. B. Farbwechsel), führen diese lediglich zu einem Repaint, bei dem der Browser die betroffenen Pixelbereiche neu füllt, ohne die Geometrie erneut zu berechnen. Den effizientesten Weg stellt das Compositing dar, bei dem der Browser ausschließlich existierende Bildebenen (Layers) neu anordnet. Da dies oft auf einem eigenen Thread erfolgt, laufen insbesondere Animationen auf dieser Stufe besonders flüssig. Eine besondere Leistungsproblematik entsteht durch das sogenannte *Layout Thrashing* [19]. Dieses tritt auf, wenn JavaScript-Code Layout-Informationen abfragt (z. B. via `offsetWidth`), nachdem zuvor Stile verändert wurden [19]. Da der Browser für die Abfrage einen aktuellen Wert liefern muss, erzwingt er eine sofortige, synchrone Neuberechnung (*forced synchronous layout*), was zu einer ineffizienten Nutzung des Main Threads führt [19].

2.1.2. Herausforderungen durch hochfrequente Updates (Main-Thread-Blocking)

Frame-Budget und Main-Thread-Überlastung

Damit eine Benutzeroberfläche flüssig wirkt, orientieren sich Browser nach [5] an einer Bildwiederholrate von etwa 60 Bildern pro Sekunde. Daraus ergibt sich ein Zeitfenster von

$$t_{\text{Frame}} = \frac{1000 \text{ ms}}{60} \approx 16,7 \text{ ms} \quad (2.1)$$

pro Frame. In der Praxis ist dieses Zeitfenster sogar noch knapper, da der Browser einen Teil dieser Zeit für interne Prozesse beansprucht. Letztlich verbleiben für die eigentliche Anwendungslogik oft nur etwa 10 ms. Innerhalb dieser kurzen Zeitspanne durchläuft der Browser die Rendering-Pipeline, die je nach Art der Änderung Prozesse wie die JavaScript-Ausführung, die Stil- und Layoutberechnung sowie das abschließende Painting und Compositing umfasst. Da diese Verarbeitungsschritte überwiegend sequenziell auf dem Main Thread ablaufen, führt jede Verzögerung innerhalb eines Einzelschritts zu einer Blockade des gesamten Renderprozesses [4]. Ergänzend definiert die Long Tasks API des World Wide Web Consortium (W3C) eine 50 ms-Schwelle zur Identifikation blockierender Prozesse [14]. Eine Aufgabe im Main Thread gilt als *Long Task*, sobald ihre ununterbrochene Ausführungsdauer diesen Wert übersteigt.

Auswirkungen auf Responsiveness und Benutzererfahrung

Ein häufiges Problem ist *Input Delay*: Dabei verarbeitet der Browser Benutzereingaben wie Klicks oder Tastatureingaben nicht sofort, sondern erst nach Abschluss laufender Aufgaben, was zu einer spürbaren Verzögerung zwischen Eingabe und visueller Rückmeldung führt [2, 14]. Als Maß für die Bewertung der Responsivität über die gesamte Besuchsdauer hinweg dient die Kennzahl INP (ausführliche Definition in Abschnitt 2.4.1). Ein weiteres Phänomen ist das sogenannte Jank [5]. Dieses entsteht immer dann, wenn die Berechnung eines Frames länger dauert als das in Abschnitt 2.1.2 definierte Zeitfenster. Da der Browser in diesem Fall Frames überspringen muss, wirken Bewegungsabläufe wie Scrollen oder Animationen für den Nutzer stockend.

Bedeutung für moderne UI-Frameworks (Synthese)

Die in der Rendering-Pipeline entstehenden Kosten, insbesondere durch Layout- und Paint-Vorgänge (Reflows/Repaints), bilden die Grundlage für die Architektur moderner UI-Frameworks. Ziel dieser Frameworks ist es, die Anzahl direkter Eingriffe in das DOM zu minimieren. Anstatt jede Änderung sofort umzusetzen, bündelt das Framework Aktualisierungen. So ist sichergestellt, dass der Browser nur die tatsächlich veränderten Bereiche neu berechnen muss [6]. In der Praxis lassen sich dabei zwei unterschiedliche Strategien unterscheiden: Bibliotheken mit Virtual-DOM-Ansatz und compilerbasierte Frameworks. Diese grundlegenden Herangehensweisen werden in den folgenden Abschnitten detailliert analysiert.

2.2. Architektur von React

Dieser Abschnitt erläutert, wie React durch ein Virtual DOM (VDOM), Reconciliation und ergänzende Mechanismen diese Performance-Kosten adressiert.

2.2.1. Virtual DOM und Reconciliation

Das VDOM ist eine Zwischenschicht im Speicher, die die Benutzeroberfläche mittels JavaScript-Objekten (React-Elementen) nachbildet. Wenn sich der Zustand (State) ändert, generiert React eine neue, virtuelle Darstellung des betroffenen React-Elementbaums. Durch das Bündeln mehrerer Änderungen (Batching) gibt React diese gesammelt an das reale DOM weiter, um die Anzahl teurer Layout-Neuberechnungen im Browser zu reduzieren [6]. Um die notwendigen Aktualisierungen zu identifizieren, nutzt React den Prozess der Reconciliation. Ein zentraler Bestandteil dieses Prozesses ist der Diffing-Algorithmus, ein rekursives Verfahren zum Vergleich der virtuellen Baumstrukturen. Während der Vergleich zweier beliebiger Baumstrukturen theoretisch eine Komplexität von $O(n^3)$ aufweist, reduziert React diesen Aufwand durch heuristische Annahmen (z. B. Elementtypen und Keys) in der Praxis auf annähernd $O(n)$ [20]. Da Zustandsänderungen standardmäßig Re-Renders entlang der nachgelagerten Teilbäume auslösen, steigt der Rechenaufwand dennoch direkt mit der Tiefe und Breite der betroffenen Komponentenstruktur [6].

2.2.2. Der React Compiler (ab React 19): Automatisierte Memoisierung vs. manuelles Opt-in

Funktionsweise des React Compilers

Um unnötige Re-Renders zu vermeiden, mussten Entwickler bisher manuell mittels `React.memo` oder Hooks wie `useMemo` und `useCallback` memoisieren [6]. Diese Form der manuellen Memoisierung ist jedoch fehleranfällig und erhöht den kognitiven Aufwand während der Entwicklung [21]. Der React Compiler verlagert diese Aufgaben in den Build-Prozess. Er untersucht den Code statisch mittels einer internen Zwischendarstellung (Intermediate Representation), analysiert den Datenfluss und fügt automatisch Optimierungen hinzu, die eine stabile Referenzgleichheit gewährleisten [8]. Im Gegensatz zu Svelte, welches Komponenten in DOM-nahen Code umwandelt, optimiert der React Compiler lediglich bestehende Komponenten durch intelligentes Caching, ohne das Laufzeitmodell grundlegend zu verändern [6].

Potenziale und Grenzen im Kontext der Skalierung

Die Automatisierung reduziert den Umfang an Boilerplate-Code und entlastet Entwickler von der Pflege von Abhängigkeitslisten (Dependency-Arrays) [21]. Da der Compiler ausschließlich bestehende Komponenten memoisiert, bleiben die grundlegenden architekturbedingten Kosten von VDOM-Diffing und Reconciliation (Abschnitt 2.2.1) sowie die damit verbundenen Heap-Allokationen jedoch bestehen [6, 22].

2.2.3. Fiber-Engine: Scheduling und Rendering

Um synchron-blockierende Updates im Komponentenbaum zu verhindern, überarbeitete React die interne Verarbeitung grundlegend [23].

Aufbau der Fiber-Architektur

Fiber stellt eine interne Struktur zur Organisation und Planung von Arbeitsschritten (Scheduling) dar, wobei ein einzelner Fiber die kleinste unterbrechbare Arbeitseinheit repräsentiert [23]. Hierzu überführt Fiber den UI-Baum in eine mit expliziten Zeigern (Kind, Geschwister, Elternteil) verkettete Struktur, wodurch React laufende Prozesse bei Bedarf verwerfen, unterbrechen, den aktuellen Stand zwischenspeichern und die Ausführung ohne Kontextverlust später fortsetzen kann. Da Fiber den Call Stack lediglich nachbildet, bleibt die $O(n)$ -Komplexität beim Diffing unverändert [20, 23]. Um diese Berechnungen ohne sofortige Auswirkung auf die Anzeige durchzuführen, nutzt die Architektur das Prinzip des *Double Buffering*, bei dem ein im Hintergrund bearbeiteter Baum (*work-in-progress*) erst nach Abschluss den sichtbaren Baum (*current*) ersetzt, wie in Abb. 2.3 veranschaulicht [24]. Während Fiber-Knoten persistent sind und wiederverwendet werden, entstehen React-Elemente bei jedem Render-Vorgang neu [25]. Die JavaScript-Engine V8 verwaltet diese kurzlebigen Objekte effizient im sogenannten *jungen Speicherbereich* (Young Generation). Um die Auswirkungen auf den Main Thread bei der regelmäßigen Garbage Collection (GC) so gering wie möglich zu halten, lagert V8 diese Prozesse teilweise auf parallele Hilfs-Threads aus [26]. Wenn jedoch eine sehr hohe Anzahl an UI-Elementen gleichzeitig gerendert wird, summieren sich diese kleinen Rechen- und Speicheraufwände. Dadurch entsteht ein spürbarer Overhead, was die Skalierbarkeit im Vergleich zu Ansätzen ohne VDOM einschränken kann [7].

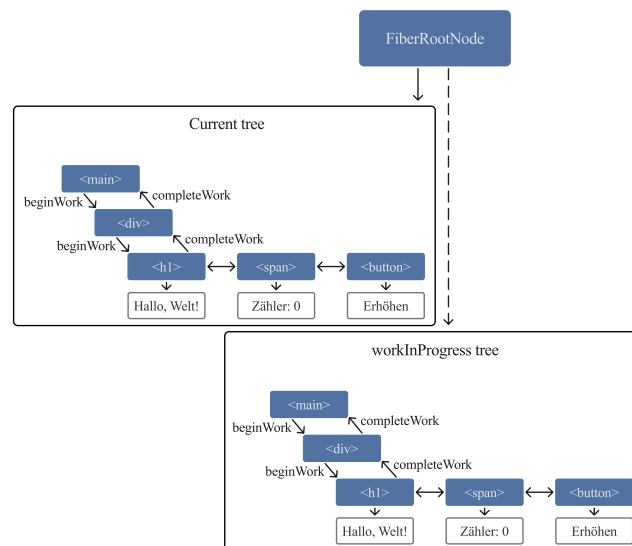


Abb. 2.3.: Architekturmodell der React Fiber-Engine: Double Buffering und Umschalten des FiberRootNode auf den Work-in-Progress Tree während der Commit-Phase. (Eigene Darstellung in Anlehnung an [6]).

Scheduling und Priorisierung von Updates

Um den Main Thread nicht zu blockieren, zerlegt das System anstehende Aufgaben per *Time Slicing* in kleinere Arbeitsschritte. Über Render Lanes (Bitmasken) priorisiert das System Aufgaben nach Dringlichkeit: Nutzerinteraktionen (z. B. Klicks) erhalten stets Vorrang, Hintergrundprozesse laufen nachrangig [6]. Der Scheduler nutzt einen kooperativen Yielding-Ansatz: Nach der Bearbeitung einzelner Fiber-Knoten prüft der Scheduler, ob ein internes Zeitbudget (5 ms) überschritten ist; ist dies der Fall, gibt er die Kontrolle an den Browser zurück [27]. Übersteigt das Aufkommen an Updates die Kapazität des Schedulers, führt dieser Verarbeitungsstau zu verzögerten Render-Vorgängen und dazu, dass das Zeitfenster pro Frame trotz Priorisierung nicht mehr eingehalten werden kann [28]. Neben der Kompensation des Reconciliation-Overheads erfüllt das *Time Slicing* eine weitere Funktion: Der kooperative Yielding-Ansatz begrenzt die maximale ununterbrochene Blockierungsdauer des Main Threads [23, 29]. Im Gegensatz dazu verfügt Svelte über kein integriertes *Time Slicing* [23, 30]. Stattdessen fasst das Effect-Batching (Abschnitt 2.3.3) ausstehende Zustandsänderungen zusammen und arbeitet sie gebündelt im darauffolgenden Microtask ab [31]. Eine Verteilung noch ausstehender Arbeit auf nachfolgende Frames ist in dieser Architektur nicht vorgesehen [23, 30].

Concurrent Rendering

Aufbauend auf der Fiber-Struktur und der Priorisierung ermöglicht Concurrent Rendering die Vorbereitung neuer UI-Zustände im Hintergrund, ohne die sichtbare Oberfläche direkt zu verändern [29]. Für eine hohe Reaktionsfähigkeit gliedert sich der Ablauf in zwei Phasen [32]:

1. **Render-Phase (asynchron):** Das System identifiziert mittels Diffing die erforderlichen Änderungen. Dieser Prozess ist unterbrechbar.
2. **Commit-Phase (synchron):** React nimmt die Änderungen am realen DOM vor. Dieser Schritt ist stets synchron und nicht unterbrechbar [32].

Die Dauer der Commit-Phase hängt direkt von der Anzahl der zu aktualisierenden DOM-Elemente ab [6]. Bei hoher UI-Dichte kann diese Phase das Frame-Budget ununterbrechbar belegen, was zu Jank führt und die Interaktionslatenz (INP) unmittelbar verlängert [33].

2.3. Architektur von Svelte

Während der in Abschnitt 2.2 untersuchte Ansatz von React stark auf eine komplexe Laufzeitumgebung und den Vergleich eines VDOM setzt, wählt Svelte einen anderen Ansatz. Statt Änderungen erst im Browser zur Laufzeit zu berechnen, verlagert Svelte diese Arbeit in den Build-Prozess [34].

2.3.1. Compilerbasierter Ansatz

Beim Build-Prozess überführt der Compiler den deklarativen Code in einen abstrakten Syntaxbaum (AST) und generiert daraus spezifische Code-Fragmente für das Erstellen, Aktualisieren und Entfernen einzelner DOM-Knoten [7, 34]. Welche dieser Fragmente bei einer Datenänderungen ausgeführt werden, bestimmt in Svelte 5 die Signal-Architektur zur Laufzeit (vgl. Abschnitt 2.3.2),

sodass das System ausschließlich die unmittelbar betroffenen Operationen ausführt. Durch den Verzicht auf ein VDOM und die damit verbundenen Reconciliation-Prozesse reduziert sich die Rechenlast auf dem Main Thread sowie der Speicherbedarf erheblich [7, 12]. Dies führt zu weniger Objektzuweisungen im Heap und entlastet die Garbage Collection, wodurch pro Frame mehr Zeit für eine flüssige Darstellung zur Verfügung steht [35].

2.3.2. Reaktivität und Update-Mechanismen

Im Gegensatz zum klassischen Svelte-Compiler, der Abhängigkeiten ausschließlich statisch zur Kompilierzeit ermittelt hat, setzt Svelte 5 auf das Modell der Signals [9]. Entwickler nutzen hierzu spezifische Compiler-Anweisungen, sogenannte Runes (z. B. `$state` und `$derived`). Während Runes die syntaktische Schnittstelle bilden, identifizieren Signals zur Laufzeit automatisch die Abhängigkeiten zwischen den Werten [36].

Mechanismus der feingranularen Reaktivität

Die Signal-Architektur ermöglicht eine feingranulare Reaktivität [9, 37], bei der die gesamte Verarbeitung, von der Erkennung einer Zustandsänderung über die Auswertung der betroffenen Abhängigkeiten bis zur gezielten DOM-Aktualisierung, auf die jeweils betroffenen Knoten begrenzt bleibt [9, 37]. Zustandsänderungen lassen sich so auch bei Arrays aus primitiven Datentypen genau nachverfolgen, ohne die ursprüngliche Datenstruktur anzupassen [38]. Im Vergleich zur Reconciliation in React, die oft mehrere Abgleichsschritte über Teilbäume erfordert, reduziert dieser Ansatz den Aufwand pro Interaktion meist auf eine einzige gezielte Operation [11].

Speichermodell und Overhead von Runes

Intern bildet Svelte Signals als einfache JavaScript-Objekte (POJOs) ab, die sowohl bei der Erstellung als auch beim Zugriff deutlich ressourcensparender sind als komplexe Klasseninstanzen [39]. Im Unterschied zu den baumartigen Fiber-Strukturen von React (Abschnitt 2.2.3) schafft dieser compilerbasierte Ansatz die strukturellen Voraussetzungen für eine effizientere Skalierung, insbesondere bei großen Mengen an UI-Elementen [11]. Der Overhead pro Signal ist zwar minimal, doch bei jedem Update muss das System jede reaktive Abhängigkeit einzeln auswerten (Abschnitt 2.3.2). Bei gleichzeitiger Aktualisierung aller n Elemente wächst der Gesamtaufwand ebenfalls mit $O(n)$ – dieselbe Komplexitätsklasse wie die Reconciliation in React (Abschnitt 2.2.1), allerdings mit einem geringeren Aufwand pro Element. Ab welchem n dieser Unterschied praktisch vernachlässigbar wird, lässt sich theoretisch nicht bestimmen.

Vermeidung von Render-Flaschenhälsen

Die feingranulare Signal-Architektur reduziert die CPU-Belastung deutlich, da die Implementierung gezielt darauf ausgelegt ist, ineffiziente Ausführungen in der JavaScript-Engine (V8) zu vermeiden und besonders häufig genutzte Codebereiche (*Hot Paths*) zu optimieren [39]. In Kombination mit sehr schnellen Ausführungszeiten und nur minimalen Eingriffen ins DOM sinkt dadurch die Wahrscheinlichkeit für Long Tasks (Abschnitt 2.1.2). Daraus lässt sich ableiten, dass Svelte selbst

unter hoher Last kein aufwendiges Laufzeit-Scheduling wie etwa das Time Slicing in React benötigt, da die einzelnen Verarbeitungsschritte von vornherein klein und effizient gehalten sind.

2.3.3. Effizienz der imperativen DOM-Updates

Ausführung imperativer DOM-Befehle

Zur Erstellung von DOM-Strukturen nutzt der Compiler Verfahren wie das Klonen von `<template>`-Elementen oder den schrittweisen programmatischen Aufbau [40]. Die Darstellung von Template-Ausdrücken und deren Logik basiert intern auf Effekten. Diese sorgen automatisiert dafür, dass Datenänderungen unmittelbar zu einer Anpassung des DOM führen. Für Spezialfälle, in denen bestimmte Operationen noch vor der eigentlichen DOM-Aktualisierung erfolgen müssen, stellt Svelte zusätzlich `effect.pre` bereit [31].

Effect-Batching und Microtasks

Ändert sich der Zustand in einer Svelte-Anwendung, passt Svelte das DOM nicht sofort nach jeder einzelnen Änderung an [30]. Stattdessen bündelt Svelte Zustandsänderungen per Microtask: Das Framework fasst mehrere gleichzeitige Änderungen zusammen, bevor DOM-Anpassungen und Effects folgen [31]. Dadurch lässt sich das in Abschnitt 2.1.1 beschriebene Layout Thrashing wirksam vermeiden.

Theoretische Grenzen (JS-zu-DOM-Flaschenhals)

Trotz dieser Optimierungen bleibt Svelte an die grundlegenden Grenzen der Browser-Architektur gebunden. Zwar sind die JavaScript-Engine und die Rendering-Engine eigenständige Komponenten, sie arbeiten jedoch über klar definierte Schnittstellen wie das DOM und die Web-IDL-Bindings eng zusammen [41]. Jeder Zugriff auf diese Verbindung zwischen JavaScript und der Rendering-Schicht bringt dabei einen systembedingten Overhead mit sich [42]. Bei einer extremen kombinierten Last aus UI-Dichte und Update-Frequenz können daher auch unter Svelte blockierende Long Tasks und Interaktionslatenzen auftreten, da sich die Kosten für wiederholte DOM-Zugriffe summieren (Abschnitt 2.1.2).

2.4. Vergleichbare Metriken und Performancebegriffe

2.4.1. Interaktionslatenz und Antwortverhalten (INP)

Definition und Phasen der Metrik

Die Metrik Interaction to Next Paint (INP) misst, wie schnell eine Webseite auf Nutzerinteraktionen über die gesamte Nutzungsdauer hinweg reagiert [2]. Die technische Zusammensetzung der Metrik sowie die zeitliche Abfolge der einzelnen Phasen sind in Abb. 2.4 dargestellt. Aus technischer Sicht lässt sich dieser Messwert in drei Phasen unterteilen [2]:

- **Input Delay (Eingabeverzögerung):** Zeitspanne vom Beginn der Nutzerinteraktion bis zur Verarbeitung des ersten Event-Handlers, verursacht durch Main-Thread-Blockaden (vgl. Abschnitt 2.1.2).
- **Processing Time (Verarbeitungsdauer):** Hierbei handelt es sich um die Zeit zur vollständigen Ausführung aller Callbacks der Event-Handler und der damit verbundenen Framework-Logik.
- **Presentation Delay (Darstellungsverzögerung):** Diese Phase misst die Zeit nach Abschluss der Skriptausführung bis zu dem Punkt, in dem der Browser die Änderungen sichtbar auf dem Bildschirm darstellt.

Google definiert auf Basis dieser Metrik drei Leistungskategorien: INP-Werte bis einschließlich 200 ms gelten als *Good*; Werte zwischen 200 ms und 500 ms als *Needs Improvement*; Werte oberhalb von 500 ms als *Poor* [2].

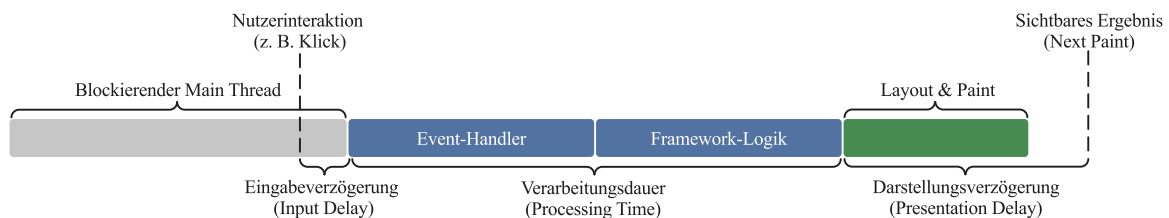


Abb. 2.4.: Zeitliches Phasenmodell der Metrik Interaction to Next Paint (INP) vom Auslösen des Events bis zum Rendering. (Eigene Darstellung in Anlehnung an [2]).

Messung aller Interaktionen und Abgrenzung zu FID

Im Gegensatz zum *First Input Delay* (FID), welcher nur die erste Interaktion erfasst, bewertet INP die Stabilität der Performance während der gesamten Sitzung. Der finale INP-Wert entspricht in der Regel der Interaktion mit der höchsten Gesamtverzögerung. Damit einzelne Ausreißer das Ergebnis nicht verzerren, ignoriert die Metrik bei jeweils 50 Interaktionen den höchsten Wert. Auf diese Weise entsteht ein realistischeres Bild der tatsächlichen, stabilen Performance [2].

Relevanz für die Messung der UI-Skalierung

INP dient als zentrale Metrik, um den in Abschnitt 2.5.2 beschriebenen Tipping Point empirisch zu identifizieren. Sie macht messbar, ab welcher kombinierten Last aus UI-Dichte und Update-Frequenz die *Processing Time* oder das *Presentation Delay* das verfügbare Frame-Budget überschreitet und zu spürbaren Verzögerungen führt.

2.4.2. Analyse der Speichereffizienz (Heap-Snapshots)

Funktionsweise von Heap-Snapshots

Ein Heap-Snapshot stellt eine detaillierte Momentaufnahme des Objektgraphen innerhalb des Speichers der JavaScript-Laufzeitumgebung dar [43]. Er dient dazu, die Speicherverteilung zu

untersuchen, mögliche Speicherlecks zu erkennen oder Objekte zu identifizieren, die übermäßig viele Ressourcen im System beanspruchen [44].

Shallow Size vs. Retained Size

Zur genauen Messung des Speicherverbrauchs sind zwei zentrale Metriken zu unterscheiden: Die *Shallow Size* umfasst den Speicherplatz, den ein Objekt für sich selbst beansprucht, ohne dabei den Speicher der referenzierten Objekte einzubeziehen. Die *Retained Size* hingegen beschreibt, wie viel Speicher freigegeben werden kann, wenn ein bestimmtes Objekt aus dem Speicher entfernt wird. Sie umfasst dabei nicht nur das Objekt selbst, sondern auch alle abhängigen Elemente, die ausschließlich über dieses Objekt referenziert wurden und nach dessen Entfernung keine Verbindung mehr zu den Garbage-Collector-Wurzeln haben [43].

Analyse des Objekt-Graphen: Distance und Dominators

Die *Distance* gibt die Anzahl der Schritte von der Wurzel des Garbage Collectors (GC Root) zum Objekt an. Weisen Objekte gleichen Typs ungewöhnlich große Distanzen auf, deutet das oft auf komplexe und speicherintensive Referenzketten hin [43]. Strukturell basiert die Retained Size im Wesentlichen auf dem Konzept der *Dominators* (Dominatoren). Dabei gilt ein Objekt d als Dominator eines anderen Objekts n , wenn jeder mögliche Pfad vom Root-Knoten zu n zwangsläufig über d führt [45]. Innerhalb des Speicher-Graphen lassen sich Dominator-Beziehungen als Baumstruktur darstellen [43]. Diese Struktur ist besonders wichtig für die Retained Size, da beim Entfernen eines Objekts auch alle davon abhängigen Objekte im zugehörigen Zweig für die Garbage Collection freigegeben werden. Diese baumartige Hierarchie sowie das Zusammenspiel der Metriken Distance, Shallow Size und Retained Size werden in Abb. 2.5 zusammenfassend visualisiert. Für den empirischen Framework-Vergleich ist die Retained Size die geeignetste Kennzahl, da sie den internen Framework-Overhead vom Anwendungszustand trennt. Die methodische Umsetzung dieser Messung erläutert Abschnitt 4.3.5.

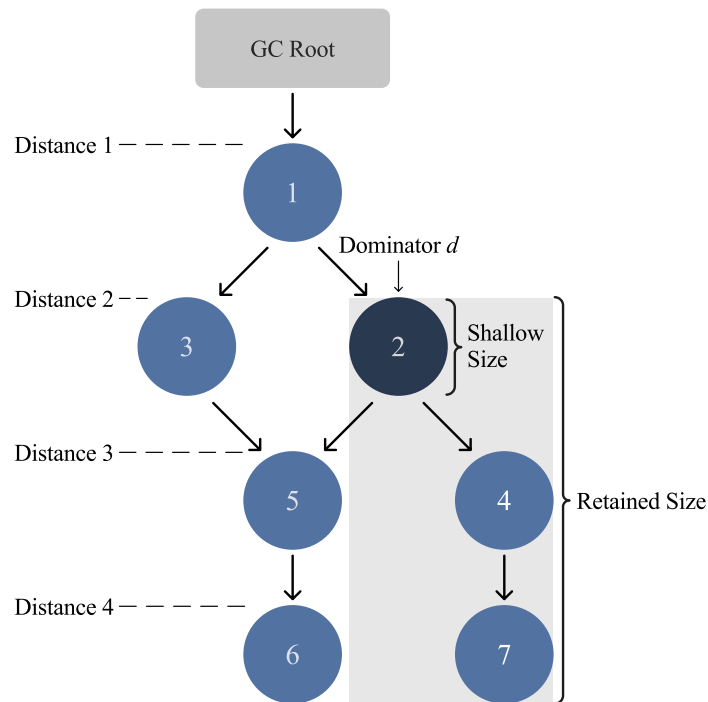


Abb. 2.5.: Modell eines Speicher-Graphen zur Veranschaulichung der Dominator-Beziehungen sowie der Differenzierung zwischen Shallow und Retained Size. (Eigene Darstellung in Anlehnung an [43]).

2.4.3. Scripting- vs. Rendering-Zeit im Browser-Profil

Kategorien im Performance-Trace

Werkzeuge wie das Chrome DevTools Performance-Panel visualisieren die Main-Thread-Auslastung in Kategorien [46]:

- **Scripting (Gelb):** Diese Kategorie beschreibt die gesamte Zeit, die die JavaScript-Engine für das Parsen, Kompilieren und Ausführen von JavaScript-Code benötigt [46, 47].
- **Rendering (Lila):** Die *Rendering-Zeit* umfasst alle Phasen, in denen der Browser nach Zustandsänderungen die Stile der Elemente neu berechnet (*Recalculate Style*) und anschließend die Geometrie des Dokumentenbaums bestimmt (*Layout*) [46].
- **Painting:** Diese Phase beschreibt den Prozess, bei dem der Browser die grafischen Zeichenbefehle in Pixel umwandelt, bevor der Browser diese im nächsten Schritt final zur Bildschirmausgabe zusammensetzt [48].

Bedeutung für die Tipping-Point-Analyse

Die getrennte Analyse dieser Kategorien hilft dabei zu erkennen, wo Leistungsengpässe entstehen. Während die *Scripting-Zeit* den internen Aufwand der Framework-Logik (z. B. Reconciliation oder

Signal-Auswertung) widerspiegelt, zeigt die Rendering-Zeit die Last der Browser-Engine beim Umsetzen der DOM-Änderungen [46, 48]. Dies ist essenziell für die Untersuchung der theoretischen Skalierungsgrenzen (Abschnitt 2.3.3).

2.5. Theoretische Vergleichsmodelle und Hypothesen

Dieses Kapitel überführt die architektonischen Merkmale aus Abschnitt 2.2 und Abschnitt 2.3 in ein vergleichendes Modell. Ziel ist es, die Auswirkungen von UI-Dichte (n) und Update-Frequenz (f) auf die Systemleistung theoretisch herzuleiten.

2.5.1. Synthese der Abstraktionskosten

Der Vergleich zwischen dem VDOM-basierten Ansatz und der direkten DOM-Manipulation zeigt grundlegende Unterschiede in der Ressourcenallokation. Während der in Abschnitt 2.2.1 beschriebene Diffing-Prozess von React bei Zustandsänderungen einen algorithmischen CPU-Overhead zur Laufzeit verursacht, entfällt dieser Reconciliation-Overhead bei der Compiler-Architektur von Svelte (Abschnitt 2.3.1) fast vollständig. Dies betrifft auch den Speicherbedarf: Durch den VDOM-Ansatz werden fortlaufend virtuelle Knoten erzeugt, wodurch die Belastung der Garbage Collection linear mit der Anzahl der Elemente n ansteigt [23, 49]. Svelte umgeht diese Zwischenstrukturen und entlastet so die JavaScript-Laufzeitumgebung (Abschnitt 2.3.1). Unabhängig davon sind beide Architekturen jedoch auf die native DOM-Schnittstelle angewiesen, was bei hoher Skalierung zu einem unvermeidbaren Flaschenhals im Main Thread führt (Abschnitt 2.3.3).

2.5.2. Komplexität und Skalierung (n vs. f)

Umfang der Neuberechnungen bei Zustandsänderungen

In der React-Architektur skaliert der Berechnungsaufwand bei Zustandsänderungen unabhängig von tatsächlichen DOM-Veränderungen direkt proportional zur Anzahl der Knoten im betroffenen Teilbaum (Abschnitt 2.2.1). Im Gegensatz dazu entkoppelt das feingranulare Reaktivitätsmodell von Svelte (Abschnitt 2.3.2) den Aufwand von der Baumgröße. Da keine Diffing-Schritte über nicht betroffene Komponenten nötig sind, bestimmt sich die Ausführungszeit primär durch die Anzahl der tatsächlich veränderten Abhängigkeiten (Abschnitt 2.3.2).

Der Multiplikator-Effekt und der Tipping Point

Die Kombination aus hoher Frequenz f und hoher Dichte n führt zu einer gegenseitigen Verstärkung der Last. Da React bei jeder Änderung den entsprechenden Teilbaum erneut überprüft (Abschnitt 2.2.1), kann die CPU-Auslastung bei hoher UI-Dichte und gleichzeitig hoher Aktualisierungsrate das Frame-Budget von 16,7 ms (Abschnitt 2.1.2) vergleichsweise schnell ausschöpfen. Svelte begrenzt diesen Effekt durch direkte Operationen am relevanten Knoten (Abschnitt 2.3.2). Aus den architektonischen Skalierungsunterschieden lässt sich ein theoretischer Tipping Point ableiten. Dieser bezeichnet den Übergang, an dem die aufsummierte Last den Main Thread durch Long Tasks blockiert (Abschnitt 2.1.2). Bedingt durch baumgrößenabhängige Re-Render-Zyklen und die

synchrone, nicht unterbrechbare Commit-Phase (Abschnitt 2.2.3) ist davon auszugehen, dass React diesen Punkt früher erreicht als Svelte. Bei vollständiger Aktualisierung aller n Elemente pro Tick wächst auch der Aufwand der feingranularen Reaktivität linear mit n (Abschnitt 2.3.2). Ab einem bestimmten Sättigungspunkt kann daher auch Svelte den Main Thread durch Long Tasks blockieren (Abschnitt 2.3.3). Da Sveltes Effect-Batching die gesamte Verarbeitung synchron innerhalb desselben Microtask-Zyklus abschließt (Abschnitt 2.3.3), während Reacts Time Slicing Arbeitsschritte über mehrere Frames verteilen kann (Abschnitt 2.2.3), ist eine Umkehr der Tipping-Point-Rangfolge bei extremer kombinierter Last theoretisch nicht auszuschließen.

2.5.3. Erwartetes Skalierungsverhalten

Aus den in Abschnitt 2.5.2 beschriebenen Skalierungsunterschieden lässt sich erwarten, dass die Scripting-Zeit beider Frameworks unterschiedlich stark mit n wächst. Die Reconciliation durchläuft dabei stets den gesamten betroffenen Teilbaum mit $O(n)$, unabhängig davon, wie viele Knoten sich tatsächlich verändert haben (Abschnitt 2.2.1). Hinzu kommt der in Abschnitt 2.2.3 beschriebene GC-Overhead durch kurzlebige React-Elemente, der konzeptionell linear mit n wächst (Abschnitt 2.5.1). Svelte hingegen begrenzt den Scripting-Aufwand durch die feingranulare Reaktivität (Abschnitt 2.3.2) auf die Auswertung der tatsächlich veränderten Abhängigkeiten. Da keine Diffing-Schritte über unveränderte Komponenten anfallen (Abschnitt 2.5.2), ist davon auszugehen, dass der Aufwand nicht mit der Gesamtgröße des Komponentenbaums wächst, sondern mit der Anzahl der Änderungen. Auch wenn beide Architekturen bei vollständiger Aktualisierung mit $O(n)$ skalieren, ist der Aufwand pro Element bei Svelte theoretisch geringer, da der Abgleich des Komponentenbaums und die damit verbundenen Heap-Allokationen entfallen (Abschnitt 2.3.2). Aus diesem Unterschied ergibt sich die Annahme, dass der Scripting-Aufwand pro Tick in React stärker mit n wächst als in Svelte. Bei vollständiger Aktualisierung aller n Elemente wertet Svelte ebenfalls n Abhängigkeiten aus; der verbleibende Vorteil besteht ausschließlich im geringeren Aufwand pro Element (Abschnitt 2.3.2). Über die Skalierung der Scripting-Zeit hinaus lassen sich auch Auswirkungen auf die Interaktionslatenz (INP) ableiten. Für **React** kann Concurrent Rendering (Abschnitt 2.2.3) das Input Delay stabilisieren, indem das System rechenintensive Render-Vorgänge unterbricht und Nutzerinteraktionen priorisiert (Abschnitt 2.2.3). Ab einem bestimmten Sättigungspunkt belegt die nicht unterbrechbare, synchrone Commit-Phase den Main Thread jedoch so lange (Abschnitt 2.2.3), dass das System nachfolgende Eingaben verzögert verarbeitet. Dies kann sowohl das *Input Delay* als auch das *Presentation Delay* erhöhen (Abschnitt 2.4.1) und sich negativ auf die Interaktionslatenz (INP) auswirken. Für **Svelte** ist aufgrund des geringeren Aufwands pro Tick eine Überlastung des Main Threads erst bei höherer Last zu erwarten. Allerdings schließt Sveltes Effect-Batching (Abschnitt 2.3.3) die gesamte Aktualisierung synchron ab, ohne Arbeit auf Folge-Frames zu verteilen (Abschnitt 2.2.3). Im Gegensatz dazu ermöglicht Reacts kooperatives Yielding (Abschnitt 2.2.3) eine Unterbrechung zugunsten von Nutzerinteraktionen. Daraus lässt sich ableiten, dass bei Last unterhalb des Frame-Budgets (Abschnitt 2.1.2) Reacts Scheduling die INP stabilisieren kann, auch wenn die absoluten Scripting-Kosten höher sind als bei Svelte.

2.5.4. Hypothesen

Auf Basis der architektonischen Analysen und des daraus abgeleiteten Skalierungsverhaltens (vgl. Abschnitt 2.5.3) definiert diese Arbeit für den empirischen Vergleich zwischen React 19 und Svelte 5 die folgenden vier Hypothesen:

- **H1: Svelte 5 weist bei steigender UI-Dichte eine geringere Interaktionslatenz (INP) auf als React 19; bei Last unterhalb des Frame-Budgets könnte Reacts prioritätsbasiertes Scheduling diesen Vorteil aufheben.**
- **H2: Der Scripting-Aufwand pro Tick von React 19 wächst bei steigender UI-Dichte n stärker als der von Svelte 5.**
- **H3: Der Speicherverbrauch pro einzelner Komponente (Retained Size) ist bei React 19 höher als bei Svelte 5.**
- **H4: React 19 erreicht den Tipping Point bei kombinierter Last früher als Svelte 5; bei extremer Last könnte Sveltes synchrones Effect-Batching ohne Frame-Yielding diese Rangfolge umkehren.**

3. Stand der Forschung

Dieses Kapitel gibt einen Überblick über den aktuellen Forschungsstand und baut auf den theoretischen Skalierungsmodellen aus Abschnitt 2.5 auf.

3.1. Existierende Framework-Benchmarks

Als Referenz für die Leistungsfähigkeit unter kontrollierten Bedingungen dient der *js-framework-benchmark* von Stefan Krause [13].

3.1.1. Methodik synthetischer Benchmarks

Der Benchmark misst die Performance verschiedener JavaScript-Frameworks, indem er die Ausführungs- und Rendering-Zeiten gängiger Operationen in großen, zufällig erzeugten Datentabellen erfasst. Die relevanten Daten beziehen sich auf den *Keyed-Modus*, bei dem eindeutige Identifikatoren eine präzise Zuordnung von Datenänderungen zu DOM-Elementen ermöglichen (Abschnitt 2.2.1). Die Ermittlung der Gesamtperformance erfolgt auf Basis des gewichteten geometrischen Mittels [13].

3.1.2. Empirische Performance-Profile: React 19 und Svelte 5

Die Ergebnisse zeigen deutliche Unterschiede zwischen Svelte (v5.42.1) und den React-Varianten (React v19.2.0 und React Compiler v19.0.0).

Ausführungszeit (Duration)

Beim initialen Erstellen von 1.000 Zeilen liegt Svelte mit 21,8 ms nur leicht vor React (ca. 24,0 ms) [13]. Bei komplexeren Änderungen vergrößert sich der Abstand: Das Vertauschen zweier Zeilen erfordert bei Svelte 12,8 ms, während React hierfür 88,8 ms bis 89,9 ms benötigt. Auch bei der Skalierung auf 10.000 Zeilen bleibt Svelte mit 236,6 ms effizienter als React (411,1 ms bis 447,6 ms) [13]. Im gewichteten geometrischen Mittel der Duration-Faktoren erreicht Svelte einen Wert von 1,15, während React zwischen 1,53 und 1,59 liegt [13]. Die geringen Unterschiede zwischen den React-Varianten über alle Metriken hinweg zeigen zudem, dass der React Compiler gegenüber einer manuellen Hook-Implementierung keinen messbaren Laufzeitvorteil erzielt. Dies bestätigt empirisch den in Abschnitt 2.2.2 hergeleiteten Befund, dass die grundlegenden VDOM-Kosten trotz automatisierter Memoisierung bestehen bleiben.

Speichereffizienz (Memory Allocation)

Die Analyse bestätigt die erwarteten Unterschiede im Speicher-Overhead der Fiber-Engine (vgl. Abschnitt 2.2.3). Beim Hinzufügen von 1.000 Zeilen belegt Svelte 2,88 MB Heap-Speicher, während React zwischen 4,41 MB und 4,58 MB beansprucht [13]. Auch nach mehreren Zyklen bleibt React mit 1,96 MB nahezu doppelt so speicherintensiv wie Svelte mit 1,02 MB [13].

Verhalten bei partiellen Updates

Bei Aktualisierungen jedes zehnten Elements benötigt Svelte 11,0 ms, React dagegen 13,9 ms bis 14,0 ms [13]. Dies belegt das effizientere Skalierungsverhalten der feingranularen Reaktivität gegenüber der Reconciliation (Abschnitt 2.5.2).

3.1.3. Kritische Würdigung: Die methodische Grenze

Der js-framework-benchmark liefert zwar aussagekräftige Ergebnisse zur Effizienz von DOM-Operationen und Speicherallokationen. Eine reale Dauerbelastung, bei der viele UI-Elemente gleichzeitig mit hoher Frequenz (z. B. 60 Hz) interagieren, bleibt jedoch unberücksichtigt. Zudem wird die Metrik Interaction to Next Paint (INP) nicht abgebildet, wodurch die Frage offen bleibt, ab welchem Punkt die summierte Last das Frame-Budget durch Long Tasks (Abschnitt 2.1.2) dauerhaft überschreitet.

3.2. Akademische Leistungsstudien und verwandte Arbeiten

Während Industrie-Benchmarks (vgl. Abschnitt 3.1) vor allem die reine Ausführungsgeschwindigkeit betrachten, analysiert die wissenschaftliche Forschung, welche architektonischen Faktoren diese Leistung beeinflussen.

3.2.1. Analysen zum Virtual-DOM-Overhead

Studien belegen die in Abschnitt 2.5.1 beschriebenen Abstraktionskosten. Bai (2023) identifiziert das VDOM als Flaschenhals, der durch die Erzeugung virtueller Knoten einen ca. 1,7-fach höheren Aufwand gegenüber direkten DOM-Manipulationen verursacht [49]. Putra et al. (2025) messen im Idle-Zustand für Svelte 3 einen Speicherbedarf von 7,8 MB, während React 18 14,0 MB benötigt [11]. Levlin (2020) zeigt, dass eine Änderung an einem Einzelelement bei React 16 rund 16,58 ms beansprucht, was fast das gesamte Frame-Budget (Abschnitt 2.1.2) ausschöpft, während Svelte 3 hierfür lediglich 0,11 ms benötigt [10].

3.2.2. Laufzeit- vs. Compiler-Architekturen im direkten Vergleich

Der Architekturunterschied zwischen einem Laufzeit-Framework wie React und einem Compiler-Ansatz wie Svelte zeigt sich auch in weiteren vergleichenden Metriken. Laut Ollila et al. arbeitet Svelte 3 beim reinen Scripting mindestens 50 % effizienter als React 17, da der Compiler statische Inhalte bereits zur Build-Zeit optimiert [12]. Sowohl bei Untersuchungen zur DOM-Manipulation beim

Erzeugen von Elementen als auch in Vergleichstests von Webanwendungen zeigt Svelte deutlich kürzere Lade- und initiale Rendering-Zeiten. Beim Löschen von Komponenten bleibt die Effizienz dagegen weitgehend gleich. Laut Literatur liegen die Vorteile vor allem im Kompilierungsansatz und der daraus resultierenden kleineren *Bundle Size*, wodurch weniger Daten geladen und verarbeitet werden müssen [50, 51]. Dies deckt sich mit den Messungen von Levlin (2020). In seinen Tests erreicht Svelte 3 eine Bundle-Größe von etwa 3,6 kB, während React 16 bei 6,4 kB liegt. Allerdings relativiert Levlin diese Zahlen und macht deutlich, dass VDOM-basierte Architekturen insbesondere bei umfangreichen Aktualisierungen Vorteile haben. Bei der Aktualisierung von 10.000 bereits gerenderten Elementen erzielte React 16 mit durchschnittlich 17,86 ms deutlich bessere Ergebnisse als Svelte 3, das hierfür 885,03 ms benötigte [10].

3.3. Identifikation der Forschungslücke

Trotz der vorhandenen Daten fehlen für die Bewertung moderner, hochgradig reaktiver Anwendungen zwei Grundlagen:

3.3.1. Fokus auf veraltete Framework-Generationen

Zum einen beruhen die in der Literatur aufgeführten Studien auf älteren Framework-Versionen [10–12]. Häufig kommen dabei Svelte 3 sowie React 16 oder 18 zum Einsatz [10, 11]. Aktuelle architektonische Paradigmenwechsel bleiben somit unberücksichtigt. Dazu gehören unter anderem der React Compiler (React 19), der die Memoisierung automatisiert (Abschnitt 2.2.2), sowie die auf Runes aufbauende Signal-Architektur in Svelte 5, die Reaktivität zur Laufzeit feingranular steuert (Abschnitt 2.3.2). Da Svelte 3 dieses Reaktivitätsmodell noch nicht besaß, sind Studien, die auf dieser Version basieren, für die Bewertung von Svelte 5 unter Dauerlast nur bedingt aussagekräftig.

3.3.2. Fehlende Untersuchung von kombinierter Dauerlast

Zum anderen konzentrieren sich die Testfälle in Studien und Benchmarks meist auf erste Renderzeiten oder isolierte, nacheinander ausgeführte Listenoperationen unter kontrollierten Bedingungen [10–13]. Die Auswirkungen einer kombinierten Stressbelastung, bestehend aus extrem hoher UI-Dichte zusammen mit hoher Update-Frequenz, auf den Main Thread sind bisher kaum untersucht. Wichtige Metriken wie die Interaction to Next Paint (INP) oder das Auftreten von Long Tasks und ein dauerhaftes Überschreiten des 16,7-ms-Frame-Budgets (vgl. Abschnitt 2.1.2) wurden für neuere Framework-Versionen bislang nicht umfassend berücksichtigt.

3.3.3. Ableitung für diese Arbeit

Genau an diesem Punkt setzt die vorliegende Bachelorarbeit an. Um den in Abschnitt 2.5.2 theoretisch hergeleiteten *Tipping Point* der beiden Architekturen fundiert zu bewerten, ist ein gezielter Stresstest erforderlich. Dieser muss die modernsten Framework-Generationen unter einer simulierten Dauerlast evaluieren und den direkten Einfluss auf die Responsivität (INP) sowie die Speicherallokation messbar machen.

4. Methodik und Versuchsaufbau

Basierend auf den theoretischen Grundlagen in Kapitel 2 und der in Kapitel 3 aufgezeigten Forschungslücke beschreibt dieses Kapitel den empirischen Versuchsaufbau zur Überprüfung der vier Hypothesen aus Abschnitt 2.5.4.

4.1. Definition der Test-Szenarien und Testmatrix

Für die empirische Überprüfung der Hypothesen H1 bis H4 aus Abschnitt 2.5.4 werden die beiden unabhängigen Variablen UI-Dichte n und Update-Frequenz f betrachtet. Die UI-Dichte beschreibt die Anzahl gleichzeitig gerendeter Komponenten, während die Update-Frequenz f die Häufigkeit von Datenänderungen angibt und aus dem Zeitabstand Δt zwischen zwei aufeinanderfolgenden Änderungen abgeleitet wird. Gemeinsam definieren beide Variablen den Untersuchungsbereich der Experimente, der sowohl praxisrelevante Lastszenarien als auch den theoretisch bestimmten Tipping Point (Abschnitt 2.5.2) einschließt.

4.1.1. Szenario A: Variierende UI-Dichte

Die UI-Dichte ergibt sich aus der Anzahl n der gleichzeitig im Grid dargestellten Zellen. Insgesamt sind acht Abstufungen definiert:

$$n \in \{100, 400, 900, 1600, 4000, 6000, 8000, 10\,000\} \quad (4.1)$$

Die Skalierung ist so angelegt, um sowohl den unteren Lastbereich ($n \leq 900$) als auch den Übergangsbereich ($n \geq 1600$), in dem die Abstraktionskosten beider Frameworks unterschiedlich stark wachsen, ausreichend detailliert zu erfassen. Die Stufen $n \in \{1600, 4000, 6000, 8000\}$ bilden den Bereich ab, in dem Reacts baumabhängige Reconciliation-Kosten erwartungsgemäß stärker skalieren als Sveltes feingranulare Reaktivität (Abschnitt 2.3.2). Die höchste Stufe $n = 10\,000$ stellt bei hohen Updatefrequenzen eine Extrembelastung dar, die auch Svelte an seine theoretischen Grenzen bringt (Abschnitt 2.3.3). In jedem Tick werden exakt $\lceil n \cdot 0,05 \rceil$ Zellen verändert, also 5 % der Gesamtmenge. Diese Größe stellt sicher, dass pro Frame ausreichend sichtbare Renderarbeit entsteht, um Unterschiede in der Reaktivität beider Frameworks erkennbar zu machen, ohne das gesamte Grid neu zu aktualisieren. Würden alle Zellen gleichzeitig aktualisiert, gingen die Effekte der feingranularen Signal-Auflösung (Abschnitt 2.3.2) verloren, da dann ohnehin jede Zelle neu gerendert werden müsste. Die Auswahl der pro Tick veränderten Zellen erfolgt durch eine zufällige Auswahl ohne Zurücklegen [52]. Ein Bailout-Guard stellt zudem bei jeder Änderung sicher, dass sich der neue Zellwert vom bisherigen unterscheidet (vgl. Abschnitt 5.1.3).

4.1.2. Szenario B: Variierende Update-Frequenz

Die Update-Frequenz ergibt sich aus dem Tick-Intervall Δt . Vier Stufen bilden unterschiedliche Anwendungsszenarien ab (Tabelle 4.1):

Die Stufe $\Delta t = 16 \text{ ms}$ nähert sich dem in Abschnitt 2.1.2 beschriebenen kritischen Frame-Budget von $1000 \text{ ms}/60 \approx 16,7 \text{ ms}$ an und stellt mit $\approx 62,5 \text{ Hz}$ die praktisch maximale Updaterate dar, bei der ein Framework theoretisch noch eine flüssige Darstellung gewährleisten kann. Die dazwischen liegenden Stufen (50 ms und 33 ms) dienen dazu, den Übergangsbereich zwischen geringer und kritischer Systemlast präzise zu bestimmen und die Entwicklung der in Abschnitt 2.5.1 behandelten Abstraktionskosten über den gesamten Frequenzbereich hinweg nachzuvollziehen. Die Tick-Anzahl je Frequenzstufe wird so festgelegt, dass ein Durchlauf etwa 10 s dauert (Ausnahme: $\Delta t = 16 \text{ ms}$ mit 500 Ticks und rund 8 s). Bei sehr hoher Dichte ($n \geq 8000$) kann die tatsächliche Laufzeit, insbesondere beim 16-ms-Intervall, in React deutlich über der vorgesehenen Zieldauer liegen, da der Web-Worker-Timer auf einem separaten Thread Ticks schneller erzeugt, als der Main-Thread sie verarbeiten kann (Abschnitt 4.4.3). Die primäre Performance-Metrik *scriptingMsPerTick* (Abschnitt 4.3.4) ist so normalisiert, dass sie von der tatsächlichen Gesamtlaufzeit unabhängig bleibt.

Tab. 4.1.: Frequenzstufen des Szenarios B einschließlich der jeweiligen Frequenz, der Tick-Anzahl pro Messdurchlauf sowie des betrachteten Anwendungskontexts:

Intervall Δt	Frequenz f	Ticks/Run	Betrachteter Anwendungskontext
100 ms	10 Hz	100	Seltene UI-Updates (z. B. zyklische Dashboard-Aktualisierung)
50 ms	20 Hz	200	Moderate Aktualisierungsrate (z. B. Live-Monitoring von Serverprozessen)
33 ms	$\approx 30 \text{ Hz}$	300	Hohe Updatefrequenz, Halbierung der Bildwiederholrate
16 ms	$\approx 62 \text{ Hz}$	500	Volllast nahe der 60-fps-Grenze

4.1.3. Kombinierte Testmatrix

Aus den acht Dichtestufen (Szenario A) und den vier Frequenzstufen (Szenario B), jeweils für beide Frameworks, ergibt sich die vollständige Testmatrix:

$$8 \times 4 \times 2 = 64 \text{ Szenarien} \quad (4.2)$$

Jedes Szenario wird $N = 10$ Mal wiederholt (vgl. Abschnitt 4.3.2), sodass für die Performance-Messung insgesamt 640 Benchmark-Läufe anfallen. Mit dem separat ausgeführten Speicher-Benchmark (Abschnitt 4.3.5) sowie dem INP-Benchmark (Abschnitt 4.3.1) ergibt sich eine Gesamtanzahl von 1920 Läufen bei einer geschätzten Gesamtlaufzeit von ca. sieben bis acht Stunden. Tabelle 4.2 zeigt die Testmatrix für ein einzelnes Framework. Die Zellenwerte geben die Tick-Anzahl des jeweiligen Szenarios an.

Tab. 4.2.: Testmatrix mit allen Kombinationen aus UI-Dichte (n) und Tick-Intervall (Δt) für ein Framework. Zellenwerte: Ticks pro Run.

n	Tick-Intervall Δt			
	100 ms	50 ms	33 ms	16 ms
100	100	200	300	500
400	100	200	300	500
900	100	200	300	500
1600	100	200	300	500
4000	100	200	300	500
6000	100	200	300	500
8000	100	200	300	500
10 000	100	200	300	500

4.1.4. Gemessene Metriken und Hypothesenzuordnung

In jedem Szenario werden fünf Metriken bestimmt, die den vier Hypothesen aus Abschnitt 2.5.4 zugeordnet sind:

- **INP (Interaction to Next Paint):** Entsprechend der in Abschnitt 2.4.1 definierten Metrik ermittelt INP die Reaktionsfähigkeit der Benutzeroberfläche auf Klicks unter bestehender Tick-Last. Sie prüft H1 (Svelte-INP-Vorteil bei steigender UI-Dichte; bei niedriger Last könnte Reacts Scheduling überwiegen) und H4 (früherer Tipping Point bei React; mögliche Umkehr bei extremer Last).
- **scriptingMsPerTick:** Die auf die Tick-Anzahl normierte Scripting-Zeit aus dem CDP-Performance-Trace (Abschnitt 2.4.3), d. h. die durchschnittliche JavaScript-Ausführungszeit pro Datenänderung. Sie prüft H2 (stärkeres Wachstum des Scripting-Aufwands bei React gegenüber Svelte bei steigendem n).
- **renderingMsPerTick:** Ebenfalls normierte Rendering-Zeit aus demselben CDP-Trace. Zusammen mit *scriptingMsPerTick* ermöglicht sie eine vollständige Bewertung der Main-Thread-Auslastung und ergänzt H2 um die in Abschnitt 2.1.1 beschriebenen Layout- und Style-Recalculation-Kosten.
- **JS-Heap-Delta (Δ_{Heap}):** Differenz des *jsHeapUsedSize* zwischen Ausgangszustand (nach initialem Grid-Render und erzwungener GC) und Endzustand (nach allen Ticks, erneut nach erzwungener GC). Sie prüft H3 (höherer Speicherverbrauch bei React) und macht die in Abschnitt 2.4.2 beschriebenen Unterschiede der Heap-Allokation messbar.
- **Long Tasks (*longTaskCount*, *longTaskMs*):** Anzahl und Gesamtdauer von Main-Thread-Aufgaben, die die W3C-Schwelle von 50 ms überschreiten (Abschnitt 2.1.2). Long Tasks bilden den Tipping Point unmittelbar ab, da jede Grenzüberschreitung eine messbare Blockierung des Main Threads darstellt. Diese Kennzahl dient daher als wichtigste empirische Grundlage für H4 (früherer Tipping Point bei React). Eine Division durch die Tick-Anzahl entfällt im

Unterschied zu *scriptingMsPerTick*, da es sich um einzelne Ereignisse handelt. Der absolute Wert pro Messdurchlauf ist daher aussagekräftiger (Abschnitt 4.3.1).

4.2. Testumgebung und Standardisierung

Für reproduzierbare Messungen muss die Testumgebung vollständig definiert und über alle Durchläufe hinweg unverändert bleiben. Allmähliche Änderungen bei Hardwareauslastung, Softwareeinstellungen oder Versionsständen von Abhängigkeiten stellen verdeckte Fehlerquellen dar und führen zu Leistungsabweichungen, die nicht auf die Frameworks zurückgehen.

4.2.1. Hardware-Spezifikation

Alle Messungen werden auf derselben Workstation vorgenommen. Tabelle 4.3 fasst die relevanten Hardwarekomponenten zusammen. Da das Experiment ausschließlich die Auslastung des Main-Threads durch Scripting und Layout betrachtet (Abschnitt 2.4.3), bleibt die GPU-Leistung für die relevanten Messgrößen ohne Bedeutung. Die Phasen Paint und Compositing werden in den Hypothesen H1–H4 nicht berücksichtigt (Abschnitt 2.5.4).

Tab. 4.3.: Hardwarekomponenten der für alle Benchmark-Läufe verwendeten Messmaschine.

Komponente	Spezifikation
CPU	Intel Core i9-9900K (8 Kerne / 16 Threads, Single-Core-Boost 5,0 GHz, All-Core-Boost 4,7 GHz)
RAM	64 GB DDR4-3200
GPU	Intel UHD Graphics 630 (integriert)
Betriebssystem	Windows 11 Education

4.2.2. Software-Versionierung und Build-Konfiguration

Tabelle 4.4 listet die zentralen Versionen auf. In `package.json` sind ausschließlich exakte Versionsnummern definiert (kein `^`, `~` oder `latest`). Installationen erfolgen ausschließlich über `npm ci` unter Verwendung des im Repository hinterlegten Lockfiles.

Tab. 4.4.: Software-Versionen aller experimentrelevanten Abhängigkeiten. Exakte Versionspins verhindern unbemerkte Änderungen an Abhängigkeiten zwischen einzelnen Messläufen.

Komponente	Version	Rolle im Experiment
React	19.2.6	Primäres Framework (VDOM + Compiler)
babel-plugin-react-compiler	1.0.0	Automatisierte Memoisierung im Build
@vitejs/plugin-react	6.0.2	Babel-Integration in die Vite-Pipeline (React Compiler)
Svelte	5.55.7	Primäres Framework (Signals, Runes)
@sveltejs/vite-plugin-svelte	7.1.2	Svelte-Kompilierungspipeline in Vite
Vite	8.0.13	Build-Tool und Preview-Server
TypeScript (react-app, svelte-app)	6.0.3	Statische Typisierung beider Apps
TypeScript (shared-logic)	5.9.3	Kompilierung der MockEngine
Playwright	1.52.0	Teststeuerung und Zugriff auf das CDP
Chromium	via Playwright	Rendering-Engine

Vite-Production-Build als Messvoraussetzung

Alle automatisierten Messungen basieren ausschließlich auf einem Vite-Production-Build, da im Development-Modus zentrale Optimierungen wie die Minifizierung fehlen [53] und der aktive *StrictMode* Komponenten doppelt rendert sowie bestimmte Hooks mehrfach ausführt [54]. Einzige Ausnahme ist die einmalige qualitative Heap-Snapshot-Analyse (Abschnitt 4.3.5), die bewusst im Development-Build durchgeführt wird, um unminifizierte Konstruktornamen für die manuelle Analyse der Dominator-Struktur zu erhalten. Der React Compiler ist als Babel-Plugin Teil der Build-Pipeline und übernimmt die Optimierung des Komponentencodes während des Build-Vorgangs (Abschnitt 2.2.2). Vor jedem Messlauf werden die Production-Builds beider Frameworks neu erzeugt und anschließend über `vite preview` als lokale statische Server bereitgestellt. Playwright greift ausschließlich auf diese optimierten Bundles zu.

Bare-Metal-Ausführung

Das Experiment läuft nativ auf dem Host-Betriebssystem und verzichtet auf zusätzliche Virtualisierungsschichten wie Docker Desktop unter Windows. Diese Entscheidung basiert auf dem in der

Literatur belegten signifikanten Overhead sowie erhöhten Schwankungen bei Zeitmessungen, die durch solche Abstraktionsschichten entstehen [55]. Im Kontext dieser Untersuchung wirkt sich dies insbesondere in zwei Bereichen kritisch aus:

1. **CPU-seitige Timer:** Die Virtualisierung beeinträchtigt die Stabilität des hier eingesetzten Web-Worker-Timers (Abschnitt 4.3.1), da der Hypervisor-seitige Software-Overhead die Genauigkeit virtueller Zeitmessungs-Operationen negativ beeinflussen kann [55].
2. **CPU-Scheduling und Jitter:** Hypervisoren weisen physische CPU-Ressourcen dynamisch verschiedenen virtuellen Instanzen zu. Dadurch entstehen unvorhersehbare Kontextwechsel und zeitliche Überlagerungen, die sich direkt als Jitter in der Programmausführung äußern. Besonders bei zeitkritischen Messungen verursachen diese Scheduling-Effekte zusätzliche Latenzschwankungen und erschweren somit die exakte Bestimmung der **reinen Framework-Performance unter Last** [55].

Minimierung von OS-Interferenzen

Zur Minimierung störender Hintergrundaktivitäten des Betriebssystems werden vor jedem Messlauf die folgenden Vorkehrungen getroffen:

- Schließen aller nicht-essenziellen Anwendungen und Browser-Tabs
- Deaktivierung automatischer System-Updates für die Dauer des Messlaufs
- Durchführung des Benchmarks ohne aktive Netzwerkverbindungen (beide Anwendungen laufen als lokale statische Server)
- Einhaltung einer ausreichenden Aufwärmphase nach dem Systemstart, damit Hintergrunddienste einen stabilen Ruhezustand erreichen
- Umstellung des Windows-Energiesparplans auf *Höchstleistung*, um dynamisches CPU-Frequenzscaling während der Messung zu verhindern [56]

Trotz der getroffenen Maßnahmen lässt sich Thermal Throttling nicht vollständig ausschließen. Die temperaturbedingte Verringerung der CPU-Taktfrequenz erhöht die Streuung der Messergebnisse (IQR) über die zehn Messwiederholungen, verändert jedoch die Rangfolge der Frameworks nicht systematisch. Der Einsatz von Median und IQR als robuste Lage- und Streuungsmaße (Abschnitt 4.3.3) reduziert die Auswirkungen dieser Schwankungen auf die Auswertung.

4.3. Automatisierung und statistische Methodik

4.3.1. Teststeuerung mittels Playwright und CDP-Trace-Analyse

Die vollständige Testmatrix läuft automatisiert über das Playwright-Testframework. Playwright erfüllt dabei drei wesentliche Aufgaben: Es steuert den Chromium-Browser, synchronisiert sich mit dem Benchmark-Ablauf der Anwendungen und greift über das Chrome DevTools Protocol (CDP) auf Performance-Daten zu.

Browser-Konfiguration

Playwright startet Chromium im Headless-Modus (`headless: true`, das ab Playwright 1.35 für Chromium automatisch den `-headless=new`-Codepfad aktiviert) mit einem festen Viewport von 1280×720 Pixeln. Dieser Modus verwendet denselben Codepfad wie der reguläre Chrome-Browser, sodass die gesamte Rendering-Pipeline und die DevTools uneingeschränkt zur Verfügung stehen [57] und eine realitätsnahe Messung der in Abschnitt 2.4.3 definierten Profiling-Kategorien ermöglichen. Die feste Viewport-Größe sorgt dafür, dass layoutabhängige Größen wie die Zellanzahl pro Zeile konstant bleiben und die Klickzielposition für den INP-Trigger unabhängig von n stabil ist (Abschnitt 4.3.1). Ein einzelner Playwright-Worker stellt sicher, dass keine parallelen Browser-Instanzen ausgeführt werden, die sich CPU-Ressourcen teilen und die Messung gegenseitig beeinflussen würden.

Benchmark-Ablauf und Synchronisation

Beide Testanwendungen implementieren einen kontrollierten Benchmark-Ablauf, der präzise Messungen ohne fest vorgegebene Wartezeiten (`sleep`-Kommandos) ermöglicht. Der Ablauf trennt eine Initialisierungsphase, in der das Grid vollständig gerendert und der Web Worker vorbereitet werden, von der eigentlichen Messphase, in der die Ausführung des Tick-Loops erfolgt. Diese Trennung stellt sicher, dass die Erfassung ausschließlich die Rendering-Arbeit des Tick-Loops umfasst und das initiale Laden des JavaScript-Bundles sowie das erste Grid-Rendering ausschließt. Playwright startet den Tick-Loop erst nach einer optionalen Baseline-Heap-Messung, sodass diese in einem stabilen, ruhenden Ausgangszustand erfolgt. Die technische Umsetzung der Synchronisation zwischen Playwright und Testanwendung wird in Abschnitt 5.4.3 erläutert.

CDP-Trace-Analyse für Scripting- und Rendering-Zeit

Für die Scripting- und Rendering-Metrik zeichnet die CDP-Schnittstelle einen Performance-Trace auf [58]. Dieser umfasst exakt die Messphase des Tick-Loops. Die aufgezeichneten Ereignisse werden gemäß Abschnitt 2.4.3 den Scripting- und Rendering-Kategorien zugeordnet und getrennt aufsummiert [59]. Nach der Normierung der resultierenden Millisekundensummen auf die Tick-Anzahl ergeben sich die szenariounabhängigen Größen *scriptingMsPerTick* und *renderingMsPerTick* (Abschnitt 4.3.4). Details zur CDP-Trace-API finden sich in Abschnitt 5.4.6.

Long-Task-Messung via PerformanceObserver

Parallel zum CDP-Trace erfasst ein *PerformanceObserver* Long Tasks (Abschnitt 2.1.2). Um Long Tasks der initialen Renderphase (Seitenaufbau, erstes Grid-Layout) auszuschließen, wird dieser erst 100 ms nach dem vollständigen Rendern des Grids aktiviert. In die Messung fließen damit ausschließlich Long Tasks des folgenden Tick-Loops ein. Je Wiederholung werden zwei Metriken erhoben: *longTaskCount* (Ereignisse über der 50 ms-Schwelle) und *longTaskMs* (Gesamtdauer in Millisekunden). Um sicherzustellen, dass der letzte Long Task eines Durchlaufs nicht verloren geht, werden unmittelbar nach Abschluss des Tick-Loops noch verbleibende Einträge gesichert, bevor

der *PerformanceObserver* getrennt wird. Die Auswertung basiert auf Median und IQR über alle Wiederholungen (Abschnitt 4.3.3). Die Implementierung ist in Abschnitt 5.4.3 beschrieben.

INP-Messung und Klick-Trigger

Die Messung von INP erfolgt im JavaScript-Kontext der Testanwendung mithilfe eines *PerformanceObserver*, der Interaktionsereignisse ab dem browserseitigen Mindestschwellenwert von 16 ms erfasst [60]. Pro Interaktion werden die drei INP-Phasen Input Delay, Processing Time und Presentation Delay (Abschnitt 2.4.1) bestimmt. Für eine praxisnahe Messung löst Playwright alle 100 ms einen synthetischen Mausklick aus. Diese Frequenz führt bei starker Auslastung des Main Threads zu einem Rückstau verzögerter Eingaben (delayed inputs) und bildet damit das Verhalten ungeduldiger Nutzer auf blockierten Benutzeroberflächen realistisch nach [2, 4]. So lassen sich selbst extreme Interaktionslatenzen zuverlässig erfassen. Der Klick-Loop umgeht bewusst die Standard-Playwright-Klick-API. Deren automatische Actionability-Prüfungen [61] können bei hoher Tick-Frequenz Timeouts auslösen, sodass ausgerechnet die kritischsten Blockierungen nicht in die Messung eingehen würden. Stattdessen werden Klicks direkt und unabhängig vom Zustand des JavaScript-Threads übermittelt. Der Klick-Loop startet vor dem Beginn des Tick-Loops, damit bereits beim ersten Tick Klick-Ereignisse eintreffen. Die pro Wiederholung gesammelten INP-Einträge werden über alle $N = 10$ Wiederholungen zusammengeführt und gemeinsam ausgewertet. Die abschließende Berechnung folgt der W3C-Spezifikation [2, 60]: Entscheidend ist die maximale Interaktionsdauer, wobei je 50 Einträge ein Ausreißer unberücksichtigt bleibt. Die technische Umsetzung ist in Abschnitt 5.4.5 dargestellt.

4.3.2. Messwiederholungen und Warmup-Lauf

Die Anzahl von $N = 10$ Wiederholungen je Szenario wurde gewählt, um einen praktikablen Mittelweg zwischen statistischer Aussagekraft und Rechenzeit zu erreichen. Bereits bei 64 Szenarien ergibt sich eine geschätzte Gesamtlaufzeit von sieben bis acht Stunden (vgl. Abschnitt 4.1.3). Eine Erhöhung auf $N = 20$ würde den Aufwand auf ca. 15 Stunden steigern. Gleichzeitig sind weniger als zehn Wiederholungen ebenfalls ungeeignet, da der IQR dann nicht sinnvoll interpretierbar ist. Bei $N = 5$ würde beispielsweise jedes Quartil nur auf einem einzelnen Messwert beruhen, sodass ein Ausreißer Q1 oder Q3 vollständig bestimmen könnte. Vor den zehn Messungen findet pro Szenario ein *Warmup-Lauf* statt, dessen Resultate verworfen werden. Er dient dazu, den browserseitigen Code-Cache, eine V8-interne Zwischenspeicherung bereits kompilierten Codes, zu befüllen [62] und die dynamische Just-In-Time-Kompilierung (JIT) häufig genutzter Programmteile in Maschinencode anzustoßen [63]. Für jede Messwiederholung lädt Playwright die Seite neu (`page.goto()`) [64]. Da `page.goto()` eine neue Navigation startet, wird der JavaScript-Zustand vollständig neu erstellt und Heap-Allokationen aus vorherigen Durchläufen nicht übertragen. V8 ermöglicht die Zwischenspeicherung von generiertem Code direkt im Arbeitsspeicher der jeweiligen Instanz [62]. Ob dieser Cache über Navigationen hinweg teilweise erhalten bleibt, ist ohne V8-interne Profiling-Daten nicht nachweisbar. Da jedoch alle Wiederholungen denselben Ablauf aus Navigation und Benchmark-Ausführung durchlaufen, bleibt die Vergleichbarkeit der Messungen gewährleistet.

4.3.3. Statistische Auswertung: Median und IQR

Für jedes Szenario dient der Median als zentraler Wert. Die Streuung lässt sich über den Interquartilsabstand (IQR) beschreiben, der der Differenz zwischen dem 75. und dem 25. Perzentil entspricht. Beide Kennwerte sind robust gegenüber Ausreißern, wie sie bei Browser-Messungen durch gelegentliche Garbage-Collection-Pausen entstehen können (Abschnitt 4.4.3). Der Median beschreibt den typischen Leistungswert eines Frameworks unter Last, der IQR hingegen misst die Stabilität: Niedrige Werte deuten auf konstantes Verhalten hin, hohe auf kontinuierliche Leistungsschwankungen. Daraus ergibt sich ein unabhängiges Ergebnis für die Tipping-Point-Analyse (Abschnitt 2.5.2). Auf die Annahme einer Normalverteilung sowie auf parametrische Kennzahlen wie Mittelwert und Standardabweichung wird bewusst verzichtet. Wie in Abschnitt 4.4.3 gezeigt, können sporadische GC-Ereignisse einzelne Messungen durch mehrere Millisekunden andauernde Blockierungen des Main-Threads erheblich erhöhen. Solche Ausreißer verzerren die Verteilung der Messwerte, wodurch das arithmetische Mittel die typische Leistung eines Frameworks nicht mehr zuverlässig wiedergibt [65].

4.3.4. Normalisierung der Scripting-Zeit: `scriptingMsPerTick`

Ein zentrales methodisches Problem der kombinierten Testmatrix liegt in den unterschiedlich langen Laufzeiten der Szenarien. Die Szenarien der Testmatrix unterscheiden sich nicht nur in n , sondern auch in der Tick-Anzahl: $\Delta t = 100$ ms erzeugt 100 Ticks, $\Delta t = 16$ ms hingegen 500 Ticks pro Durchlauf. Die absolute Scripting-Gesamtzeit `scriptingMs` skaliert direkt mit der Tick-Anzahl und eignet sich daher nicht als Vergleichswert über Frequenzstufen hinweg. Um dieses Problem zu lösen, kommt eine Normalisierung der Scripting-Zeit auf die Anzahl der tatsächlich verarbeiteten Ticks zum Einsatz:

$$\text{scriptingMsPerTick} = \frac{\text{scriptingMs}}{\text{tickCount}} \quad (4.3)$$

Diese Größe beschreibt den durchschnittlichen CPU-Aufwand des Frameworks pro Datenänderung und ist unabhängig von der vorgegebenen oder tatsächlichen Gesamtlaufzeit des Szenarios. Damit lässt sich das Skalierungsverhalten über alle n -Stufen hinweg direkt vergleichen, weshalb sie als primäre Vergleichsgröße für Hypothese H2 dient. `renderingMsPerTick` wird nach demselben Prinzip ermittelt.

4.3.5. Zweigeteilte Speicheranalyse

Wie in Abschnitt 2.4.2 begründet, nutzt die Speicheranalyse einen zweistufigen Ansatz, der qualitative und quantitative Verfahren kombiniert.

Qualitative Analyse: Manueller Heap-Snapshot im Development-Build

Im ersten Teil findet im *Development-Build* (unminifiziert, mit aussagekräftigen Konstruktornamen) einmalig für beide Frameworks die Erstellung eines manuellen Heap-Snapshots mithilfe der Chrome DevTools statt. Als Grundlage dient ein kleines Grid mit 100 Zellen, das die Dominator-Struktur übersichtlich darstellt (Abschnitt 2.4.2). Der Snapshot zeigt die interne Fiber-Baumstruktur von React,

insbesondere `FiberNode`-Instanzen und deren Retained Size (Abschnitt 2.2.3), und ermöglicht einen qualitativen Vergleich mit dem signalbasierten Speichermodell von Svelte 5 (Abschnitt 2.3.2). Im Production-Build ersetzt die Minifizierung Konstruktornamen durch nichtssagende Kürzel (z.B. `FiberNode` → `t`). Dadurch verliert der Heap seine semantische Struktur, was eine automatisierte Identifikation von Framework-Komponenten unmöglich macht. Zudem können Heap-Snapshots bei $n = 10\,000$ mehrere hundert Megabyte umfassen und sind für automatisierte Läufe unpraktisch. Der qualitative Snapshot bleibt daher ein einmaliger, manueller Schritt.

Quantitative Analyse: JS-Heap-Delta via CDP im Production-Build

Im zweiten Teil erfolgt die automatisierte Ermittlung der Speicherverbrauchs-Skalierung über alle Dichtestufen. Vor der Baseline-Messung werden drei erzwungene Garbage-Collection-Zyklen ausgeführt, um temporäre Heap-Allokationen aus der Initialisierungsphase zu beseitigen. Dies ist notwendig, da V8 den Heap in mehrere Generationen aufteilt und ein einzelner Zyklus nicht zwingend alle davon abdeckt [26]. Zwischen den Zyklen liegt jeweils eine kurze Pause, die sicherstellt, dass der laufende Zyklus vollständig abgeschlossen ist, bevor der nächste einsetzt. Nach Abschluss aller Ticks wird dieselbe GC-Sequenz wiederholt und der finale Heap-Wert gemessen. Das Heap-Delta

$$\Delta_{\text{Heap}} = \text{JSHeapUsedSize}_{\text{final}} - \text{JSHeapUsedSize}_{\text{baseline}} \quad (4.4)$$

beschreibt den nach der Garbage Collection verbleibenden Speicherzuwachs durch den Tick-Loop und dient damit als reproduzierbare Kenngröße für den Speicherbedarf unter Dauerlast. Alle drei Messdurchläufe — Performance, Speicher und INP — laufen in jeweils separaten Playwright-Projekten, um gegenseitige Einflüsse zu vermeiden. `performance.spec.ts` und `memory.spec.ts` dürfen nicht gleichzeitig aktiv sein, da die CDP-Kategorien `Tracing` und `Heap-Profiler` sowohl Messwerte als auch den entstehenden Overhead verfälschen können [66]. `inp.spec.ts` ist aus inhaltlichen Gründen isoliert: Der aktive Click-Loop erzeugt `EventDispatch`-Ereignisse, die in einen parallel laufenden CDP-Trace als Scripting-Zeit einfließen und `scriptingMsPerTick` beeinflussen würden. Im Kontext der Speichermessung können die synthetischen Klicks zudem das GC-Verhalten verändern und damit Δ_{Heap} verzerren (vgl. Abschnitt 4.4.3). Die technische Umsetzung ist in Abschnitt 5.4 beschrieben.

4.4. Validität und Gütekriterien

4.4.1. Interne Validität: Sicherstellung identischer Logik in beiden Implementierungen

Für die interne Validität eines Framework-Vergleichs ist ausschlaggebend, ob beobachtete Leistungsunterschiede tatsächlich auf die Frameworks zurückzuführen sind oder durch unbeabsichtigte Unterschiede in der Implementierung entstehen. Um dies zu gewährleisten, wurden Maßnahmen auf vier Ebenen definiert.

Identische Eingabedaten durch eine framework-unabhängige MockEngine

Zur Sicherstellung identischer Testbedingungen dient die `MockEngine` im gemeinsamen Paket `@benchmark/shared-logic`. Sie erzeugt aus denselben Eingabeparametern (Grid-Größe, Tick-Anzahl, Mutationsrate, Seed) für beide Frameworks einen identischen `TestPlan`: gleiche Anfangszustände aller Zellen, identische Mutationsfolgen je Tick sowie konsistente Zell-IDs für jede Veränderung. Grundlage ist der deterministische Pseudozufallszahlengenerator `Mulberry32` [67], dessen fester Seed sicherstellt, dass beide Frameworks in jedem Szenario exakt dieselbe Renderarbeit ausführen. Da der `TestPlan` einmalig vor dem ersten Renderdurchlauf berechnet wird und danach unveränderlich bleibt, geht dieser Aufwand nicht in die gemessenen Metriken ein. Die Implementierung ist in Abschnitt 5.1.3 beschrieben.

Strukturelle Vergleichbarkeit der Implementierungen

Beide Framework-Implementierungen sind nach denselben architektonischen Prinzipien aufgebaut, die direkt aus den Anforderungen der internen Validität hervorgehen:

- **Prop-Drilling:** Der Zustand des Grids liegt vollständig in der Wurzelkomponente. Primitive Daten (`value` als `Integer`) werden per Props weitergereicht. Zusätzliche Mechanismen wie State-Management-Frameworks, Context-API oder Svelte-Stores kommen nicht zum Einsatz. Dadurch sind die Update-Pfade in beiden Frameworks strukturell vergleichbar und enthalten keine zusätzlichen Zwischenschichten.
- **Identische Zellstruktur:** Jede Zelle hat eine feste Größe von 30×30 Pixeln. Die festen Maße vermeiden dynamische Layoutberechnungen (vgl. Abschnitt 2.1.1), die die Rendering-Metrik frameworkunabhängig erhöhen würden.
- **Stabile Keys:** Für die Iteration nutzen beide Frameworks die stabile Zell-ID (`item.id`) und nicht den Array-Index. Instabile Keys führen in React zu unnötigen DOM-Unmount- und Remount-Vorgängen [68] und verfälschen damit die Messergebnisse.
- **Bailout-Guard:** Der Datengenerator stellt sicher, dass jede Mutation eine tatsächliche Wertänderung bewirkt. Ohne diese Sicherstellung könnten identische Werte dazu führen, dass Framework-Optimierungen, wie die reaktiven Updates bei Svelte-Runen oder die automatische Memoization des React Compilers, keine Änderung feststellen und daher keine Aktualisierung ausführen [8, 9]. Dies hätte zur Folge, dass weniger Rendereffort entsteht als beabsichtigt, wodurch es zu einem unbemerkten Messfehler kommt.
- **Identischer Web-Worker-Timer:** eine inhaltlich identische `ticker.worker.ts` mit drift-korrigiertem `setTimeout` (Abschnitt 5.1.4). Dadurch ist die Tick-Frequenz für beide Frameworks identisch und bleibt unabhängig von der jeweiligen Rendering-Auslastung des Main-Threads (vgl. Abschnitt 4.4.3).

Der einzige bewusst gewählte strukturelle Unterschied zwischen den Implementierungen ist methodisch erforderlich: React benötigt für die Änderungserkennung im Diffing-Prozess eine neue

Objektreferenz (vgl. Abschnitt 2.2.1 und [20, 69]), während Svelte 5 durch sein Signal-System feingranulare Änderungen direkt am bestehenden Objekt identifiziert (vgl. Abschnitt 2.3.2 und [9, 38]). Dabei handelt es sich nicht um einen Implementierungsfehler, sondern um ein grundlegendes Merkmal der jeweiligen Reaktivitätsmodelle, dessen Einfluss auf Speicherverbrauch und CPU-Last in dieser Arbeit empirisch untersucht wird.

Verbotene Optimierungen

In der React-Implementierung bleibt der Einsatz von `React.memo`, `useMemo` und `useCallback` bewusst aus, um die architektonischen Eigenschaften beider Frameworks im Ausgangszustand vergleichbar zu machen. Manuelle Optimierungen würden die systembedingten Unterschiede zwischen Reacts VDOM-basiertem Laufzeitmodell und Sveltes compilerbasiertem Ansatz reduzieren und somit die empirische Vergleichbarkeit beeinträchtigen. Darüber hinaus kann der React Compiler seine automatische Memoisierung unbeeinflusst anwenden (Abschnitt 2.2.2). In der Svelte-Implementierung werden ausschließlich Svelte-5-Runes (`$state`, `$derived`, `$props`) verwendet. Svelte-4-Syntax (`export let`, reaktive Statements `$:`, Svelte Stores) ist vollständig ausgeschlossen (Abschnitt 2.3.2).

4.4.2. Externe Validität: Übertragbarkeit auf reale Anwendungen

Übertragbarkeit des Testszenarios

Das Test-Szenario einer dynamischen Heatmap-Matrix mit hochfrequenten Datenänderungen spiegelt zentrale Anforderungen moderner datenintensiver Webanwendungen wider. Dies gilt insbesondere für Echtzeit-Dashboards (z. B. Monitoring-Systeme) und interaktive Visualisierungen, bei denen kontinuierliche Datenströme Teile der Benutzeroberfläche mit hoher Frequenz aktualisieren [3]. Die gewählten Parameter $n \in \{100, 400, 900, 1600, 4000, 6000, 8000, 10\,000\}$ und $\Delta t \in \{100, 50, 33, 16\}$ ms decken diesen Lastbereich systematisch ab und schließen den Abschnitt ein, in dem die in Abschnitt 2.5.2 bestimmte Sättigungsgrenze erreicht wird.

Grenzen der externen Validität

Gleichzeitig ist das Testszenario bewusst vereinfacht, um eine hohe Kontrollierbarkeit und präzise Messungen zu ermöglichen. Einige Aspekte realer Anwendungen werden jedoch nicht abgebildet:

- **Komponentenvielfalt:** Reale Anwendungen bestehen häufig aus unterschiedlich aufgebauten Komponentenbäumen mit stark variierendem Rendering-Aufwand, wodurch die gemessenen Skalierungskurven davon abweichen könnten.
- **Netzwerklatenz:** Die Datenänderungen basieren auf einem lokal vorab berechneten Testplan und nicht auf tatsächlich über das Netzwerk empfangenen Daten. Schwankungen der Tick-Rate, die durch Netzwerkeinflüsse entstehen, sind somit vollständig ausgeschlossen. Dies erhöht zwar die Kontrollierbarkeit, schränkt jedoch die Übertragbarkeit auf Szenarien mit unregelmäßigem Dateneingang ein.

- **Routing und Code-Splitting:** Einflüsse der Navigation auf Bundle-Ladezeiten und Hydrationsprozesse in größeren Anwendungen bleiben in diesem Experiment unberücksichtigt.
- **Komplexe Interaktionsprozesse:** Die INP-Messung basiert auf synthetischen Klick-Ereignissen. Komplexere Interaktionen (Formulareingaben, Drag-and-Drop, Tastatursteuerung) können andere INP-Verläufe zeigen, da sich Art und Dauer der Event-Handler-Callbacks unterscheiden.

Die Ergebnisse dieser Arbeit gelten in erster Linie für *datenintensive, hochfrequent aktualisierte* Systeme. Für überwiegend statische Anwendungen mit seltenen Datenmutationen, beispielsweise klassische Content-Management-Systeme oder Formularanwendungen, sind die Schlussfolgerungen nur begrenzt relevant, da dort die in Abschnitt 2.5.1 beschriebenen Abstraktionskosten kaum Einfluss auf die Skalierung haben.

4.4.3. Messgenauigkeit und Störfaktoren

Garbage Collection

Garbage-Collection-Pausen der V8-Engine stellen den wichtigsten zufälligen Störfaktor bei Browser-Performance-Messungen dar. Insbesondere Major-GC-Ereignisse können den Main Thread für mehrere Millisekunden blockieren und so in einzelnen Durchläufen deutlich erhöhte Scripting-Werte verursachen [26]. Für die Heap-Messung (Abschnitt 4.3.5) wird dieser Einfluss durch eine dreifach erzwungene Garbage Collection vor jeder Baseline- und Endmessung kontrolliert. Da V8 Speicherobjekte in mehrere Generationen aufteilt, deckt ein einzelner erzwungener GC-Lauf nicht zwingend alle davon ab [26]. Mehrere aufeinanderfolgende Zyklen mit kurzen Pausen dazwischen stellen sicher, dass der Heap einen stabilen Ausgangszustand erreicht (vgl. Abschnitt 5.4.4). Für die Scripting-Messung via CDP-Trace kann GC-Overhead nicht aktiv ausgeschlossen werden, da eine erzwungene GC im Tick-Loop die gemessene Frameworklast verfälschen würde. Entstehende Ausreißer werden stattdessen durch robuste statistische Maße wie Median und IQR kontrolliert (Abschnitt 4.3.3).

JIT-Optimierungen

V8 durchläuft bei der Ausführung mehrere Optimierungsstufen für häufig genutzte Codepfade, beginnend bei der Interpretation bis zur Erzeugung von Maschinencode [26]. Ohne entsprechende Maßnahmen würden frühe Messungen zusätzlich die Kosten dieser JIT-Kompilierung enthalten und daher höhere Scripting-Zeiten aufweisen als spätere Durchläufe. Der in Abschnitt 4.3.2 beschriebene Warmup-Lauf begrenzt diesen Einfluss; die verbleibenden Grenzen des Ansatzes sind dort erläutert. Darüber hinaus passen JIT-Compiler den Optimierungsgrad während der Laufzeit an, indem sie häufig genutzte Methoden auf Basis von Profilinformatoren erneut und aggressiver kompilieren [70]; dieser Effekt tritt innerhalb eines Messlaufs auf und wird durch den Warmup-Lauf nicht kontrolliert.

Browser-interne Interferenzen und Gesamtbewertung

Neben GC und JIT können weitere Browser-interne Prozesse die Messung beeinflussen:

- **Hintergrundprozesse des Browsers:** Chromium führt intern speicherverwaltende und diagnostische Hintergrundaufgaben aus. Die Verwendung eines sauberen, neu gestarteten Profils ohne Erweiterungen und ohne Netzwerkanbindung minimiert äußere Einflüsse deutlich.
- **Timer-Ungenauigkeit des Main-Threads:** Der HTML-Standard garantiert keine exakte Ausführung von Timern und berücksichtigt Verzögerungen durch CPU-Last sowie konkurrierende Aufgaben. Deshalb reagiert `setInterval` im Main-Thread unter hoher Auslastung unzuverlässig. Ein Web-Worker-Timer (Abschnitt 4.3.1) ist davon entkoppelt, da Worker in einem separaten Agenten mit eigener Event-Loop laufen und unabhängig von UI-Skripten arbeiten [71].
- **Thermal Throttling:** Anhaltende CPU-Volllast kann zu einer temporären Reduktion des Takts führen. Dieser Effekt lässt sich im vorliegenden Versuchsaufbau nicht vollständig vermeiden (vgl. Abschnitt 4.2.2).

Die beschriebenen Maßnahmen stellen sicher, dass die Messungen unter möglichst konstanten Bedingungen durchgeführt werden und die Vergleichbarkeit der Ergebnisse gewährleistet ist. Verbleibende externe Einflüsse können zwar nicht vollständig ausgeschlossen werden, ihr Einfluss auf die Auswertung wird jedoch minimiert und bei der Interpretation der Ergebnisse als Limitation berücksichtigt.

5. Implementierung

5.1. Architektur des Monorepos und Shared Logic

Das Experiment ist als npm-Workspace-Monorepo aufgebaut, um sicherzustellen, dass beide Framework-Anwendungen identische Eingabedaten aus demselben Paket `@benchmark/shared-logic` erhalten (vgl. Abschnitt 4.4.1). In der zentralen `package.json` sind vier Pakete deklariert: `@benchmark/shared-logic`, `react-app`, `svelte-app` und `@benchmark/runner`. npm löst Abhängigkeiten zwischen diesen Paketen lokal auf, ohne dass ein separater Paketregistrierungs-Schritt erforderlich ist. Beide Framework-Anwendungen nutzen `@benchmark/shared-logic` als direkte Abhängigkeit. Während der Installation bindet npm das Paket über einen Symlink im jeweiligen `node_modules`-Verzeichnis ein [72]; dadurch greifen beide Anwendungen auf dieselbe versionierte Implementierung zu. Abbildung 5.1 veranschaulicht die Struktur der Pakete und ihre Beziehungen.

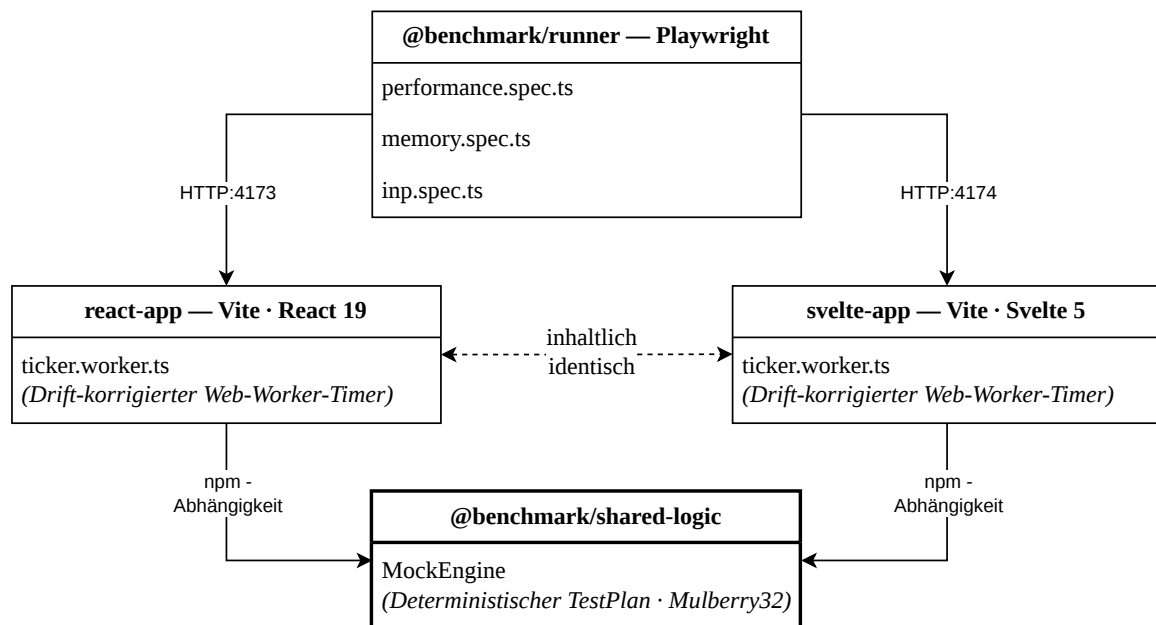


Abb. 5.1.: Paketstruktur und Abhängigkeitsbeziehungen des npm-Workspace-Monorepos.

5.1.1. Das Paket `@benchmark/shared-logic`

Das Paket `@benchmark/shared-logic` ist als reines TypeScript-Paket ohne Laufzeitabhängigkeiten implementiert; die Kompilierung erfolgt einmalig mit `tsc`.

5.1.2. Datenmodell: Interfaces **GridItem**, **TickMutation** und **TestPlan**

Das Paket exportiert drei TypeScript-Interfaces, die das Datenmodell beider Anwendungen beschreiben (siehe Listing A.1 im Anhang). **GridItem** repräsentiert eine einzelne Zelle des Grids mit eindeutiger numerischer ID sowie dem aktuellen Anzeigewert. **TickMutation** beschreibt eine Zustandsänderung einer einzelnen Zelle und spezifiziert deren ID sowie den zugehörigen Folgewert. **TestPlan** fasst den vollständigen Ablauf eines Szenarios zusammen: **initialState** enthält den Ausgangszustand aller Zellen, **ticks** ist ein zweidimensionales Array, dessen äußere Dimension die zeitliche Abfolge der Ticks beschreibt, während die innere Dimension die Zellen angibt, die pro Tick gleichzeitig aktualisiert werden. Der **TestPlan** entsteht einmalig vor dem ersten Render und bleibt anschließend unverändert (vgl. Abschnitt 4.4.1).

5.1.3. Implementierung der **MockEngine**

Die Klasse **MockEngine** stellt die Methode `generateTestPlan(gridSize, tickCount, mutationRate, seed=123)` bereit. Zur Erzeugung deterministischer Zufallswerte kommt der Pseudozufallszahlengenerator **Mulberry32** zum Einsatz [67], da `Math.random()` keine standardisierte Seed-Initialisierung bietet. Bei gleichem Seed liefert **Mulberry32** reproduzierbare Float-Werte im Intervall $[0, 1)$. Die Umsetzung auf Basis von 32-Bit-Ganzzahlarithmetik verursacht keinen messbaren Mehraufwand im Benchmark. Die Methode `generateTestPlan` durchläuft zwei Phasen. In der ersten Phase wird **initialState** befüllt: Für jede der `gridSize` Zellen erzeugt **Mulberry32** einen Startwert im Bereich $[0, 100)$. Gleichzeitig speichert **currentValues** in einem Shadow-Array gleicher Länge die aktuellen Zellwerte, die in der zweiten Phase für die Bailout-Guard-Prüfung benötigt werden. In der zweiten Phase entstehen `tickCount` Tick-Einträge. Die Zahl der je Iteration zu mutierenden Zellen ergibt sich aus $\lceil \text{gridSize} \cdot \text{mutationRate} \rceil$. Die Bestimmung der Indizes erfolgt zufällig und ohne Zurücklegen (vgl. Abschnitt 4.1.1); hierfür dient ein vorab angelegtes Array aller Zell-Indizes der Länge `gridSize`, das über alle Ticks hinweg wiederverwendet wird. In jeder Iteration werden nur die ersten `mutationCount` Positionen des Arrays durch gezielte Tauschoperationen neu angeordnet. Dabei entstehen genau `mutationCount` eindeutige Indizes ohne Duplikate. Im Vergleich zu einem vollständigen Shuffle mit $O(n)$ reduziert sich der Aufwand je Iteration auf $O(\text{mutationCount})$ [52]; die Wiederverwendung des Pools vermeidet zudem Heap-Allokationen pro Tick. Für jeden selektierten Index stellt der Bailout-Guard (vgl. Listing A.2 im Anhang) sicher, dass der generierte Folgewert nicht mit dem gespeicherten Vorgänger übereinstimmt. Der neue Wert aktualisiert **currentValues** und gelangt zusammen mit der ID als **TickMutation** in das entsprechende Tick-Array.

5.1.4. Der drift-korrigierte Web-Worker-Timer (**ticker.worker.ts**)

Beide Framework-Anwendungen verwenden eine inhaltlich identische Datei `ticker.worker.ts`, die als separater Web Worker läuft. Nach dem Empfang einer Startnachricht mit dem gewünschten Tick-Intervall startet er einen Tick-Scheduler, der Drift durch kumulativen Zeitabgleich korrigiert [73] (vollständiger Code: Listing A.3 im Anhang). Bei jedem Aufruf von `scheduleNextTick` findet ein Abgleich der realen Ausführungszeit mit `expectedTime` statt. Die

dabei gemessene Abweichung (*drift*) wird vom nächsten Delay abgezogen, sodass Verspätungen einzelner Ticks in nachfolgenden Intervallen kompensiert werden. Der Worker sendet lediglich ein Zeitsignal. Die eigentlichen Mutationsdaten empfangen die Anwendungen über einen internen Tick-Zähler aus dem vorab berechneten `TestPlan`. Durch die Ausführung des Workers in einem eigenen Agent-Thread mit separater Event-Loop bleibt die Tick-Frequenz unabhängig von der Rendering-Last des Main Threads [71] (vgl. Abschnitt 4.4.3).

5.2. Technische Umsetzung der React-Referenzanwendung

Die React-Referenzanwendung ist als Vite-Projekt mit React 19 umgesetzt und besteht aus drei Ebenen: `App.tsx` bildet den Einstiegspunkt der Anwendung, `Grid.tsx` übernimmt die Anordnung der Inhalte und `Cell.tsx` stellt die einzelnen Zellen dar. Dieser Aufbau erfüllt die in Abschnitt 4.4.1 formulierte Anforderung der strukturellen Vergleichbarkeit. Gleichzeitig dient die Anwendung als Grundlage zur Untersuchung des React-Compilers in einer typischen Komponentenstruktur mit geringer Verschachtelungstiefe.

5.2.1. Komponentenstruktur und Prop-Weitergabe

Der vollständige `TestPlan` entsteht in `App.tsx` bereits auf Modulebene vor der Definition der Komponenten (vgl. Abschnitt 4.4.1), sodass die Berechnung nur einmal beim Laden des Bundles erfolgt und nicht bei jedem Render-Aufruf. Der Initialzustand des Grids liegt in einem `useState`-Array vom Typ `GridItem[]` vor. Darüber hinaus existieren zwei weitere `useState`-Felder für den Ablaufstatus (`phase`) und die UI-Fortschrittsanzeige (`tickCount`). `App.tsx` gibt das Array als `items`-Prop an `Grid.tsx` weiter, das lediglich die Layoutstruktur bereitstellt (vollständiger Komponentencode: Listing A.4 im Anhang). `Grid.tsx` übergibt an `Cell` den Integer `value` anstelle des gesamten `GridItem`-Objekts. Dadurch kann der React-Compiler die Komponente memoisieren und die erneute Ausführung der Renderfunktion vermeiden, solange sich der Wert nicht ändert. Lokale Zustandsänderungen führen weiterhin zu regulären Re-Renders. Zudem beschränkt sich der Prop-Vergleich des React Compilers auf einen einfachen Integer-Gleichheitscheck (`===`), was die strukturelle Vergleichbarkeit mit der Svelte-Implementierung sicherstellt (vgl. Abschnitt 4.4.1). `Cell.tsx` repräsentiert die einzelnen Zellen des Grids. Die Komponente rechnet den übergebenen Wert zur visuellen Hervorhebung in eine Farbe im Farbmodell Hue, Saturation, Lightness (HSL) um. Für den INP-Trigger (vgl. Abschnitt 4.3.1) verwaltet die Komponente zusätzlich einen lokalen `clicked`-Zustand.

5.2.2. Unveränderliche Referenzaktualisierung im Tick-Handler

React erkennt Zustandsänderungen anhand von Objektreferenzen (Abschnitt 2.2.1). Der Tick-Handler in `App.tsx` nutzt dieses Prinzip (vollständiger Code: Listing A.5 im Anhang): Der Spread-Operator `[...prev]` erzeugt zunächst eine flache Kopie des Arrays. Für jede der $[n \cdot 0,05]$ mutierten Zellen (vgl. Abschnitt 4.1.1) erfolgt anschließend das Einfügen eines neuen `{id, value}`-Objekts an der jeweiligen Position. Alle übrigen Positionen behalten ihre ursprünglichen Objektreferenzen. Diese Eigenschaft ist entscheidend für die Korrektheit der Messung: Da `Grid`

das gesamte `items`-Array als Prop erhält, löst jede Tick-Mutation ein erneutes Rendering aus. Anschließend vergleicht der React-Compiler für jede `Cell` den aktuellen `value`-Prop mit dem zuvor gespeicherten Wert (`===`) [20]. Nur bei einer Änderung führt die jeweilige Zelle ihre Renderfunktion erneut aus. Abbildung 5.2 zeigt diesen Mechanismus im Vergleich zum Ansatz der direkten Zustandsänderung in Svelte 5. Die flache Kopie des Arrays wirkt sich auch auf die Speicheranalyse aus (Abschnitt 4.3.5): Pro Tick entstehen $[n \cdot 0,05]$ neue `GridItem`-Objekte sowie ein neues Array der Länge n . Die ersetzten Objekte werden zu Garbage-Collection-Kandidaten.

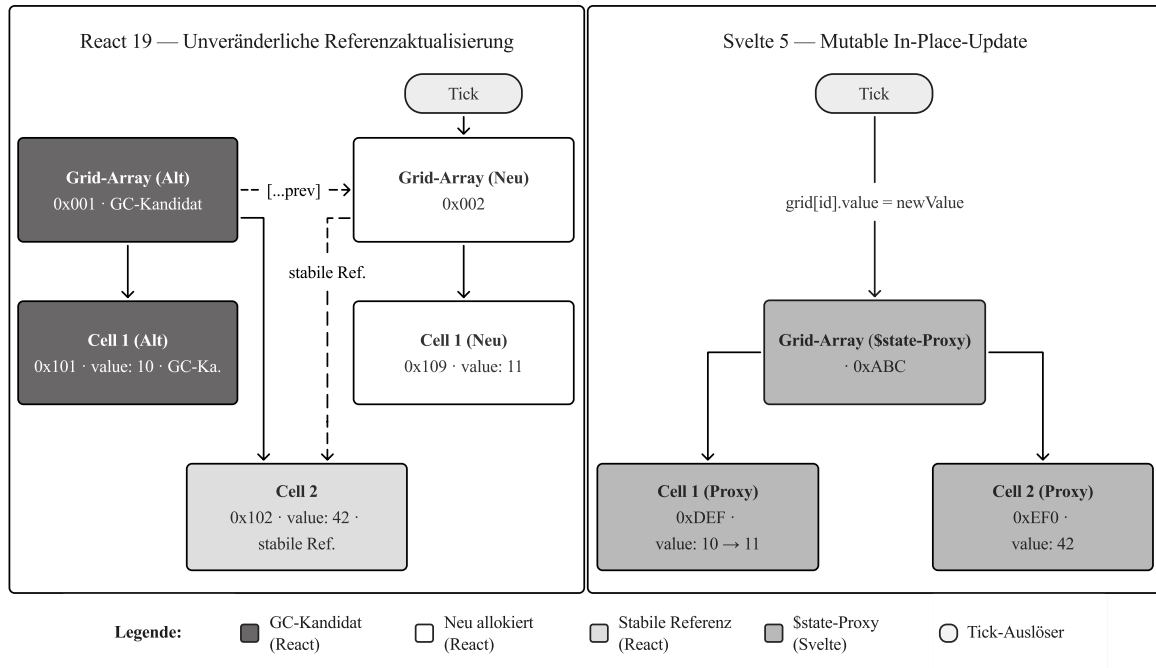


Abb. 5.2.: Gegenüberstellung der Update-Strategien: React 19 arbeitet mit der Erzeugung neuer Heap-Objekte bei jeder Änderung (dunkelgrau markiert als Garbage-Collection-Kandidaten), während Svelte 5 bestehende Signal-Objekte direkt aktualisiert und so zusätzliche Speicherallokationen weitgehend vermeidet (Eigene Darstellung, KI-unterstützt).

5.2.3. Steuerung durch den Web Worker und Benchmark-Ablauf

`App.tsx` instanziiert den Web Worker (Abschnitt 5.1.4) in einem `useEffect`-Hook mit leerer Abhängigkeitsliste (`[]`), sodass die Initialisierung nur einmal nach dem ersten Rendern erfolgt. Zusätzlich registriert der Hook einen `'benchmark-start'`-Event-Listener auf `window`. Die vollständige Startlogik ist in Listing A.6 im Anhang abgebildet. Das Flag `IS_BENCHMARK_MODE` ist `true`, sobald der Query-Parameter `gridSize` in der URL vorhanden ist; andernfalls löst die Anwendung das Ereignis selbst aus und startet unmittelbar. Das `data-testid`-Attribut des Root-Elements wechselt von `'benchmark-ready'` auf `'benchmark-done'`, sobald der Worker beendet wurde und `setPhase('done')` den zugehörigen React-Re-Render ausgelöst hat (vgl. Abschnitt 4.3.1).

5.2.4. Integration des React Compilers

Die Datei `vite.config.ts` bindet den React-Compiler als Babel-Plugin in die Build-Pipeline ein (vollständige Konfiguration: Listing A.7 im Anhang). Das Plugin `@vitejs/plugin-react` übernimmt die Standard-Umwandlung von JavaScript XML (JSX), während `@rollup/plugin-babel` die anschließende Compiler-Umwandlung ausführt. Die Funktion `reactCompilerPreset()` bindet `babel-plugin-react-compiler` als Preset ein und aktiviert es in der in Abschnitt 4.2.2 festgelegten Version 1.0.0. Der Compiler analysiert jede Komponente statisch auf Seiteneffekte und veränderte Props. Komponenten ohne solche Eigenschaften werden automatisch memoisiert, sodass das Ergebnis des letzten Renderns zwischengespeichert und eine erneute Ausführung bei unveränderten Eingaben verhindert wird [8]. Ob der Compiler korrekt aktiv ist, lässt sich auf zwei Ebenen überprüfen. Auf Build-Ebene enthält das erzeugte JavaScript-Bundle typische Compiler-Artefakte, darunter direkte Aufrufe der internen Funktion `_c` sowie Zugriffe auf ein Cache-Array, das durch das Symbol `$` dargestellt wird. Auf Laufzeitebene kennzeichnet der React DevTools Profiler optimierte Komponenten mit der Annotation «Memo», wobei ein zusätzliches Sparkle-Symbol anzeigt, dass die Memoisierung automatisch durch den Compiler erfolgte [74]. Beide Indikatoren sind im vorliegenden Setup erfüllt: Die Komponenten `App`, `Grid` und `Cell` wurden vollständig optimiert. Wie in Abschnitt 4.4.1 dargelegt, enthält der Quellcode keine manuellen Optimierungen mittels `React.memo`, `useMemo` oder `useCallback`. Zudem hat der `<StrictMode>`-Wrapper in `main.tsx` im Production-Build keine Auswirkungen und beeinflusst die Messergebnisse daher nicht (vgl. Abschnitt 4.2.2).

5.3. Technische Umsetzung der Svelte-Referenzanwendung

Die Umsetzung in Svelte folgt der in Abschnitt 5.2 beschriebenen Dreistufenarchitektur (`App.svelte`, `Grid.svelte`, `Cell.svelte`) und stellt damit die strukturelle Vergleichbarkeit sicher (vgl. Abschnitt 4.4.1). Svelte-4-Elemente wie (`export let`, reaktive Statements `$:` und Svelte Stores) kommen nicht zum Einsatz; alle Dateien verwenden ausschließlich die Runes-Syntax von Svelte 5 (vgl. Abschnitt 4.4.1).

5.3.1. Komponentenstruktur und Prop-Weitergabe

In Svelte 5 gibt es innerhalb einer `.svelte`-Datei zwei Script-Blöcke: `<script module>` enthält Code auf Modulebene, der beim Laden genau einmal ausgeführt wird; `<script>` umfasst instanzbezogenen Code, der bei jedem Mount einer Komponente läuft. In `App.svelte` kommt diese Trennung gezielt zum Einsatz: Der `TestPlan` entsteht im `<script module>`-Block (vgl. Abschnitt 4.4.1 und Listing A.8 im Anhang), wodurch die Berechnung nicht bei jedem Rendern erneut erfolgt. Das `Grid` ist im komponentenbezogenen `<script>`-Block als `$state`-Array deklariert (vgl. Listing A.9 im Anhang). Der Spread-Operator `{...item}` erzeugt für jedes `GridItem`-Objekt eine flache Kopie. Dadurch arbeitet die Anwendung mit eigenen, veränderbaren Instanzen, während der `TestPlan` selbst unverändert bleibt. `$state()` erlaubt es, Änderungen an einzelnen Eigenschaften zur Laufzeit gezielt nachzuverfolgen (vgl. Abschnitt 2.3.2). Die Prop-

Weitergabe an `Grid.svelte` und `Cell.svelte` erfolgt über die Rune `$props()` (vgl. Listing A.10 im Anhang). `Grid.svelte` iteriert mit einem schlüsselbasierten `{#each}`-Block und übergibt ausschließlich den primitiven `value`-Integer an `Cell`. `Cell.svelte` verwaltet intern einen `clicked`-Zustand als `$state`. Beide Entscheidungen sind strukturell identisch mit der React-Implementierung (vgl. Abschnitt 5.2.1).

5.3.2. Direkte Zustandsmutation im Tick-Handler

Der Unterschied liegt in der Art der Zustandsänderung. Anstatt bei jeder Änderung ein neues Array zu erzeugen (vgl. Abschnitt 5.2.2), mutiert Svelte 5 die vorhandenen Objekte direkt (vollständiger Tick-Handler: Listing A.11 im Anhang). Der `$state`-Proxy erkennt und verarbeitet die Zuweisung `grid[id].value = newValue`. Der `{#each}`-Block in `Grid.svelte` generiert zur Laufzeit pro Iteration eine feingranulare reaktive Bindung auf `item.value`. `Cell.svelte` selbst enthält dabei keine eigene Reaktivitätslogik, sondern empfängt lediglich den aktualisierten primitiven Wert als Prop. Eine Property-Änderung aktualisiert ausschließlich die betroffene `Cell`, ohne dass das Array kopiert wird, neue Objektreferenzen erzeugt werden oder der gesamte Komponentenbaum durchlaufen wird (vgl. Abschnitt 2.3.2), wodurch die Referenzen auf die einzelnen Objekte in `grid` während des gesamten Benchmark-Verlaufs stabil bleiben. Reacts `setGrid`-Aufruf erzeugt hingegen pro Tick neue Objekte und eine neue Array-Instanz (vgl. Abschnitt 5.2.2). Abbildung 5.2 zeigt diesen Kontrast: React hinterlässt Garbage-Collection-Kandidaten, während Svelte den Heap-Speicherbedarf stabil hält. Dieser strukturelle Unterschied gilt als erwartete Ursache des in Hypothese H3 untersuchten JS-Heap-Deltas (vgl. Abschnitt 2.5.4).

5.3.3. Steuerung durch den Web Worker und Benchmark-Ablauf

Ein `$effect`-Block übernimmt die Instanziierung des Workers (vollständige Implementierung: Listing A.12 im Anhang). Anders als `useEffect` in React benötigt `$effect` kein Dependency-Array [21, 36], da reaktive Abhängigkeiten automatisch ermittelt werden (vgl. Abschnitt 2.3.2). Da dieser Effekt keinen reaktiven State verwendet, besitzt er keine Abhängigkeiten und entspricht somit `useEffect(fn, [])`; der Rückgabewert ist eine Cleanup-Funktion, die vor jeder erneuten Ausführung sowie beim Unmount der Komponente läuft. Die `IS_BENCHMARK_MODE`-Steuerung und das `data-testid`-Protokoll sind inhaltlich identisch mit der React-Implementierung (vgl. Abschnitt 5.2.3). Innerhalb des Tick-Handlers zeigt sich ein weiterer Unterschied: `tickIndex` ist als `let tickIndex = 0` nicht reaktiv und löst keine Updates aus. Dieses Verhalten entspricht in React der Verwendung von `useRef` (Abschnitt 5.2.2), stellt in Svelte 5 jedoch den Normalfall dar, da Variablen ohne `$state` für das Reaktivitätssystem unsichtbar bleiben. Parallel dazu existiert `tickCount` als `$state`, das ausschließlich den UI-Zähler in der Statuszeile aktualisiert. Diese Trennung macht die Granularität von Sveltes Reaktivitätsmodell sichtbar: Nur explizit deklarierte `$state`-Variablen erzeugen Rendering-Arbeit.

5.4. Technische Umsetzung des Benchmark-Pakets

Das Paket `@benchmark/runner` setzt den in Abschnitt 4.3 beschriebenen automatisierten Testablauf um und speichert die Ergebnisse in einem maschinenlesbaren Format. Es ist in drei voneinander unabhängige Playwright-Testdateien aufgeteilt, von denen jede einen eigenen Messdurchlauf abdeckt: `performance.spec.ts`, `memory.spec.ts` und `inp.spec.ts`. Gemeinsam genutzte Logik liegt in fünf Hilfsmodulen im Verzeichnis `src/helpers/`: `scenarios.ts`, `cdp.ts`, `longTasks.ts`, `inp.ts` und `results.ts`. Die Begründung für die Trennung in drei separate Playwright-Projekte ist in Abschnitt 4.4.3 dargelegt.

5.4.1. Playwright-Konfiguration: `playwright.config.ts`

In der Datei `playwright.config.ts` werden die grundlegenden Konfigurationen für alle drei Projekte festgelegt (vollständige Konfiguration: Listing A.13 im Anhang). `workers: 1`, `headless: true` und der Viewport von 1280×720 Pixeln setzen die in Abschnitt 4.3.1 begründeten Anforderungen um. Das Argument `-disable-software-rasterizer` deaktiviert den SwiftShader-Fallback und stellt sicher, dass Rendering-Arbeit ausschließlich auf den Hardware-GPU-Codepfaden stattfindet, nicht auf CPU-Emulation [75]. Auf Maschinen ohne Hardware-GPU lässt sich das Argument über `NO_GPU` deaktivieren.

5.4.2. Szenariensteuerung: `scenarios.ts`

Die Datei `scenarios.ts` definiert die vollständige Testmatrix als TypeScript-Konstanten. Die Funktion `allScenarios()` erzeugt durch eine dreifache Schleife über alle Grid-Größen, Frequenzstufen und Frameworks die 64 Szenarien der Testmatrix (Abschnitt 4.1.3); der vollständige Code findet sich in Listing A.14 im Anhang. Jede der drei Spec-Dateien ruft `allScenarios()` auf und registriert pro Szenario einen `test()`-Block. Playwright übergibt die Szenariokonfiguration per URL-Query-Parameter an die Anwendungen:

```
http://localhost:4173?gridSize=N&tickInterval=T&tickCount=C
```

Auf diese Weise lässt sich die vollständige Testmatrix ohne Neukompilierung der Anwendungen ausführen (vgl. Abschnitt 4.3.1).

5.4.3. Performance-Messung: `performance.spec.ts`

`performance.spec.ts` misst je Szenario die Kennzahlen `scriptingMsPerTick`, `renderingMsPerTick`, `longTaskCount` und `longTaskMs`. Die Synchronisation zwischen Playwright und der Testanwendung basiert auf drei Signalen:

1. `data-testid="benchmark-ready"`: Dieses Attribut kennzeichnet den Initialzustand des Root-Elements beider Anwendungen und ist im DOM sichtbar, sobald der erste Render-Zyklus abgeschlossen ist, d. h. das Grid vollständig gerendert wurde. Playwright wartet gezielt auf diesen Marker, bevor CDP-Traces gestartet werden. Damit beschränkt sich die Erfassung auf die Rendering-Arbeit des Tick-Loops und schließt das initiale Laden des JavaScript-Bundles sowie das erste Grid-Rendering aus.

2. `window.dispatchEvent(newCustomEvent('benchmark-start'))`: Playwright löst dieses Ereignis nach dem Start der Testumgebung aus. Die Anwendung sendet daraufhin den Startbefehl an den bereits erstellten Web Worker, was den drift-korrigierten (Abschnitt 5.1.4) Tick-Loop aktiviert (vgl. Abschnitt 4.3.1).
3. `data-testid="benchmark-done"`: Die Anwendung setzt dieses Attribut nach Abschluss aller Ticks. Playwright nutzt es als eindeutiges Signal für das Ende des Messzeitraums.

Vor dem eigentlichen Messloop erfolgt einmalig ein Warmup-Lauf (vgl. Abschnitt 4.4.3), der die V8-JIT-Kompilierung vorab auslöst. Der Ablauf jeder der zehn regulären Messwiederholungen gliedert sich in drei Phasen: Seitenaufruf mit CDP-Session-Aufbau, Start des Long-Task-Beobachters und des CDP-Trace nach `benchmark-ready`, sowie Auswertung und Ergebnisspeicherung nach `benchmark-done`. Nach allen zehn Wiederholungen berechnet die Spec für jede der vier Metriken Median und IQR (vgl. Abschnitt 4.3.3) und schreibt das Ergebnis als JSON-Datei (vgl. Abschnitt 5.4.7).

Long-Task-Beobachter: `longTasks.ts`

Die Hilfsfunktion `injectLongTaskObserver(page)` fügt über `page.evaluate()` einen *PerformanceObserver* mit `type: 'longtask'` und `buffered: false` in den Seitenkontext ein. Durch `buffered: false` und das 100 ms-Delay nach `benchmark-ready` bleiben Post-Render-Aktivitäten außerhalb des Messfensters (vgl. Abschnitt 4.3.1). Nach dem Warten auf den `benchmark-done`-Marker ruft `collectLongTasks(page)` zunächst `observer.takeRecords()` auf, um noch nicht asynchron übermittelte Einträge zu sichern [14, 76]. Abschließend werden `longTaskCount` und `longTaskMs` berechnet und zurückgegeben.

5.4.4. Speichermessung: `memory.spec.ts`

`memory.spec.ts` bestimmt das JS-Heap-Delta Δ_{Heap} gemäß Abschnitt 4.3.5. Analog zu `performance.spec.ts` erfolgt zunächst ein Warmup-Lauf ohne Messung. Jede Messwiederholung öffnet nach `page.goto()` eine neue CDP-Session und die Domänen `HeapProfiler` und `Performance` aktiviert. Die Funktion `forceGC(cdp, page)` implementiert die in Abschnitt 4.4.3 beschriebene dreifache GC-Sequenz: drei CDP-Aufrufe von `HeapProfiler.collectGarbage` mit jeweils 100 ms Wartezeit nach jedem Aufruf. Pro Messwiederholung erfolgen zwei Aufrufe von `forceGC`: nach `benchmark-ready` zur Erfassung des Heap-Baseline-Werts H_{base} sowie nach Abschluss aller Ticks vor dem Auslesen des Heap-Endstands H_{final} . Über zehn Wiederholungen ergeben sich Median und IQR des Heap-Deltas Δ_{Heap} .

5.4.5. INP-Messung: `inp.spec.ts`

`inp.spec.ts` setzt die in Abschnitt 4.3.1 beschriebene INP-Messung um. Da INP über alle $N = 10$ Wiederholungen hinweg gepoolt ist, sammelt die Spec pro Szenario einen gemeinsamen `allINPEntries`-Puffer.

INP-Beobachter und Klick-Loop

`injectINPObserver(page)` injiziert über `page.evaluate()` direkt nach `page.goto()` einen *PerformanceObserver* in den Seitenkontext. Der Beobachter registriert Events des Typs `'event'` mit `durationThreshold: 0`; der browserseitige Mindestschwellenwert von 16 ms gilt dennoch [60]. Für jede erfasste Interaktion werden die drei INP-Phasen berechnet (Formel: Listing A.15 im Anhang). Die Klick-Schleife startet vor dem `benchmark-start`-Event und führt alle 100 ms einen Klick an einer vorab ermittelten Position aus (vgl. Abschnitt 4.3.1).

INP-Berechnung nach W3C-Spezifikation

Die Funktion `calcINP(allEntries)` setzt die W3C-Spezifikation um (vollständige Implementierung: Listing A.16 im Anhang). Die Einträge werden absteigend nach Dauer sortiert; der schlechteste Wert nach Ausschluss des obersten $\lfloor N/50 \rfloor$ -Anteils bildet den INP-Wert (vgl. Abschnitt 4.3.1).

5.4.6. CDP-Hilfsfunktionen: **cdp.ts**

Die Hilfsfunktion `startTrace` startet die Aufzeichnung im `ReportEvents`-Modus und sammelt eingehende Trace-Ereignisse in einem Array. `stopTrace` beendet die Aufzeichnung und wartet auf das Signal `Tracing.tracingComplete` [58]. Die Funktion `parseTrace(events, tickCount)` ordnet vollständige Trace-Ereignisse anhand ihres Namensfelds den in Abschnitt 2.4.3 definierten Kategorien zu [46]; die Konstantendefinitionen sind in Listing A.17 im Anhang aufgeführt. `parseTrace` berücksichtigt ausschließlich Ereignisse mit `ph: 'X'`, bei denen `cat` den Wert `'devtools.timeline'` enthält und `dur` größer null ist. Die Ereignisdauern werden von Mikrosekunden in Millisekunden umgerechnet und mit `tickCount` normalisiert, was `scriptingMsPerTick` und `renderingMsPerTick` liefert (vgl. Abschnitt 4.3.4).

5.4.7. Ergebnisspeicherung und Auswertungsexport: **results.ts**

Nach jeder abgeschlossenen Messreihe legt `saveResult()` die Ergebnisse als formatierte JSON-Datei in einem datumsbasierten Verzeichnis (`results/YYYY-MM-DD/`) ab. Der Dateiname enthält Informationen zu Metrik, Framework, Grid-Größe und Tick-Intervall, beispielsweise `perf-react_n1600_f50.json`. Die Performance- und Speicher-Dateien enthalten neben Median und IQR je Metrik auch die Rohdaten aller Einzelwiederholungen, sodass die statistische Auswertung im Nachhinein nachvollziehbar ist. INP-Dateien speichern ausschließlich das Ergebnis (`INPResult`); da die Einträge aller Wiederholungen vor der Berechnung zu einem gemeinsamen Pool zusammengeführt werden, liegen keine separaten Wiederholungswerte vor. Nach Abschluss aller drei Benchmark-Läufe führt `writeSummaryCSV(dateStr)` die einzelnen JSON-Dateien anhand des im Dateinamen enthaltenen Szenario-Bezeichners (`framework_nN_fT`) in einer `summary.csv` zusammen. Pro Zeile enthält diese die sechs Primärmetriken eines Szenarios. Die Performance-Metriken `scriptingMsPerTick`, `renderingMsPerTick`, `longTaskCount` und `longTaskMs` liegen jeweils als Median und IQR vor; das Heap-Delta `heapDelta` ausschließlich

als Median (`heapDelta_median_bytes`). INP wird als einzelner W3C-Wert exportiert, ergänzt um die Phasenwerte sowie die Anzahl der berücksichtigten Einträge (*pooledEntries*). Die `summary.csv` dient als Grundlage für die statistische Auswertung in Kapitel 6.

6. Ergebnisse und Auswertung

Alle 1920 Benchmark-Läufe der in Abschnitt 4.1.3 definierten Testmatrix wurden erfolgreich abgeschlossen; kein einzelner Lauf musste verworfen werden. Sämtliche im Folgenden genannten Messwerte sind, sofern nicht durch eine eigene Abbildung oder Tabelle im Fließtext belegt, in Tabelle B.1 im Anhang vollständig dokumentiert. Ab $n = 8000$ ist die Messstreuung (IQR) in mehreren Szenarien sichtbar erhöht. Als wahrscheinlichste Ursache kommen die Browser-internen Störeinflüsse (Abschnitt 4.4.3) in Betracht, die sich nicht vollständig kontrollieren lassen.

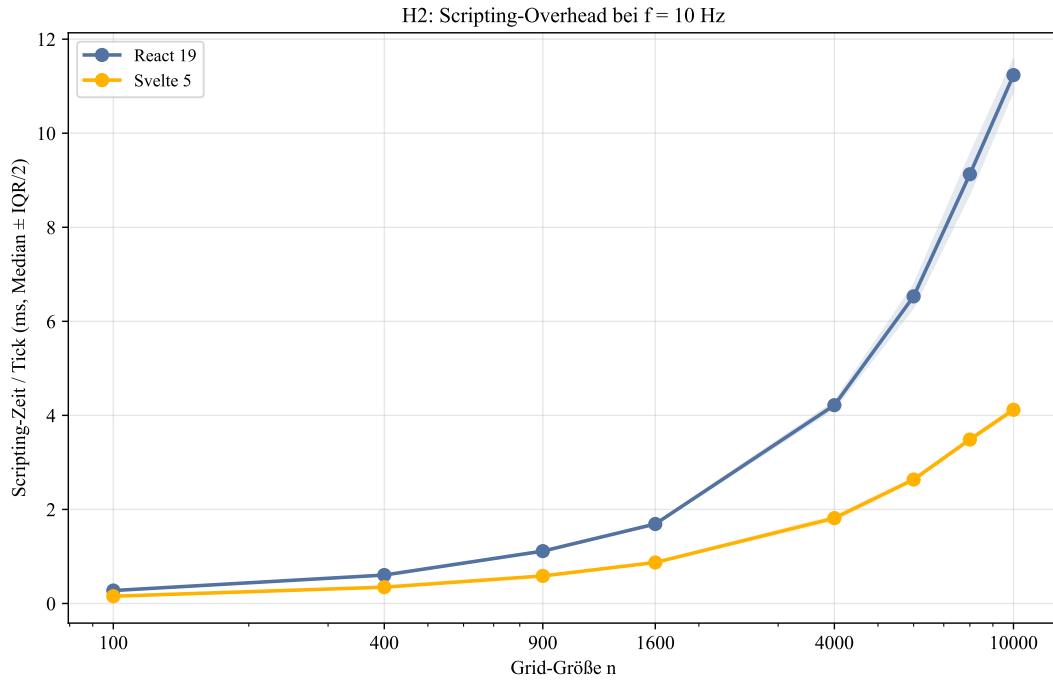
6.1. H2: Scripting-Aufwand pro Tick bei steigender UI-Dichte

6.1.1. Scripting-Zeit, Overhead-Faktor und Frequenzeinfluss

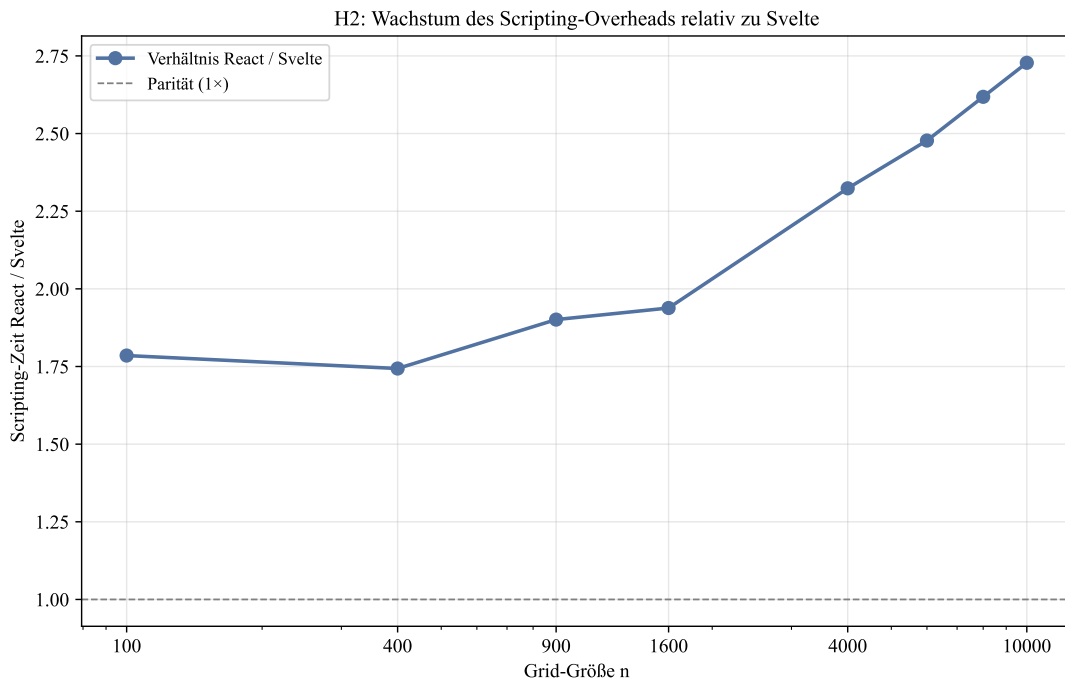
Mit zunehmender UI-Dichte n steigen die medianen *scriptingMsPerTick*-Werte beider Frameworks stetig an (Abb. 6.1a). Die Kurve von React verläuft dabei durchgehend steiler als die von Svelte; bei $n = 10\,000$ und $f = 10\text{ Hz}$ beträgt die Scripting-Zeit für React 11,2 ms gegenüber 4,1 ms für Svelte. Ab $n \geq 8000$ erhöht sich die Messstreuung (IQR) deutlich, was auf die in Abschnitt 4.4.3 diskutierten browserinternen Störeinflüsse zurückgeführt werden kann. Der in Abb. 6.1b gezeigte Overhead-Faktor (Verhältnis von React- zu Svelte-Scripting-Zeit) wächst bei $f = 10\text{ Hz}$ mit zunehmendem n : Der relative Scripting-Mehraufwand von React gegenüber Svelte nimmt mit steigender UI-Dichte zu. Beide Frameworks weisen zwar eine Skalierung von $O(n)$ auf (Abschnitte 2.3.2 und 2.5.3), unterscheiden sich jedoch im Aufwand pro Element. Reacts Reconciliation verarbeitet bei jeder Zustandsänderung stets den gesamten betroffenen Teilbaum, unabhängig davon, wie viele Knoten tatsächlich verändert wurden (Abschnitt 2.5.3). Zusätzlich werden bei jedem Render-Vorgang erneut React-Elemente erzeugt. Sveltes feingranulare Reaktivität begrenzt den Aufwand hingegen auf die tatsächlich betroffenen Abhängigkeiten (Abschnitt 2.3.2), was den Aufwand pro Element reduziert (Abschnitt 2.3.2). Für $f = 10\text{ Hz}$ weisen beide Frameworks bei jeder n -Stufe nahezu identische Rendering-Zeiten auf: Bei $n = 4000$ erreichen beide 4,74 ms pro Tick; bei $n = 10\,000$ betragen die Werte 13,31 ms (React) gegenüber 13,55 ms (Svelte, Abweichung 1,8 %). Der messbare Unterschied zwischen den Frameworks liegt bei dieser Frequenz somit überwiegend im Scripting (Abschnitt 2.4.3). Die INP-Phasenzerlegung (Abschnitt 6.3.2) zeigt bei $f = 10\text{ Hz}$ allerdings einen deutlichen Unterschied in der *presentationDelay* (51,9 ms vs. 78,1 ms). Dieser Unterschied wird durch die Rendering-Zeit pro Tick nicht erfasst; die Ursache lässt sich mit den erhobenen Metriken nicht eindeutig bestimmen (vgl. Abschnitt 6.6). Bei $f = 20\text{ Hz}$ treten jedoch Unterschiede auf: Für $n = 10\,000$ betragen die Rendering-Zeiten 11,87 ms (React) beziehungsweise 13,71 ms (Svelte), was einer Abweichung von 15 % entspricht. Als mögliche Ursache kommt Sveltes Effect-Batching in Betracht (Abschnitt 2.3.3): Die gebündelten DOM-Updates werden innerhalb eines einzigen Microtasks abgearbeitet, ohne dass verbleibende Arbeit auf spätere Frames verteilt wird (Abschnitt 2.2.3). Dadurch könnte der Rendering-Aufwand bei $f = 20\text{ Hz}$ pro Tick gebündelt

anfallen. React hingegen gibt durch kooperatives Yielding (Abschnitt 2.2.3) die Kontrolle zwischen einzelnen Fiber-Schritten an den Browser zurück. Der genaue Mechanismus lässt sich anhand der vorliegenden Daten nicht eindeutig bestimmen. Der Rendering-Unterschied ist bei $f = 20$ Hz am größten (15 %) und gleicht sich bei $f \approx 62$ Hz nahezu aus (4,95 ms vs. 4,92 ms). Mit steigender Tick-Frequenz nimmt die Scripting-Zeit pro Tick insgesamt ab, jedoch nicht gleichförmig und nicht gleich stark bei beiden Frameworks. Bei Svelte steigt die Scripting-Zeit bei $n = 10\,000$ von 4,12 ms (10 Hz) zunächst auf 4,29 ms (20 Hz) an, bevor sie auf 3,66 ms (62 Hz) fällt. Für React ($n = 10\,000$) fällt *scriptingMsPerTick* von 11,2 ms (10 Hz) auf 3,42 ms (62 Hz): ein Rückgang um 70 %. Für Svelte sinkt der Wert im selben Bereich von 4,1 ms auf 3,66 ms (Rückgang um 11 %). Beide Frameworks nähern sich mit steigender Frequenz einem gemeinsamen Mindestaufwand an: dem durch die $O(n)$ -Komplexität bedingten Basisaufwand, den keine Laufzeitoptimierung unterschreiten kann ($\lceil n \cdot 0,05 \rceil$ Signal-Auswertungen bzw. die Fiber-Reconciliation, vgl. Abschnitt 2.3.2). Da Reacts Ausgangswert bei niedrigen Frequenzen weiter von diesem Minimum entfernt liegt als der von Svelte, fällt der relative Rückgang für React entsprechend größer aus. Der große React-Rückgang deutet auf frequenzabhängig wegfallenden Overhead hin, möglicherweise durch JIT-Optimierungen auf häufig ausgeführten Code-Pfaden (Abschnitt 4.4.3). Sveltes geringer Rückgang ergibt sich daraus, dass der Spielraum nach unten von vornherein klein ist. Der genaue Mechanismus lässt sich ohne V8-internes Profiling nicht bestimmen. Bei $f \approx 62$ Hz und $n = 10\,000$ ergibt sich dadurch ein Scripting-Schnittpunkt: React erreicht 3,42 ms (IQR 0,13 ms), Svelte 3,66 ms (IQR 0,10 ms); damit ist Svelte hier erstmals geringfügig langsamer als React. Auffällig ist, dass Reacts Scripting-Zeit bei $f \approx 62$ Hz von $n = 8000$ (3,53 ms) auf $n = 10\,000$ (3,42 ms) leicht zurückgeht, während sie bei allen anderen Frequenzen durchgehend mit n wächst. Da der Unterschied von 0,11 ms kleiner ist als der IQR beider Messpunkte ($n = 8000$: 0,20 ms; $n = 10\,000$: 0,13 ms), lässt er sich nicht sicher vom Messrauschen unterscheiden. Der Scripting-Schnittpunkt ergibt sich daher überwiegend aus Sveltes $O(n)$ -Angleichung (Abschnitt 2.3.2) und dem ungleichmäßigen frequenzabhängigen Rückgang im Scripting, nicht dadurch, dass Reacts Scripting-Zeit gestiegen wäre. Damit schränkt der Scripting-Schnittpunkt H2 ein, ohne H2 zu widersprechen. Die Rendering-Zeiten verlaufen bei $n = 10\,000$ uneinheitlich. Bereits bei $f = 10$ Hz sind sie nahezu identisch. Nach dem Maximum des Abstands bei $f = 20$ Hz fallen die React-Werte stetig: von 13,31 ms (10 Hz) über 11,87 ms (20 Hz) und 8,40 ms (30 Hz) auf 4,95 ms (62 Hz). Sveltes Werte bleiben zunächst bei 13,55 ms (10 Hz) bzw. 13,71 ms (20 Hz), bevor sie über 9,45 ms (30 Hz) auf 4,92 ms (62 Hz) abfallen. Bei $f \approx 62$ Hz nähern sich beide Frameworks einem gemeinsamen Wert von ca. 4,9 ms an. Der genaue Mechanismus dieser frequenzabhängigen Abnahme lässt sich anhand der vorliegenden Messdaten nicht eindeutig bestimmen.

6. Ergebnisse und Auswertung



(a) Scripting-Zeit (Median $\pm$ IQR) bei $f = 10$ Hz



(b) Overhead-Faktor React / Svelte über n bei $f = 10$ Hz

Abb. 6.1.: H2 – Scripting-Auswertung: (a) mediane *scriptingMsPerTick* beider Frameworks bei $f = 10$ Hz; (b) Quotient React / Svelte bei $f = 10$ Hz über alle n -Stufen (vgl. Abschnitt 2.4.3).

6.1.2. Bewertung von H2

Bei $f \leq 30$ Hz bestätigen die Messdaten H2: Der Scripting-Aufwand pro Tick von React wächst mit steigendem n stärker als der von Svelte, was auf die Reconciliation-Kosten der Fiber-Architektur und die feingranulare Reaktivität von Svelte zurückzuführen ist (Abschnitte 2.2.3 und 2.3.2). Anzu-merken ist, dass sich der genaue Anteil der Compiler-Memoisierung an diesen Scripting-Kosten im gewählten Versuchsaufbau nicht isolieren lässt (vgl. Abschnitt 6.6). Hinzu kommt, dass bei $f \approx 30$ Hz der Abstand im letzten n -Intervall ($n = 8000$ auf $n = 10\,000$) von 3,94 ms auf 3,35 ms zurückgeht: Sveltes Scripting-Zuwachs (+0,79 ms) übersteigt dort den von React (+0,20 ms). Da der Unterschied von 0,59 ms jedoch innerhalb der Summe der IQR-Werte beider Frameworks bei $n = 8000$ liegt (React-IQR: 0,40 ms, Svelte-IQR: 0,19 ms), lässt er sich nicht sicher vom Messrauschen unterscheiden. Bei $f \approx 62$ Hz gilt H2 nur eingeschränkt: Svelte erreicht bei $n = 10\,000$ mit 3,66 ms erstmals eine höhere Scripting-Zeit als React (3,42 ms). Dieser Scripting-Schnittpunkt lässt sich durch zwei Faktoren erklären: die in Abschnitt 2.3.2 beschriebene $O(n)$ -Angleichung, die bei steigendem n auch bei einer Mutationsrate von 5 % je Tick praktisch wirksam wird, sowie den ungleichmäßigen frequenzabhängigen Scripting-Rückgang (Abschnitt 6.1.1). **H2 wird teilweise bestätigt.**

6.2. H3: Speichereffizienz unter Dauerlast

6.2.1. Qualitative Heap-Analyse: Dominator-Struktur im Dev-Build

Heap-Snapshots bei $n = 100$ im Development-Build zeigen die internen Datenstrukturen beider Frameworks in lesbarer Form (Abschnitt 4.3.5). Abbildung 6.2 stellt die aufgeklappten Objekte gegenüber; Tabelle 6.1 fasst die abgelesenen Größen zusammen.

Tab. 6.1.: Qualitative Heap-Analyse: interne Objekt-Strukturen bei $n = 100$ im Development-Build (Chrome DevTools, Heap-Snapshot nach Benchmark-Abschluss)

Metrik	React 19 (FiberNode)	Svelte 5 (Signal-POJO)
Instanzen ($n = 100$)	423	103
Shallow Size gesamt	60,9 kB	5,8 kB
Retained Size gesamt	131 kB	19,8 kB

Anmerkung: Shallow Size pro Objekt (berechnet als Gesamt ÷ Anzahl): React ≈ 144 B, Svelte 5 ≈ 56 B. Quelle: Chrome DevTools Memory-Tab, Summary-Ansicht, Dev-Build (localhost:5173 bzw. localhost:5174, gridSize=100).

6. Ergebnisse und Auswertung

Constructor	Distance	Shallow Size	Retained Size
▼ FiberNode x423	7	60.9 kB 0 %	131 kB 1 %
▼ FiberNode @128867	8	0.1 kB 0 %	6.5 kB 0 %
▶ elementType :: App() @118607	7	0.0 kB 0 %	24.5 kB 0 %
▶ type :: App() @118607	7	0.0 kB 0 %	24.5 kB 0 %
▶ map :: system / Map @128873	8	0.6 kB 0 %	7.5 kB 0 %
▶ memoizedState :: {memoizedState, baseState, baseQueue, queue, next} @275689	9	0.0 kB 0 %	5.4 kB 0 %
▶ alternate :: FiberNode @273843	9	0.1 kB 0 %	1.3 kB 0 %
▶ updateQueue :: {lastEffect, events, stores, memoCache} @275687	9	0.0 kB 0 %	0.9 kB 0 %
▶ child :: FiberNode @128843	7	0.1 kB 0 %	0.2 kB 0 %
▶ return :: FiberNode @192465	9	0.1 kB 0 %	0.2 kB 0 %
▶ __proto__ :: {constructor} @128871	8	0.0 kB 0 %	0.2 kB 0 %
▶ _debugStack :: Error @192473	9	0.2 kB 0 %	0.2 kB 0 %
▶ _debugHookTypes :: Array @132303	9	0.1 kB 0 %	0.1 kB 0 %
▶ _debugTask :: {run, <symbol>} @34675	9	0.0 kB 0 %	0.0 kB 0 %
▶ memoizedProps :: {} @192467	9	0.0 kB 0 %	0.0 kB 0 %
▶ pendingProps :: {} @192467	9	0.0 kB 0 %	0.0 kB 0 %
▶ actualDuration :: heap number @132287	9	0.0 kB 0 %	0.0 kB 0 %
▶ actualStartTime :: heap number @132291	9	0.0 kB 0 %	0.0 kB 0 %
▶ selfBaseDuration :: heap number @132295	9	0.0 kB 0 %	0.0 kB 0 %
▶ treeBaseDuration :: heap number @132299	9	0.0 kB 0 %	0.0 kB 0 %
▶ _debugInfo :: null @71	3	0.0 kB 0 %	0.0 kB 0 %
▶ _debugNeedsRemount :: false @75	4	0.0 kB 0 %	0.0 kB 0 %
▶ _debugOwner :: null @71	3	0.0 kB 0 %	0.0 kB 0 %
▶ childLanes :: smi number @268247	3	0.0 kB 0 %	0.0 kB 0 %
▶ deletions :: null @71	3	0.0 kB 0 %	0.0 kB 0 %
▶ dependencies :: null @71	3	0.0 kB 0 %	0.0 kB 0 %
▶ flags :: smi number @274589	9	0.0 kB 0 %	0.0 kB 0 %
▶ index :: smi number @268247	3	0.0 kB 0 %	0.0 kB 0 %
▶ key :: null @71	3	0.0 kB 0 %	0.0 kB 0 %
▶ lanes :: smi number @268291	3	0.0 kB 0 %	0.0 kB 0 %
▶ mode :: smi number @270539	8	0.0 kB 0 %	0.0 kB 0 %
▶ ref :: null @71	3	0.0 kB 0 %	0.0 kB 0 %
▶ refCleanup :: null @71	3	0.0 kB 0 %	0.0 kB 0 %
▶ sibling :: null @71	3	0.0 kB 0 %	0.0 kB 0 %
▶ stateNode :: null @71	3	0.0 kB 0 %	0.0 kB 0 %
▶ subtreeFlags :: smi number @270553	8	0.0 kB 0 %	0.0 kB 0 %
▶ tag :: smi number @268247	3	0.0 kB 0 %	0.0 kB 0 %

(a) React 19: FiberNode-Instanz mit allen Feldern

Constructor	Distance	Shallow Size	Retained Size
▼ {f, v, reactions, equals, rv, vv, label, updated} x103	8	5.8 kB 0 %	19.8 kB 0 %
▼ {ctx, deps, effects, equals, f, fn, reactions, rv, v, vv, parent, ac, label, updated} @186015	15	0.1 kB 0 %	0.3 kB 0 %
▶ v :: system / JSPProxy @217189	13	0.0 kB 0 %	2.9 kB 0 %
▶ __proto__ :: {constructor, __defineGetter__, __defineSetter__, __get __proto__, set __proto__, toLocaleString} @49235	3	0.1 kB 0 %	0.3 kB 0 %
▶ parent :: {ctx, deps, nodes, f, first, fn, last, next, parent, b, prev, teardown, vv, ac, component_function} @217529	12	0.1 kB 0 %	0.1 kB 0 %
▶ map :: system / Map @158601	16	0.0 kB 0 %	0.1 kB 0 %
▶ reactions :: Array @185013	16	0.1 kB 0 %	0.1 kB 0 %
▶ ctx :: {p, i, c, e, s, x, r, l, function} @104427	12	0.1 kB 0 %	0.1 kB 0 %
▶ equals :: safe_equals() @174271	6	0.0 kB 0 %	0.0 kB 0 %
▶ fn :: () @217579	16	0.0 kB 0 %	0.0 kB 0 %
▶ label :: "{#each ...}" @27791	13	0.0 kB 0 %	0.0 kB 0 %
▶ ac :: null @71	3	0.0 kB 0 %	0.0 kB 0 %
▶ deps :: null @71	3	0.0 kB 0 %	0.0 kB 0 %
▶ effects :: null @71	3	0.0 kB 0 %	0.0 kB 0 %
▶ f :: smi number @261779	11	0.0 kB 0 %	0.0 kB 0 %
▶ properties :: system / PropertyArray @217581	16	0.0 kB 0 %	0.0 kB 0 %
▶ rv :: smi number @261741	16	0.0 kB 0 %	0.0 kB 0 %
▶ updated :: null @71	3	0.0 kB 0 %	0.0 kB 0 %
▶ vv :: smi number @261861	3	0.0 kB 0 %	0.0 kB 0 %

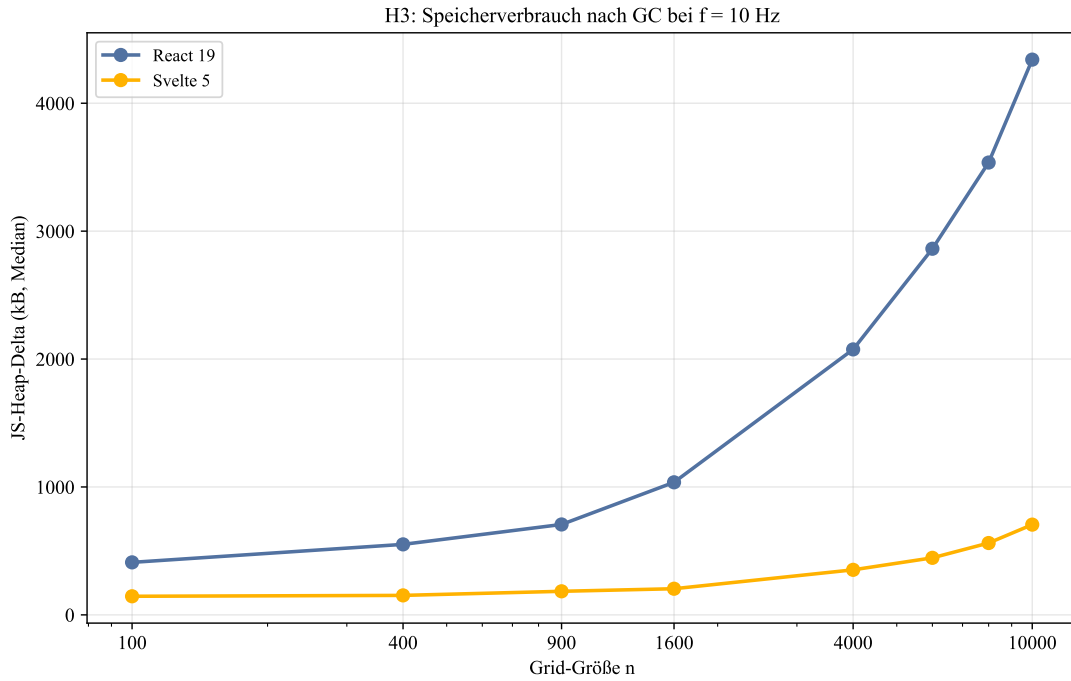
(b) Svelte 5: Signal-POJO {f, v, reactions, ...}

Abb. 6.2.: Chrome DevTools Heap-Snapshot ($n = 100$, Dev-Build): aufgeklappte interne Objekte beider Frameworks im Vergleich. Reacts FiberNode enthält neben Produktionsfeldern neun ausschließlich im Dev-Build vorhandene Debug-Felder (`_debugStack`, `actualDuration` u. a.), während Sveltes Signal-POJO auf acht Felder beschränkt ist; die Feldanzahlen sind in (a) und (b) direkt ablesbar (Architekturkontext: Abschnitte 2.2.3 und 2.3.2).

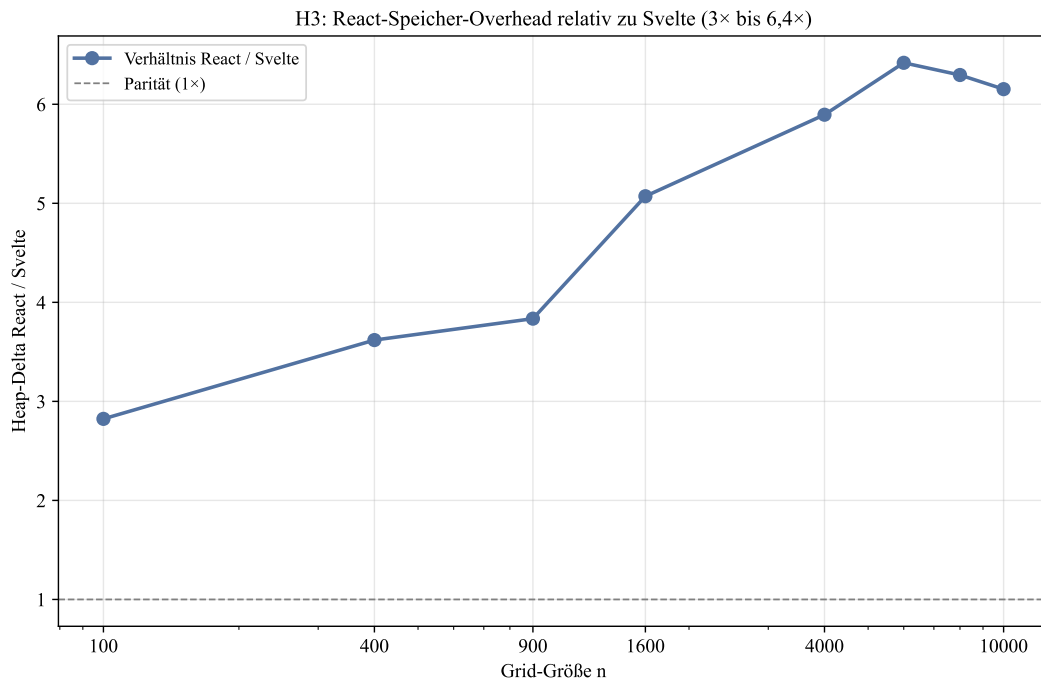
6.2.2. JS-Heap-Delta und Speicher-Overhead-Verhältnis

Das JS-Heap-Delta Δ_{Heap} wächst bei React über alle n -Stufen deutlich stärker als bei Svelte (Abb. 6.3a). Beide Kurven steigen stetig mit n . Der Abstand zwischen den Frameworks nimmt dabei mit zunehmendem n zu. Der Frequenzeinfluss auf Δ_{Heap} ist vernachlässigbar: Der React-Wert beträgt ca. 4450 kB ($f = 10$ Hz) bzw. ca. 4830 kB ($f \approx 62$ Hz) bei $n = 10\,000$. Der in Abb. 6.3b gezeigte Speicher-Overhead-Faktor liegt bei $n = 100$ bei $2,8\times$ und erreicht bei $n = 6000$ ein Maximum von $6,4\times$. Dieser Anstieg lässt sich durch die in Abschnitt 6.2.1 ermittelte Retained Size der `FiberNode`-Instanzen erklären: React benötigt pro Komponenteninstanz erheblich mehr Speicher als Sveltes `Signal-POJOs` (vgl. Abschnitt 2.3.2). Auffällig ist, dass der Overhead-Faktor nach dem Maximum ($n = 6000$: $6,37\times$) nicht weiter ansteigt, sondern für größere n wieder abnimmt ($n = 8000$: $6,24\times$, $n = 10\,000$: $6,18\times$; zuvor $n = 4000$: $5,92\times$). Diese Abflachung ab $n = 6000$ lässt sich anhand der vorliegenden Daten nicht eindeutig begründen und wird daher lediglich dokumentiert.

6. Ergebnisse und Auswertung



(a) JS-Heap-Delta (Δ_{Heap} , in kB) bei $f = 10$ Hz



(b) Speicher-Overhead-Faktor React / Svelte über n bei $f = 10$ Hz

Abb. 6.3.: H3 – Speicherauswertung: (a) medianes JS-Heap-Delta beider Frameworks bei $f = 10$ Hz; (b) Quotient React / Svelte bei $f = 10$ Hz über alle n -Stufen (vgl. Abschnitt 2.4.2).

6.2.3. Bewertung von H3

H3 formuliert die Hypothese bezogen auf die Retained Size pro Komponente (Abschnitt 2.5.4); diese ist in Abschnitt 6.2.1 qualitativ bestätigt: Bei $n = 100$ beträgt die Retained Size gesamt React 131 kB vs. Svelte 19,8 kB (vgl. Tabelle 6.1), entspricht $\approx 1,31$ kB vs. $\approx 0,20$ kB pro Komponente: Faktor $\approx 6,6\times$. Da der FiberNode im Development-Build neun zusätzliche Debug-Felder enthält (Abb. 6.2), die im Production-Build nicht vorhanden sind, ist das gemessene Verhältnis gegenüber dem Production-Build nach oben verzerrt. Das JS-Heap-Delta Δ_{Heap} misst die Skalierung quantitativ über alle n -Stufen im Production-Build (Abschnitt 4.3.5) und bildet damit die belastbarere Vergleichsbasis für H3. React weist über alle n -Stufen einen deutlich höheren Speicherverbrauch auf als Svelte; der Overhead reicht von $2,8\times$ ($n = 100$) bis $6,4\times$ ($n = 6000$). Die Richtung dieses Faktors ist konsistent mit den im Development-Build beobachteten Unterschieden in der Retained Size der internen Datenstrukturen (Abschnitt 6.2.1). Die dort ermittelten konkreten Verhältniswerte sind jedoch nicht direkt auf den Production-Build übertragbar. Die beobachtete Abflachung des Overhead-Faktors ab $n = 6000$ weicht zwar vom erwarteten Wachstum ab, steht jedoch nicht im Widerspruch zu H3. **H3 wird bestätigt.**

6.3. H1: Interaktionslatenz (INP) unter Last

6.3.1. INP-Verlauf über die Grid-Größe

Abbildung 6.4 zeigt den INP beider Frameworks bei $f = 10$ Hz und $f \approx 62$ Hz über alle n -Stufen. Der maximale gemessene INP beträgt 144 ms (React, $n = 10\,000$, $f \approx 62$ Hz) und liegt damit durchgängig im Bereich „Good“ (≤ 200 ms) der Web-Vitals-Klassifikation (Abschnitt 2.4.1). Dennoch steigt der INP beider Frameworks mit wachsender UI-Dichte spürbar an; React erreicht in der Mehrzahl der Szenarien höhere Werte als Svelte.

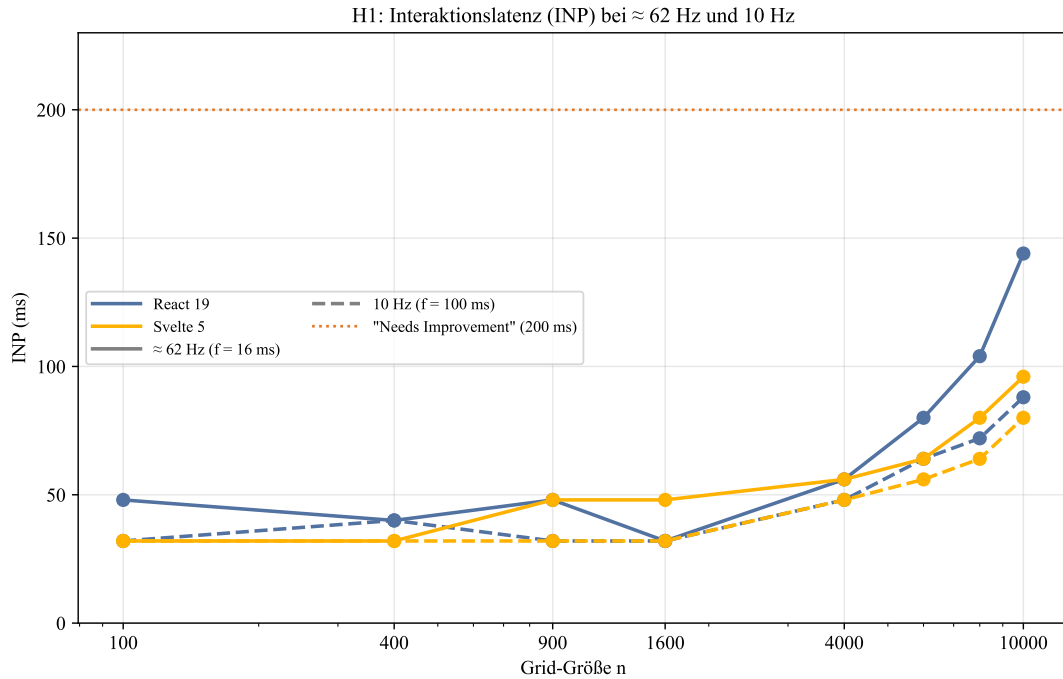


Abb. 6.4.: H1 – INP beider Frameworks bei $f \approx 62$ Hz und $f = 10$ Hz über alle n -Stufen (Abschnitt 2.4.1).

6.3.2. Zerlegung der INP-Phasen

Tabelle 6.2 teilt die INP-Komponenten *inputDelay* (iD), *processingTime* (pT) und *presentationDelay* (pD) für ausgewählte Szenarien auf und zeigt, in welcher Phase die Latenz entsteht (Abschnitt 2.4.1). Die *processingTime* trägt in keinem Szenario wesentlich zum INP bei: Sie überschreitet 1,0 ms bei React nicht (Maximum: $n = 4000$, $f = 20$ Hz) und beträgt für Svelte selbst unter Extremlast ($n = 10\,000$, $f \approx 62$ Hz) lediglich 0,7 ms. Die INP-Verschlechterung mit steigendem n wird damit ausschließlich durch *inputDelay* (blockierter Main Thread) und *presentationDelay* (Rendering nach dem Handler) bestimmt. Wie sich die Latenz auf diese beiden Phasen verteilt, hängt von der Last ab (Tabelle 6.2). Bei niedriger Last ($n \leq 1600$) macht die *presentationDelay* den größten Anteil aus: Bei React, $n = 100$, $f = 10$ Hz beträgt der *inputDelay* lediglich 0,4 ms, während die *presentationDelay* 31,5 ms ausmacht. Bei hoher UI-Dichte und niedriger Frequenz ($n = 10\,000$, $f = 10$ Hz) bleibt die *presentationDelay* zwar der größte Anteil (51,9 ms), der *inputDelay* wächst jedoch auf 36,1 ms an und trägt damit spürbar zum INP bei. Unter Extremlast kann der *inputDelay* die *presentationDelay* schließlich überschreiten: Bei React, $n = 8000$, $f \approx 62$ Hz beträgt er 69,6 ms gegenüber 34,4 ms. In nahezu allen Szenarien überwiegt die *presentationDelay* weiterhin; die beschriebene Ausnahme bleibt auf dieses eine Szenario beschränkt. Hauptursache des wachsenden *inputDelay* sind die in Abschnitt 6.1 gemessenen Scripting-Kosten, die den Main Thread blockieren. Szenariospezifische Besonderheiten werden nachfolgend diskutiert. Eine Ausnahme zeigt jedoch, dass ein niedrigerer Scripting-Aufwand nicht automatisch zu einem besseren INP führt (Tabelle 6.2, unterer Abschnitt).

Ausnahme 1 ($n = 10\,000$, $f = 20$ Hz): React erzielt einen INP von 96 ms (iD: 20,9 ms; pT: 0 ms; pD: 75,1 ms), Svelte einen INP von 104 ms (iD: 18,0 ms; pT: 0 ms; pD: 86,0 ms). Svelte weist sowohl geringere Scripting-Kosten als auch einen niedrigeren *inputDelay* auf und erzielt dennoch den höheren INP. Die Ursache liegt in der *presentationDelay* (86,0 ms vs. 75,1 ms). Sveltes höhere Rendering-Zeit bei $f = 20$ Hz (13,71 ms vs. 11,87 ms, vgl. Abschnitt 6.1.1) spricht für dieselbe Ursache, erklärt quantitativ jedoch nur einen Teil des *presentationDelay*-Unterschieds von 10,9 ms. Der genaue Mechanismus der verbleibenden Differenz lässt sich anhand der vorliegenden Daten nicht bestimmen. Das zeigt: Geringere Scripting-Kosten garantieren keinen besseren INP; der Rendering-Nachteil kann den Scripting-Vorteil überwiegen. Daneben bestätigt die Phasenerlegung einen zweiten Aspekt von H1 unmittelbar (Tabelle 6.2).

H1-Bestätigung ($n = 1600$, $f \geq 30$ Hz): Bei $\Delta t = 16$ ms beträgt der INP für React 32 ms (iD: 5,5 ms; pT: 0,1 ms; pD: 26,4 ms), für Svelte hingegen 48 ms (iD: 4,6 ms; pT: 0 ms; pD: 43,4 ms). Bei $\Delta t = 33$ ms gilt dasselbe: React 32 ms (iD: 6,7 ms; pT: 0,2 ms; pD: 25,1 ms) gegenüber Svelte 40 ms (iD: 4,5 ms; pT: 0 ms; pD: 35,5 ms). In diesen Szenarien liegen die Scripting- und Rendering-Kosten beider Frameworks auf einem niedrigen Wert (Scripting: 0,75–1,61 ms/Tick; Rendering: 1,73–1,91 ms/Tick); die Rendering-Zeiten sind dabei nahezu gleich ($f \approx 62$ Hz: 1,75 vs. 1,73 ms/Tick; $f \approx 30$ Hz: 1,91 vs. 1,87 ms/Tick). Svelte weist bei beiden Frequenzstufen den niedrigeren *inputDelay* auf (4,6 ms bzw. 4,5 ms gegenüber 5,5 ms bzw. 6,7 ms bei React). Dennoch überwiegt der Unterschied in der *presentationDelay* im Gesamtergebnis: Er beträgt 17,0 ms ($f \approx 62$ Hz) bzw. 10,4 ms ($f \approx 30$ Hz) zugunsten Reacts und bestimmt damit den höheren INP von Svelte. Die Ursache dieses pD-Unterschieds unterscheidet sich von Ausnahme 1: Dort lag sie in Sveltes höherer Rendering-Zeit. Hier sind die Rendering-Zeiten nahezu identisch, sodass ein Scheduling-Effekt als Erklärung naheliegt. Als mögliche Ursache kommt Reacts prioritätsbasiertes Scheduling infrage, das Klick-Events gegenüber Hintergrund-Updates priorisiert, während Sveltes synchrones Effect-Batching vergleichbare Priorisierungsmöglichkeiten nicht vorsieht (Abschnitte 2.2.3 und 2.3.3); der genaue Mechanismus lässt sich anhand der vorliegenden Daten nicht bestimmen. Dieser Scheduling-Vorteil Reacts tritt ausschließlich ab $f \geq 30$ Hz auf; bei $n = 1600$ und $f = 10$ Hz sowie $f = 20$ Hz sind die INP-Werte beider Frameworks gleich (32 ms, vgl. Tabelle B.1). Dieses Ergebnis ist konsistent mit H1: Bei Last unterhalb des Frame-Budgets stabilisiert Reacts Scheduling den INP, auch wenn die absoluten Scripting-Kosten höher sind – bei $n = 1600$ und $f \approx 62$ Hz beträgt der INP-Vorteil 16 ms (React 32 ms vs. Svelte 48 ms, +50 %) – ein Scheduling-Vorteil, kein Effizienz-Vorteil.

Ausnahme 2 ($n = 10\,000$, $f \approx 30$ Hz, Tabelle 6.2) bildet eine weitere, durch H1 nicht abgedeckte Einschränkung: Beide Frameworks erzielen denselben INP (104 ms), jedoch mit gegenläufiger Phasenverteilung. React weist einen *inputDelay* von 5 ms auf, was mit dem in der H1-Bestätigung beschriebenen Scheduling-Vorteil konsistent ist (Abschnitt 2.2.3), während Sveltes *inputDelay* 51,8 ms beträgt (der höchste Svelte-Wert im gesamten Datensatz). Eine *presentationDelay* von 99 ms hebt diesen Vorteil jedoch vollständig auf. Bei einer Rendering-Zeit von 8,40 ms/Tick entspricht dies der höchsten React-*presentationDelay* im Datensatz, deren Ursache sich mit den erhobenen Metriken nicht eindeutig bestimmen lässt (vgl. Abschnitt 6.6). Der Gleichstand ergibt sich damit als Überlagerung des Scheduling-Vorteils Reacts mit einem ungeklärten *presentationDelay*-Ausreißer; kein Widerspruch zu H1, aber eine weitere Einschränkung.

Tab. 6.2.: INP-Phasenerlegung für ausgewählte Szenarien (Datenquelle: `summary.csv`, vgl. Abschnitt 5.4.7)

Framework	Szenario (n / f)	INP (ms)	iD (ms)	pT (ms)	pD (ms)	Scripting (ms/Tick)
<i>Niedrige Last (pD-bestimmt)</i>						
React	100 / 10 Hz	32	0,4	0,1	31,5	0,27
<i>Hohe Last (iD wächst)</i>						
React	10 000 / 10 Hz	88	36,1	0,0	51,9	11,24
Svelte	10 000 / 10 Hz	80	1,9	0,0	78,1	4,12
<i>Extremelast (iD > pD)</i>						
React	8 000 / ≈ 62 Hz	104	69,6	0,0	34,4	3,53
Svelte	8 000 / ≈ 62 Hz	80	32,1	0,0	47,9	3,08
<i>Ausnahme 1: Svelte-INP > React trotz niedrigerem iD</i>						
React	10 000 / 20 Hz	96	20,9	0,0	75,1	11,17
Svelte	10 000 / 20 Hz	104	18,0	0,0	86,0	4,29
<i>H1-Bestätigung: Scheduling-Vorteil React bei $n = 1600$, $f \geq 30$ Hz</i>						
React	1 600 / ≈ 62 Hz	32	5,5	0,1	26,4	1,57
Svelte	1 600 / ≈ 62 Hz	48	4,6	0,0	43,4	0,75
React	1 600 / ≈ 30 Hz	32	6,7	0,2	25,1	1,61
Svelte	1 600 / ≈ 30 Hz	40	4,5	0,0	35,5	0,76
<i>Ausnahme 2: Gleichstand trotz gegenläufiger Phasenverteilung</i>						
React	10 000 / ≈ 30 Hz	104	5,0	0,0	99,0	7,43
Svelte	10 000 / ≈ 30 Hz	104	51,8	0,0	52,2	4,09

6.3.3. Bewertung von H1

Über alle vier Frequenzstufen hinweg weist Svelte bei hoher UI-Dichte ($n \geq 4000$) in zwölf von 16 Szenarien einen niedrigeren INP auf als React. In zwei weiteren Szenarien ($n = 4000$, $f = 10$ Hz und $f \approx 62$ Hz) sind die INP-Werte beider Frameworks gleich (48 ms bzw. 56 ms). Der in H1 erwartete Svelte-Vorteil bei hoher Dichte (Abschnitt 2.5.4) zeigt sich durch die höheren Scripting-Kosten von React (Abschnitt 6.1), die primär den *inputDelay* erhöhen ($n = 10\,000$, $f = 10$ Hz: 36,1 ms gegenüber 1,9 ms bei Svelte). Anzumerken ist, dass bei $n = 10\,000$ und $f \approx 62$ Hz (dem Szenario mit dem größten INP-Abstand, 96 ms vs. 144 ms) Svelte trotz geringfügig höherer Scripting-Zeit (3,66 ms vs. 3,42 ms, vgl. Abschnitt 6.1.1) einen niedrigeren *inputDelay* aufweist (34,1 ms vs. 56,9 ms). Der direkte Kausalzusammenhang Scripting $\rightarrow$ *inputDelay* greift bei $f \approx 62$ Hz somit nicht; dies deutet auf frequenzabhängige Scheduling-Effekte hin. Bei niedriger Dichte weist Svelte ebenfalls niedrigere INP-Werte auf: bei $n = 100$ ausschließlich bei $f \approx 62$ Hz (32 ms vs. 48 ms), bei $n = 400$ sowohl bei $f \approx 62$ Hz als auch bei $f = 10$ Hz (jeweils 32 ms vs. 40 ms). In allen diesen Fällen resultiert der Unterschied ausschließlich aus der *presentationDelay*, da die Scripting-Kosten auf diesen Stufen vernachlässigbar sind. Der in H1 ebenfalls vorhergesagte Scheduling-Vorteil Reacts bei Last unterhalb des Frame-Budgets bestätigt sich bei $n = 1600$ und $f \geq 30$ Hz (vgl. Abschnitt 6.3.2). Zwei Szenarien, in denen die *presentationDelay* das durch H1 vorhergesagte

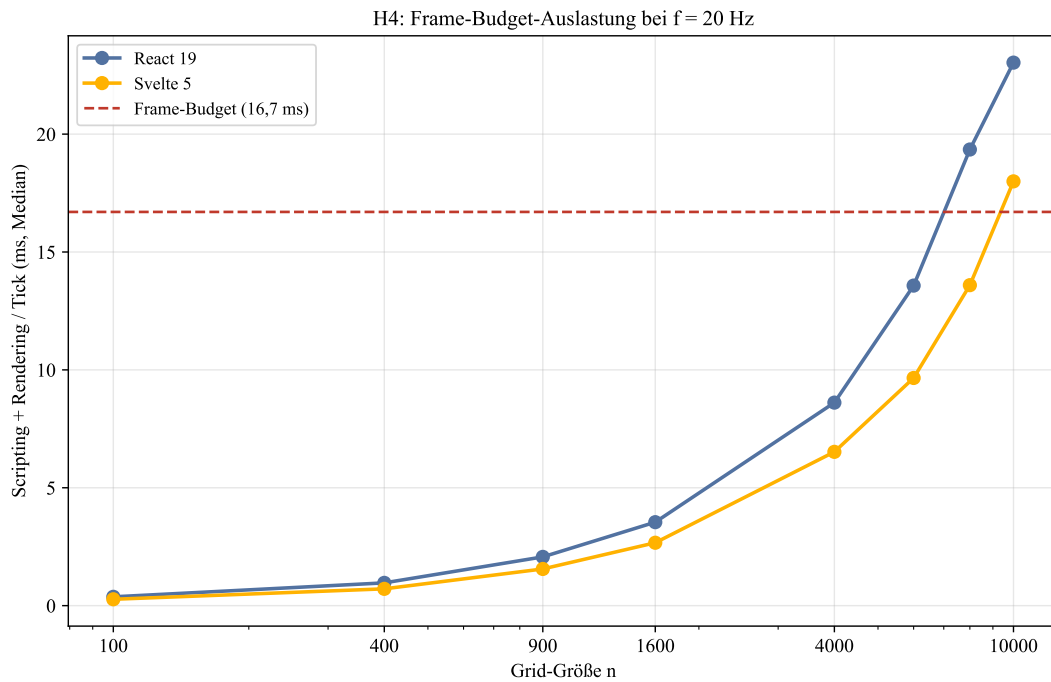
INP-Ergebnis überlagert, schränken die Gesamtaussage ein: Bei $n = 10\,000$ und $f = 20$ Hz übersteigt Sveltes INP den von React infolge einer höheren *presentationDelay* durch Rendering-Bündelung (Abschnitt 6.3.2). Bei $n = 10\,000$ und $f \approx 30$ Hz ergibt sich ein Gleichstand mit gegenläufiger, ungeklärter Phasenverteilung (Abschnitt 6.3.2). Da der Svelte-Vorteil in der deutlichen Mehrheit der Szenarien bei $n \geq 4000$ gilt und beide Teilaussagen von H1 empirisch bestätigt sind, stützen die Befunde H1 trotz dieser Einschränkungen. **H1 wird bestätigt.**

6.4. H4: Identifikation des Tipping Points

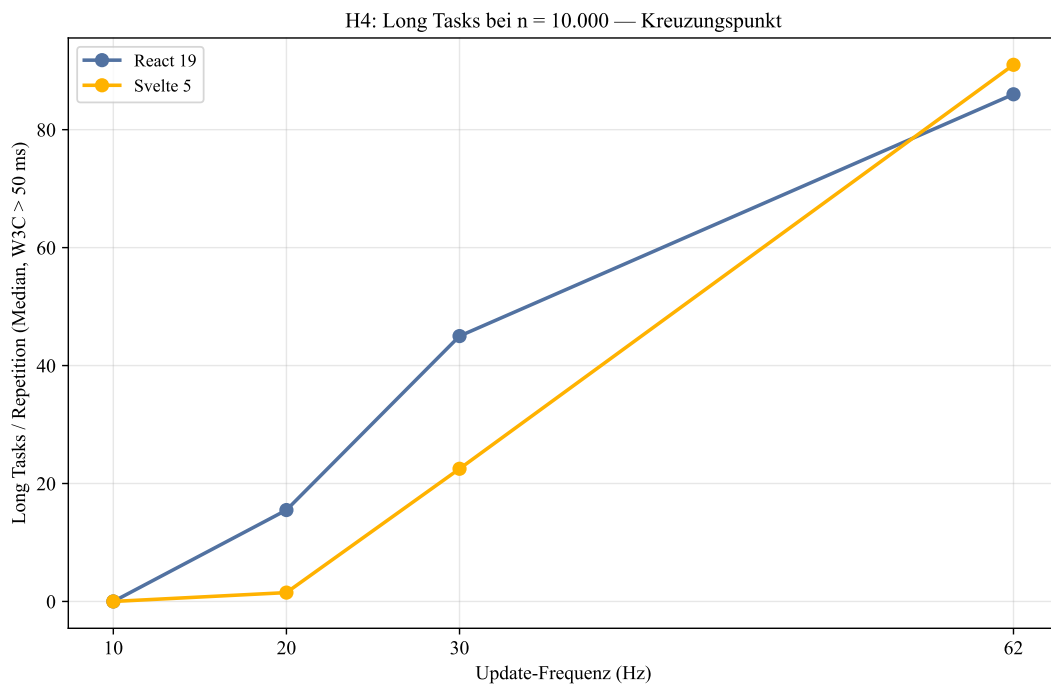
Während H2 beschreibt, wie stark der Scripting-Aufwand mit n wächst, fragt H4, ab welchem n das Frame-Budget überschritten wird und welches Framework diesen Tipping Point zuerst erreicht (Abschnitt 2.5.4).

6.4.1. Frame-Budget-Auslastung und Long Tasks

Abbildung 6.5a zeigt, bei welchem n die kombinierte Summe aus Scripting- und Rendering-Zeit die 16,7 ms-Grenze des Frame-Budgets bei $f = 20$ Hz überschreitet. React erreicht diesen Schwellenwert bei einer niedrigeren UI-Dichte als Svelte. Dies bestätigt die Vorhersage aus Abschnitt 2.5.2: Die Abstraktionskosten der Fiber-Architektur führen dazu, dass der Main Thread bei React früher vollständig ausgelastet ist als bei Svelte mit seiner feingranularen Reaktivität. Zu beachten ist, dass die 16,7 ms-Grenze als Frame-Budget nur dann direkt relevant ist, wenn Tick-Intervall und Framelänge übereinstimmen ($f \approx 62$ Hz). Bei niedrigeren Frequenzen bedeutet eine Überschreitung dieser Grenze nicht zwingend einen Frame-Drop, da der Main Thread zwischen den Ticks ausreichend Zeit für das Rendering hat (Abschnitt 2.1.2). Die Anzahl der Long Tasks (Abb. 6.5b) bestätigt dies bei niedrigen bis mittleren Frequenzen, zeigt jedoch bei $f \approx 62$ Hz eine Umkehr des Long-Task-Verhältnisses. Bei $f \approx 30$ Hz und $n = 10\,000$ erzeugt React rund 45 Long Tasks, Svelte 22,5. Die IQR-Streuung ist dabei hoch (React: 27, Svelte: 28; aus Rohdaten, nicht in Tabelle B.1); bei Svelte übersteigt der IQR sogar den Median, was die Aussagekraft dieses Vergleichs einschränkt. Ab $f \approx 62$ Hz dreht sich das Verhältnis: Svelte erzeugt 91 Long Tasks gegenüber 86 bei React. Diese Umkehrung tritt ausschließlich bei $n = 10\,000$ auf; bei $n = 8000$ und $f \approx 62$ Hz hat React mit 58 Long Tasks weiterhin deutlich mehr als Svelte mit 29. Es handelt sich somit nicht um ein generelles Muster, sondern um einen Extremlast-Effekt: Bei $f \approx 62$ Hz stapelt Sveltes synchrones Effect-Batching (Abschnitte 2.2.3 und 2.3.3) Arbeit ohne zwischenzeitliches Yielding, was zu einem überproportionalen Anstieg der Long Tasks führt (Svelte: +304 %; React: +91 %). Der Scripting-Schnittpunkt (Abschnitt 6.1.1) tritt im selben Szenario auf und ist konsistent mit diesem Befund.



(a) (Scripting + Rendering) vs. 16,7 ms-Grenze bei $f = 20$ Hz



(b) Long Tasks bei $n = 10.000$ über alle Frequenzstufen

Abb. 6.5.: H4 – Tipping-Point-Analyse: (a) Frame-Budget-Auslastung beider Frameworks bei $f = 20$ Hz mit 16,7 ms-Grenze; (b) Anzahl der Long Tasks bei $n = 10.000$ in Abhängigkeit von f (vgl. Abschnitte 2.1.2 und 4.3.1).

6.4.2. Bewertung von H4

Die Frame-Budget-Auswertung bestätigt die Hauptvorhersage von H4: React überschreitet die 16,7 ms-Grenze bei niedrigerer UI-Dichte als Svelte (Abschnitt 2.5.2). Bei $f = 10$ Hz überschreitet React die Grenze ab $n = 8000$ (19,59 ms), Svelte erst ab $n = 10\,000$ (17,67 ms); bei $f = 20$ Hz zeigt sich dasselbe Muster: React ab $n = 8000$ (19,35 ms), Svelte ab $n = 10\,000$ (18,00 ms). Diese Grenzüberschreitung pro Tick tritt ausschließlich bei $f \leq 20$ Hz auf. Die konkret ermittelten n -Schwellenwerte sind V8/Chromium-spezifisch; andere JavaScript-Engines könnten den Tipping Point bei abweichenden n/f -Kombinationen aufweisen (vgl. Abschnitt 6.6). Bei $f \approx 30$ Hz erreicht die kombinierte Scripting- und Rendering-Zeit für React maximal 15,83 ms ($n = 10\,000$: 7,43 + 8,40 ms), bei $f \approx 62$ Hz lediglich 8,37 ms (3,42 + 4,95 ms), jeweils unterhalb der Schwelle (vgl. Tabelle B.1). Die Long-Task-Analyse zeigt bei $n = 10\,000$ und $f \approx 62$ Hz eine Umkehr des Long-Task-Verhältnisses (Svelte 91 vs. React 86 Long Tasks). Diese Umkehr belegt empirisch die in H4 vorhergesagte mögliche Rangfolge-Umkehr bei extremer Last (Abschnitt 2.5.2). **H4 wird bestätigt.**

6.5. Zusammenfassung der Ergebnisse

Tabelle 6.3 fasst die Bewertungen der vier Hypothesen zusammen. H2 und H3 erklären die Befunde von H1 und H4: Die in H2 gemessenen Scripting-Kosten der Fiber-Architektur (Abschnitt 6.1) begründen, warum React in H1 früher INP-Verschlechterungen zeigt (Abschnitt 6.3) und in H4 das Frame-Budget bei niedrigerer UI-Dichte überschreitet (Abschnitt 6.4). Die in H3 gemessenen Heap-Delta-Unterschiede (Abschnitt 6.2) belegen, dass die zusätzlichen Abstraktionskosten nicht nur die CPU, sondern auch den Speicher belasten. H4 baut auf den CPU-Kurven aus H2 auf, wertet diese schwellenwertbasiert aus und bestimmt damit den Übergang vom kontrollierten Skalierungsbereich zur Main-Thread-Blockierung.

Tab. 6.3.: Zusammenfassung der Hypothesenbewertungen (H1–H4)

Hypothese	Ergebnis	Kernaussage
H1	bestätigt	Svelte erzielt bei $n \geq 4000$ in der Mehrzahl der Szenarien niedrigere INP-Werte; Reacts Scheduling-Vorteil zeigt sich erwartungsgemäß bei $n = 1600 / \geq 30$ Hz; ungeklärte Ausnahmen bei $n = 10\,000 / 20$ Hz (Svelte schlechter) und $n = 10\,000 / 30$ Hz (Gleichstand).
H2	teilweise bestätigt	React's Scripting-Aufwand pro Tick wächst bei $f \leq 30$ Hz stärker als Sveltes; bei $f \approx 62$ Hz und $n = 10\,000$ tritt ein Scripting-Schnittpunkt auf (Svelte 3,66 ms vs. React 3,42 ms).
H3	bestätigt	Qualitativ bestätigt durch Retained Size: React 131 kB vs. Svelte 19,8 kB (≈ 1310 vs. ≈ 198 B pro Komponente) bei $n = 100$ im Dev-Build (vgl. Tabelle 6.1); quantitativ im Production-Build: Heap-Delta-Faktor $2,8\times$ ($n = 100$) bis $6,4\times$ ($n = 6000$), danach leicht rückläufig.
H4	bestätigt	React überschreitet das 16,7 ms-Frame-Budget bei niedrigerem n als Svelte; bei $n = 10\,000 / 62$ Hz erzeugt Svelte mehr Long Tasks (91 vs. 86).

6.6. Limitationen

Die folgenden Einschränkungen betreffen die Interpretierbarkeit der konkreten Befunde und ergänzen die in Abschnitt 4.4.2 diskutierte externe Validität des Versuchsdesigns.

6.6.1. Fehlende Isolation des Compiler-Einflusses

React 19 wurde ausschließlich mit aktiviertem Compiler gemessen. Der Compiler lässt das komponentenbasierte Render-Modell und die Fiber-Architektur unverändert; die Reconciliation wird weiterhin ausgeführt (Abschnitt 2.2.2). Daher lässt sich im gewählten Versuchsaufbau nicht bestimmen, welcher Anteil der gemessenen Scripting-Kosten auf die Compiler-Memoisierung und welcher auf die VDOM-Reconciliation entfällt. Ein Test ohne Compiler hätte diesen Einfluss isolierbar gemacht, hätte jedoch die praktische Vergleichbarkeit beeinträchtigt, da Sveltes Vorteile ebenfalls durch Compile-Time-Optimierung entstehen (Abschnitt 2.3.1).

6.6.2. Ausschließlich Chromium

Alle Messungen basieren auf Playwrights Chromium-Instanz mit V8 als JavaScript-Engine. Da die Messergebnisse das JIT-Verhalten von V8 widerspiegeln (Abschnitt 4.4.3), könnten andere Engines – SpiderMonkey (Firefox) und JavaScriptCore (Safari) – den Tipping Point bei anderen n/f -Kombinationen aufweisen. Die Ergebnisse gelten daher primär für Chromium-basierte Umgebungen.

6.6.3. Verbleibende Browser-interne Störeinflüsse

Trotz der in Abschnitt 4.4.3 beschriebenen Vorkehrungen lassen sich Garbage-Collection-Pausen und JIT-Effekte nicht vollständig ausschließen. Der ungleichmäßige Scripting-Rückgang bei höherer Tick-Frequenz (Abschnitt 6.1.1) ist ohne V8-internes Profiling nicht eindeutig erklärbar und wurde daher im vorliegenden Kapitel als Spekulation gekennzeichnet.

6.6.4. Presentationdelay als zusammengesetzte Metrik

Die *presentationDelay* erfasst die Zeit nach Abschluss der Skriptausführung bis zur sichtbaren Darstellung der Änderungen als einzelne Kennzahl (Abschnitt 2.4.1). Eine Zerlegung in Browser-interne Teilphasen (Layout, Paint, Composite) war nicht Teil der erhobenen Metriken. Bei $n = 10\,000$ und $f = 10\text{ Hz}$ weist Svelte eine deutlich höhere *presentationDelay* auf als React (78,1 ms vs. 51,9 ms, vgl. Tabelle 6.2), obwohl die Rendering-Zeiten pro Tick nahezu identisch sind (13,55 ms vs. 13,31 ms, vgl. Abschnitt 6.1.1). Die Ursache dieser Diskrepanz lässt sich mit den erhobenen Metriken nicht eindeutig bestimmen. Anzumerken ist, dass das Phänomen nicht auf ein Framework beschränkt ist. Bei $n = 10\,000$ und $f \approx 30\text{ Hz}$ weist umgekehrt React die höchste *presentationDelay* im gesamten Datensatz auf (99 ms bei 8,40 ms/Tick Rendering-Zeit), während Sveltes *presentationDelay* dort 52,2 ms beträgt, auch hier ohne eindeutig bestimmbare Ursache. Dies stellt eine Grenze der Interpretierbarkeit der INP-Phasenerlegung dar.

7. Fazit

7.1. Beantwortung der Forschungsfrage

Die empirischen Ergebnisse zeigen, dass der Leistungsunterschied zwischen React 19 und Svelte 5 unter kombinierter Dauerlast nicht rein quantitativ, sondern architektonisch bedingt ist. Entscheidend ist dabei, dass dieselbe Architektureigenschaft jeweils Vor- und Nachteil zugleich erzeugt: Der Reconciliation-Overhead der Fiber-Engine, der bei hoher UI-Dichte zu erhöhten Scripting-Kosten, höherem Speicherbedarf und einem früheren Tipping Point führt, ermöglicht bei moderater Last und hoher Update-Frequenz ein Scheduling-Modell, das den INP stabilisiert. Umgekehrt schließt Sveltes feingranulare Reaktivität, die den Effizienzvorteil bei hoher Dichte begründet, ein vergleichbares Priorisierungsmodell aus und führt unter Extremlast zu einer Umkehr der Long-Task-Rangfolge. Eine vollständige Übersicht der Messbefunde und Hypothesenbewertungen findet sich in Tabelle 6.3. Kein Framework ist damit uneingeschränkt überlegen; welches besser skaliert, lässt sich nur lastabhängig beantworten. Die vorliegende Untersuchung schließt damit die in Abschnitte 1.2 und 3.3 identifizierten Lücken der bisherigen Forschung: Nach aktuellem Stand der Literatur lagen bislang weder vergleichbare empirische Messdaten für React 19 mit aktiviertem Compiler und Svelte 5 mit Runes unter kombinierter Dauerlast vor, noch wurden INP und Frame-Budget-Überschreitung als Metriken für diese Frameworks systematisch erfasst und ausgewertet.

7.2. Handlungsempfehlung für die Praxis

Für datenintensive Echtzeit-Anwendungen wie Monitoring-Dashboards (Abschnitt 1.1) lassen sich auf Basis der empirischen Ergebnisse vier Szenarien unterscheiden:

Für Anwendungen mit **hoher UI-Dichte** bietet Svelte 5 strukturelle Vorteile: Der geringere Scripting-Aufwand verschiebt den Tipping Point zu höheren n -Werten (Abschnitte 6.1 und 6.4); in der Mehrzahl der gemessenen Szenarien erzielt Svelte zudem niedrigere INP-Werte (Abschnitt 6.3.3). Zwei Ausnahmen schränken diesen Vorteil bei maximaler UI-Dichte ein (Abschnitt 6.3.2).

In **speicherkritischen Umgebungen**, etwa auf ressourcenbeschränkten Endgeräten oder bei Anwendungen mit vielen gleichzeitig aktiven Komponenteninstanzen, ist Svelte 5 ebenfalls klar im Vorteil, da der Heap-Verbrauch (wie die internen Datenstrukturen beider Frameworks belegen) strukturell niedriger ausfällt (Abschnitt 6.2).

Bei **moderater UI-Dichte und hoher Update-Frequenz** kann React 19 die bessere Wahl sein: Das prioritätsbasierte Scheduling der Fiber-Engine erzielt in diesem Lastbereich niedrigere INP-Werte (Abschnitt 6.3.2). Dieser Vorteil tritt ausschließlich unterhalb des Tipping Points auf, ab dem Reacts höhere Scripting-Kosten das Frame-Budget überschreiten (Abschnitt 6.4.1); bei steigender UI-Dichte überwiegen die strukturellen Vorteile von Svelte.

Bei **niedriger UI-Dichte** sind die gemessenen Unterschiede insgesamt gering; Svelte 5 zeigt vereinzelt leichte INP-Vorteile (Abschnitt 6.3.3). Die Framework-Entscheidung kann in diesem Fall anhand anderer Kriterien wie Ökosystem, Entwicklererfahrung oder bestehender Codebasis getroffen werden.

In allen Szenarien gilt: Die hier ermittelten konkreten Last-Schwellenwerte sind V8- und Chromium-spezifisch; andere JavaScript-Engines können ein abweichendes Skalierungsverhalten zeigen (Abschnitt 6.6).

7.3. Ausblick

Aus den Ergebnissen dieser Arbeit ergeben sich vier Richtungen für weiterführende Forschung.

Übertragbarkeit auf andere JavaScript-Engines. Die ermittelten Schwellenwerte und der Scripting-Schnittpunkt bei $f \approx 62$ Hz beruhen auf V8s JIT-Optimierungsprofil (Abschnitt 6.6). Eine vergleichende Studie unter SpiderMonkey und JavaScriptCore würde klären, ob die architektonischen Unterschiede zwischen signalbasierter Reaktivität und VDOM-Reconciliation engine-unabhängige Skalierungseigenschaften erzeugen oder ob die beobachteten Tipping-Point-Werte V8-spezifisch sind.

Messung des Compiler-Anteils. Die vorliegende Arbeit erfasst den kombinierten Effekt aus Fiber-Reconciliation und Compiler-Memoisierung, kann jedoch nicht bestimmen, wie viel jeder Anteil beiträgt (Abschnitt 6.6). Ein direkter A/B-Vergleich mit und ohne Compiler würde diese Trennung ermöglichen und erstmals den isolierten Overhead der automatischen Memoisierung messbar machen. Diese Information ist für die praktische Beurteilung des React Compilers relevant.

Ursachenanalyse der *presentationDelay*-Ausreißer. Bei $n = 10\,000$ konnten zwei Ausnahmen mit den erhobenen Metriken nicht erklärt werden: Sveltes INP-Nachteil bei $f = 20$ Hz und Reacts *presentationDelay*-Spitze bei $f \approx 30$ Hz (Abschnitte 6.3.2 und 6.6). Eine Chrome-Tracing-basierte Analyse würde die Zerlegung in Layout-, Paint- und Composite-Phasen ermöglichen und damit möglicherweise einen bisher unbekannten Browser-Scheduling-Effekt sichtbar machen.

Übertragbarkeit auf andere Anwendungsmuster. Der Benchmark simuliert ein teilweise aktualisiertes Grid (Abschnitt 4.1.1). Ob virtualisierte Listen, reaktive Formulare oder interaktive Diagramme ein vergleichbares Skalierungsverhalten zeigen, bleibt eine offene empirische Frage, insbesondere ob der hier beobachtete Scheduling-Vorteil Reacts bei moderater Last auf anzeigeorientierte oder interaktionsbasierte Anwendungsmuster übertragbar ist.

Literatur

- [1] MDN Contributors: *Perceived performance*. MDN Web Docs. März 2026. URL: https://developer.mozilla.org/en-US/docs/Learn/Performance/Perceived_performance (besucht am 26. 06. 2026).
- [2] J. Wagner und B. Pollard: *Interaction to Next Paint (INP)*. web.dev. Sep. 2025. URL: <https://web.dev/articles/inp> (besucht am 24. 04. 2026).
- [3] M. Furrer: „Live Dashboard“. Bachelor’s Thesis. Rotkreuz, Switzerland: Hochschule Luzern – Informatik, 2018. URL: <https://portfoliodb.hslu.ch/files/13757cae-0f1a-43c6-b56e-a69d09a97f14> (besucht am 21. 04. 2026).
- [4] MDN Contributors: *Main thread*. MDN Web Docs. Juli 2025. URL: https://developer.mozilla.org/en-US/docs/Glossary/Main_thread (besucht am 21. 04. 2026).
- [5] P. Lewis: *Rendering performance*. web.dev. Dez. 2023. URL: <https://web.dev/articles/rendering-performance> (besucht am 21. 04. 2026).
- [6] T. Kumar: *Fluent React. Build Fast, Performant, and Intuitive Web Applications*. 1. Aufl. 1005 Gravenstein Highway North, Sebastopol, CA 95472: O’Reilly Media, Feb. 2024. ISBN: 9781098138707.
- [7] R. Harris: *Virtual DOM is pure overhead*. Svelte. Dez. 2018. URL: <https://svelte.dev/blog/virtual-dom-is-pure-overhead> (besucht am 27. 04. 2026).
- [8] L. Tan, J. Savona und M. Zhang: *React Compiler v1.0*. React. 2025. URL: <https://react.dev/blog/2025/10/07/react-compiler-1> (besucht am 28. 04. 2026).
- [9] The Svelte Team: *Introducing runes*. Svelte. Sep. 2023. URL: <https://svelte.dev/blog/runes> (besucht am 02. 05. 2026).
- [10] M. Levlin: „DOM benchmark comparison of the front-end JavaScript frameworks React, Angular, Vue, and Svelte“. Master’s Thesis. Turku, Finland: Åbo Akademi University, 2020. URL: <https://www.doria.fi/handle/10024/177433> (besucht am 11. 05. 2026).
- [11] F. P. E. Putra, Hasbullah, F. Muslim und R. Paradina: „Technical Performance Comparison of Modern Frontend Frameworks Study on Svelte, React, and Vue“. In: *Brilliance: Research of Artificial Intelligence* 5.1 (Juli 2025), S. 355–364. DOI: 10.47709/brilliance.v5i1.6133. URL: <https://doi.org/10.47709/brilliance.v5i1.6133>.
- [12] R. Ollila, N. Mäkitalo und T. Mikkonen: „Modern Web Frameworks: A Comparison of Rendering Performance“. In: *Journal of Web Engineering* 21.3 (März 2022), S. 789–814. DOI: 10.13052/jwe1540-9589.21311. URL: <https://ieeexplore.ieee.org/document/10243623>.

- [13] S. Krause: *js-framework-benchmark: A comparison of the performance of a few popular javascript frameworks*. Chrome 148 results. GitHub. Mai 2026. URL: <https://github.com/krausest/js-framework-benchmark> (besucht am 11. 05. 2026).
- [14] World Wide Web Consortium: *Long Tasks API*. W3C. März 2026. URL: <https://www.w3.org/TR/longtasks-1/> (besucht am 24. 04. 2026).
- [15] MDN Contributors: *Critical rendering path*. MDN Web Docs. Feb. 2025. URL: https://developer.mozilla.org/en-US/docs/Web/Performance/Guides/Critical_rendering_path (besucht am 21. 04. 2026).
- [16] web.dev Team: *Understand the critical path*. web.dev. Nov. 2023. URL: <https://web.dev/learn/performance/understanding-the-critical-path> (besucht am 07. 05. 2026).
- [17] MDN Contributors: *Document Object Model (DOM)*. MDN Web Docs. Apr. 2026. URL: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model (besucht am 21. 04. 2026).
- [18] I. Grigorik: *Render-tree Construction, Layout, and Paint*. web.dev. März 2014. URL: <https://web.dev/articles/critical-rendering-path/render-tree-construction> (besucht am 21. 04. 2026).
- [19] J. Wagner, P. Lewis und B. Pollard: *Avoid large, complex layouts and layout thrashing*. web.dev. Mai 2025. URL: <https://web.dev/articles/avoid-large-complex-layouts-and-layout-thrashing> (besucht am 21. 04. 2026).
- [20] React Team: *Reconciliation*. React. 2024. URL: <https://legacy.reactjs.org/docs/reconciliation.html> (besucht am 24. 04. 2026).
- [21] React Team: *Introduction*. React. URL: <https://react.dev/learn/react-compiler/introduction> (besucht am 28. 04. 2026).
- [22] M. Hanol: *Beyond the Virtual DOM: Signals, Benchmarks, and the Future of Frontend Performance*. TheDataScientist. URL: <https://thedata scientist.com/beyond-the-virtual-dom-signals-benchmarks/> (besucht am 28. 04. 2026).
- [23] A. Clark: *React Fiber Architecture*. GitHub. Okt. 2016. URL: <https://github.com/acdlite/react-fiber-architecture> (besucht am 30. 04. 2026).
- [24] Liubomyr: *React Fiber: Reconciliation under the hood (with links to source code)*. Medium. März 2024. URL: <https://medium.com/@LiubomyrKL/react-fiber-reconciliation-under-the-hood-with-links-to-source-code-14408664dbd5> (besucht am 30. 04. 2026).
- [25] M. Abdelhady: *The Role of Fiber in React Rendering (Part 1): The Tree, The Walk, The Loop*. DEV. Apr. 2026. URL: <https://dev.to/congar97/the-role-of-fiber-in-react-rendering-part-1-the-tree-the-walk-the-loop-422o> (besucht am 30. 04. 2026).
- [26] P. Marshall: *Trash talk: the Orinoco garbage collector*. v8.dev. Jan. 2019. URL: <https://v8.dev/blog/trash-talk> (besucht am 30. 04. 2026).
- [27] H. Miller: *Under the hood: React*. DEV. Dez. 2025. URL: <https://dev.to/hmcodes/under-the-hood-react-3gbd> (besucht am 01. 05. 2026).

- [28] M. Mickiewicz: *React 19 Compiler vs. Svelte 5: The Virtual DOM Latency Benchmark*. sitepoint. Feb. 2026. URL: <https://www.sitepoint.com/react-19-compiler-vs-svelte-5-virtual-dom-latency-benchmark/> (besucht am 01. 05. 2026).
- [29] React Team: *React v18.0*. React. März 2022. URL: <https://react.dev/blog/2022/03/29/react-v18> (besucht am 30. 04. 2026).
- [30] K. Clark: *Svelte Gotcha — The Reactive Microtasks*. Medium. Aug. 2021. URL: <https://medium.com/geekculture/svelte-gotcha-the-reactive-microtasks-b27f00d53fb6> (besucht am 03. 05. 2026).
- [31] Svelte Contributors: *\$effect*. Svelte. URL: [https://svelte.dev/docs/svelte/\\$effect](https://svelte.dev/docs/svelte/$effect) (besucht am 03. 05. 2026).
- [32] M. Koretskyi: *Inside Fiber: in-depth overview of the new reconciliation algorithm in React*. Medium. Nov. 2018. URL: <https://medium.com/react-in-depth/inside-fiber-in-depth-overview-of-the-new-reconciliation-algorithm-in-react-e1c04700ef6e> (besucht am 30. 04. 2026).
- [33] J. Wagner, P. Walton und B. Pollard: *Optimize Interaction to Next Paint*. web.dev. Sep. 2025. URL: <https://web.dev/articles/optimize-inp> (besucht am 01. 05. 2026).
- [34] R. Harris: *Document the architecture behind Svelte*. Kommentar vom 13. Dezember 2017. Enthält grundlegende Erklärungen zur Code-Generierung und zum Verzicht auf ein Virtual DOM. Svelte. Dez. 2017. URL: <https://github.com/sveltejs/svelte/issues/1011> (besucht am 05. 05. 2026).
- [35] R. Harris: *Svelte 3: Rethinking reactivity*. Svelte. Apr. 2019. URL: <https://svelte.dev/blog/svelte-3-rethinking-reactivity> (besucht am 03. 05. 2026).
- [36] Svelte Contributors: *Svelte 5 migration guide*. Svelte. URL: <https://svelte.dev/docs/svelte/v5-migration-guide> (besucht am 02. 05. 2026).
- [37] S. A. Balasubramanian: *Signal-First Architectures: Rethinking Front-End Reactivity*. arXiv. Juni 2025. DOI: 10.48550/arXiv.2506.13815. arXiv: 2506.13815 [CS.SE]. URL: <https://arxiv.org/abs/2506.13815>.
- [38] N. Trummer: *[Runes] Allow for fine-grained reactivity in array of primitives and leave data structure intact*. GitHub. Sep. 2023. URL: <https://github.com/sveltejs/svelte/issues/9249> (besucht am 02. 05. 2026).
- [39] T. Fischer: *Svelte v5 runes benchmark results & discussion*. Svelte Repository, Discussion #13277. Mit Beiträgen von Rich Harris und Dominic Gannaway. Aufgerufen am: 5. Mai 2026. GitHub. Sep. 2024. URL: <https://github.com/sveltejs/svelte/discussions/13277> (besucht am 03. 05. 2026).
- [40] Svelte Contributors: *svelte/compiler*. Svelte. URL: <https://svelte.dev/docs/svelte/svelte-compiler> (besucht am 03. 05. 2026).
- [41] MDN Contributors: *JavaScript engine*. MDN Web Docs. URL: <https://developer.mozilla.org/en-US/docs/Glossary/Engine/JavaScript> (besucht am 05. 05. 2026).
- [42] P. Irish und P. Lewis: *Developer FAQ - Why Blink?* The Chromium Projects. URL: <https://www.chromium.org/blink/developer-faq/> (besucht am 05. 05. 2026).

- [43] M. Kearney: *Memory terminology*. Chrome for Developers. Dez. 2024. URL: <https://developer.chrome.com/docs/devtools/memory-problems/get-started> (besucht am 06. 05. 2026).
- [44] M. Kearney und S. Emelianova: *Record heap snapshots*. Chrome for Developers. Feb. 2024. URL: <https://developer.chrome.com/docs/devtools/memory-problems/heap-snapshots/> (besucht am 06. 05. 2026).
- [45] M. Bynens und T. Lorenz: *Memory Profiling*. GitHub. Okt. 2018. URL: <https://github.com/thlorenz/v8-perf/blob/master/memory-profiling.md> (besucht am 06. 05. 2026).
- [46] K. Basques und S. Emelianova: *Performance features reference*. Chrome for Developers. Apr. 2025. URL: <https://developer.chrome.com/docs/devtools/performance/reference/> (besucht am 07. 05. 2026).
- [47] Chrome DevTools Team: *Reduce JavaScript execution time*. Chrome for Developers. Mai 2019. URL: <https://developer.chrome.com/docs/lighthouse/performance/bootup-time/> (besucht am 07. 05. 2026).
- [48] T. Wiltzius: *FrameViewer How-To*. The Chromium Projects. URL: <https://www.chromium.org/developers/how-tos/trace-event-profiling-tool/using-frameviewer/> (besucht am 07. 05. 2026).
- [49] A. Bai: „Million.js: A Fast Compiler-Augmented Virtual DOM for the Web“. In: *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing (SAC '23)*. New York, NY, USA: Association for Computing Machinery, Jan. 2023, S. 161–169. ISBN: 978-1-4503-9517-5. DOI: 10.1145/3555776.3577683. URL: <https://arxiv.org/abs/2202.08409>.
- [50] G. Białecki und B. Pańczyk: „Performance analysis of Svelte and Angular applications“. In: *Journal of Computer Sciences Institute (JCSI)* 19 (Juni 2021), S. 139–143. DOI: 10.35784/jcsi.2633. URL: https://www.researchgate.net/publication/352852832_Performance_analysis_of_Svelte_and_Angular_applications (besucht am 11. 05. 2026).
- [51] Zulhijaya, M. S. Lamada und A. Wahid: „PERBANDINGAN FRAMEWORK SVELTE JS DENGAN REACT JS TERHADAP PERFORMA WEB APPLICATION BERBASIS JAVASCRIPT“. In: *Jurnal Riset Teknik Komputer (JURTIKOM)* 2.2 (Juni 2025), S. 66–81. DOI: 10.69714/f2h1kn88. URL: <https://journal.smartpublisher.id/index.php/jurtikom/article/view/717>.
- [52] D. E. Knuth: *The Art of Computer Programming: Seminumerical Algorithms*. 3. Aufl. Bd. 2. Boston, MA, USA: Addison-Wesley, Mai 2014. ISBN: 978-0-201-89684-8.
- [53] Vite Team: *Why Vite*. Vite. URL: <https://vite.dev/guide/why> (besucht am 05. 06. 2026).
- [54] React Team: *<StrictMode>*. React v19.2 Reference. React. URL: <https://react.dev/reference/react/StrictMode> (besucht am 03. 06. 2026).
- [55] V. Kirova, K. Karpov, E. Siemens, I. Zander, O. Vasylenko, D. Kachan und S. Maksymov: „Impact of Modern Virtualization Methods on Timing Precision and Performance of High-Speed Applications“. In: *Future Internet* 11.8 (Aug. 2019), S. 179. DOI: 10.3390/fi11080179. URL: <https://www.mdpi.com/1999-5903/11/8/179>.

- [56] Microsoft: *Slow performance on Windows Server when using the Balanced power plan*. Original KB number: 2207548. Microsoft Learn. Feb. 2026. URL: <https://learn.microsoft.com/en-us/troubleshoot/windows-server/performance/slow-performance-when-using-power-plan> (besucht am 08. 06. 2026).
- [57] M. Bynens und P. Kvitek: *Chrome Headless mode*. Chrome for Developers. Okt. 2024. URL: <https://developer.chrome.com/docs/chromium/headless> (besucht am 25. 05. 2026).
- [58] Chrome DevTools Protocol Team: *Tracing Domain*. Chrome for Developers. URL: <https://chromedevtools.github.io/devtools-protocol/> (besucht am 05. 06. 2026).
- [59] M. Kearney und F. Copes: *Timeline event reference*. Chrome for Developers. Apr. 2015. URL: <https://developer.chrome.com/docs/devtools/performance/timeline-reference> (besucht am 30. 05. 2026).
- [60] M. Mocny: *Event Timing API*. W3C Working Draft. World Wide Web Consortium. März 2026. URL: <https://www.w3.org/TR/2026/WD-event-timing-20260319/> (besucht am 29. 05. 2026).
- [61] Microsoft: *Playwright Docs: Auto-waiting*. Playwright. URL: <https://playwright.dev/docs/actionability> (besucht am 29. 05. 2026).
- [62] M. Alle: *Improved code caching*. V8. Apr. 2018. URL: <https://v8.dev/blog/improved-code-caching> (besucht am 12. 06. 2026).
- [63] E. Barrett, C. F. Bolz-Tereick, R. Killick, S. Mount und L. Tratt: „Virtual Machine Warmup Blows Hot and Cold“. In: *Proceedings of the ACM on Programming Languages* 1.OOPSLA (Okt. 2017). Article 52, S. 1–27. DOI: 10.1145/3133876. URL: <https://arxiv.org/abs/1602.00602>.
- [64] Microsoft: *Playwright Docs: Navigations*. Playwright. URL: <https://playwright.dev/docs/navigations> (besucht am 12. 06. 2026).
- [65] P. J. Huber: *Robust Statistics*. Wiley Series in Probability and Mathematical Statistics. New York · Chichester · Brisbane · Toronto · Singapore: John Wiley & Sons, 1981. ISBN: 0-471-41805-6.
- [66] Chromium Project: *Heap Profiling with MemoryInfra*. Google Gites. URL: https://chromium.googlesource.com/chromium/src/+66.0.3343.2/docs/memory-infra/heap_profiler.md (besucht am 05. 06. 2026).
- [67] T. Ettinger: *mulberry32.c: Mulberry32 PRNG*. GitHub Gists. 2017. URL: <https://gist.github.com/tommyettinger/46a874533244883189143505d203312c> (besucht am 01. 06. 2026).
- [68] React Team: *Rendering Lists*. *Learn React*. React. URL: <https://react.dev/learn/rendering-lists> (besucht am 05. 06. 2026).
- [69] React Team: *React API Reference: memo*. Version 19.2. React. URL: <https://react.dev/reference/react/memo> (besucht am 08. 06. 2026).
- [70] IBM Corporation: *The JIT compiler*. *IBM SDK, Java Technology Edition*. IBM Corporation. Feb. 2024. URL: <https://www.ibm.com/docs/en/sdk-java-technology/8?topic=reference-jit-compiler> (besucht am 18. 06. 2026).

- [71] WHATWG: *HTML Living Standard: 10 Web workers*. Web Hypertext Application Technology Working Group. Juni 2026. URL: <https://html.spec.whatwg.org/> (besucht am 01.06.2026).
- [72] npm Docs: *Workspaces*. npm. März 2026. URL: <https://docs.npmjs.com/cli/using-npm/workspaces> (besucht am 01.06.2026).
- [73] D. L. Mills, J. Martin, J. Burbank und W. Kasch: *Network Time Protocol Version 4: Protocol and Algorithms Specification*. RFC 5905. Internet Engineering Task Force (IETF), Juni 2010. DOI: 10.17487/RFC5905. URL: <https://www.rfc-editor.org/info/rfc5905>.
- [74] S. Gunasekaran: *Using the compiler on < React 19*. GitHub Discussion #6. GitHub. Apr. 2024. URL: <https://github.com/reactwg/react-compiler/discussions/6> (besucht am 03.06.2026).
- [75] Chromium Docs: *Using Chromium with SwiftShader*. Google / Gitiles. URL: <https://chromium.googlesource.com/chromium/src/+main/docs/gpu/swiftshader.md> (besucht am 06.06.2026).
- [76] MDN Contributors: *PerformanceObserver*. MDN Web Docs. Okt. 2024. URL: <https://developer.mozilla.org/en-US/docs/Web/API/PerformanceObserver> (besucht am 29.05.2026).

Abkürzungs- und Symbolverzeichnis

Abkürzungen

API	Application Programming Interface, Programmierschnittstelle
AST	Abstrakter Syntaxbaum (Abstract Syntax Tree)
CDP	Chrome DevTools Protocol
CPU	Central Processing Unit, Prozessor
CRP	Critical Rendering Path
CSS	Cascading Style Sheets
CSSOM	CSS Object Model
DOM	Document Object Model
FID	First Input Delay
fps	Frames per Second
GC	Garbage Collection, Speicherbereinigung
GPU	Graphics Processing Unit, Grafikprozessor
HSL	Hue, Saturation, Lightness — Farbmodell zur Darstellung von Farben
HTML	HyperText Markup Language
INP	Interaction to Next Paint
IQR	Interquartilsabstand (Interquartile Range)
JIT	Just-In-Time Compilation
JS	JavaScript
JSON	JavaScript Object Notation
JSX	JavaScript XML, Syntaxerweiterung für React-Komponenten
KI	Künstliche Intelligenz (Artificial Intelligence)
npm	Node Package Manager
OS	Operating System, Betriebssystem
POJO	Plain Old JavaScript Object, einfaches JavaScript-Objekt
RAM	Random Access Memory, Arbeitsspeicher
UI	User Interface, Benutzeroberfläche
URL	Uniform Resource Locator, einheitlicher Ressourcenzeiger
V8	V8 JavaScript Engine (Google / Chromium)
VDOM	Virtual DOM, virtuelles Document Object Model
W3C	World Wide Web Consortium
Web-IDL	Web Interface Definition Language, Schnittstellenbeschreibungssprache für Web-APIs

Lateinische Symbole

d	Dominator: Objekt, über das jeder Pfad vom GC-Root zu einem abhängigen Objekt n führt
Δ_{Heap}	JS-Heap-Delta: Differenz des <i>JSHeapUsedSize</i> zwischen Ausgangs- und Endzustand nach erzwungener GC, kB
Δt	Tick-Intervall: Zeit zwischen zwei aufeinanderfolgenden Updates, ms
f	Update-Frequenz, Hz
H_{base}	JS-Heap-Baseline: genutzter Heap-Speicher nach erzwungener GC unmittelbar vor Benchmark-Start, kB
H_{final}	JS-Heap-Endstand: genutzter Heap-Speicher nach erzwungener GC unmittelbar nach Abschluss aller Ticks, kB
N	Anzahl der Messwiederholungen pro Szenario
n	UI-Dichte: Anzahl gleichzeitig gerenderter Komponenten
$O(n)$	Landau-Notation: linearer Zeitaufwand in Abhängigkeit von n
$O(n^3)$	Landau-Notation: kubischer Zeitaufwand in Abhängigkeit von n
$\lceil \cdot \rceil$	Gaußsche Aufrundungsfunktion: kleinste ganze Zahl $\geq$ dem Ausdruck
$\lfloor \cdot \rfloor$	Gaußsche Abrundungsfunktion: größte ganze Zahl $\leq$ dem Ausdruck
t_{Frame}	Frame-Budget bei 60 fps $\approx 16,7$ ms

Abbildungsverzeichnis

2.1.	Phasenmodell des Critical Rendering Paths	4
2.2.	Strukturelle Gegenüberstellung von DOM, CSSOM und Render Tree	5
2.3.	Architekturmodell der React Fiber-Engine	8
2.4.	Zeitliches Phasenmodell der INP-Metrik	12
2.5.	Modell eines Speicher-Graphen zur Darstellung der Retained Size	14
5.1.	Paketstruktur und Abhängigkeitsbeziehungen des npm-Workspace-Monorepos. .	35
5.2.	Gegenüberstellung der Update-Strategien React 19 und Svelte 5	38
6.1.	H2 – Scripting-Auswertung beider Frameworks bei $f = 10$ Hz	47
6.2.	Chrome DevTools Heap-Snapshot ($n = 100$, Dev-Build): interne Objekte im Vergleich	49
6.3.	H3 – Speicherauswertung: JS-Heap-Delta beider Frameworks bei $f = 10$ Hz . . .	51
6.4.	H1 – INP beider Frameworks bei $f \approx 62$ Hz und $f = 10$ Hz über alle n -Stufen . . .	53
6.5.	H4 – Tipping-Point-Analyse: Frame-Budget-Auslastung und Long Tasks	57

Tabellenverzeichnis

4.1.	Frequenzstufen des Szenarios B	22
4.2.	Testmatrix (pro Framework)	23
4.3.	Hardware-Spezifikation der Messmaschine	24
4.4.	Fixierte Software-Versionen der Benchmark-Toolchain	25
6.1.	Qualitative Heap-Analyse: interne Objekt-Strukturen ($n = 100$, Dev-Build)	48
6.2.	INP-Phasenzerlegung für ausgewählte Szenarien	55
6.3.	Zusammenfassung der Hypothesenbewertungen (H1–H4)	59
B.1.	Vollständige Messergebnisse aller 64 Benchmark-Konfigurationen.	79

Anhang

A. Quellcode-Listings

Listing A.1: Kerninterfaces des Pakets @benchmark/shared-logic.

```
export interface GridItem    { id: number; value: number; }
export interface TickMutation { id: number; newValue: number; }

export interface TestPlan {
  initialState: GridItem[];
  ticks:       TickMutation[][];
}
```

Listing A.2: Bailout-Guard innerhalb der Tick-Generierung.

```
let newValue: number;
do {
  newValue = Math.floor(rand() * 100);
} while (newValue === (currentValues[id] as number));
```

Listing A.3: Drift-korrigierter Tick-Scheduler in ticker.worker.ts.

```
let expectedTime = performance.now() + intervalMs;

function scheduleNextTick() {
  const now      = performance.now();
  const drift    = now - expectedTime;
  const nextDelay = Math.max(0, intervalMs - drift);
  expectedTime  += intervalMs;

  setTimeout(() => {
    ctx.postMessage({ type: 'TICK' });
    scheduleNextTick();
  }, nextDelay);
}
```

Listing A.4: Die Komponente Grid.tsx übergibt ausschließlich den primitiven Wert value als Daten-Prop an Cell.

```
export function Grid({ items }: GridProps) {
  return (
    <div style={{ display: "flex", flexWrap: "wrap", width: 900, lineHeight: 0 }}>
      {items.map((item) => (
        <Cell key={item.id} value={item.value} />
      ))}
    </div>
  );
}
```

Listing A.5: Immutables Grid-Update in App.tsx: Nur mutierte Zellen erhalten eine neue Objektreferenz.

```
worker.onmessage = (e: MessageEvent<{ type: string }>) => {
  if (e.data.type !== 'TICK') return;
```

```
// ...
const ticks = plan.ticks;
if (tickIndexRef.current >= ticks.length) return;
const mutations = ticks[tickIndexRef.current];
setGrid((prev) => {
  const next = [...prev];
  for (const { id, newValue } of mutations) {
    next[id] = { id, value: newValue };
  }
  return next;
});
tickIndexRef.current++;
// ...
};
```

Listing A.6: Benchmark-Start-Logik in App.tsx: Der Worker startet erst auf ein externes Event hin.

```
const startWorker = () => {
  worker.postMessage({ command: 'start', intervalMs: TICK_INTERVAL_MS });
};
window.addEventListener('benchmark-start', startWorker, { once: true });

if (!IS_BENCHMARK_MODE) {
  window.dispatchEvent(new CustomEvent('benchmark-start'));
}
```

Listing A.7: Aktivierung des React Compilers in vite.config.ts.

```
import { defineConfig } from "vite";
import react, { reactCompilerPreset } from "@vitejs/plugin-react";
import babel from "@rollup/plugin-babel";

export default defineConfig({
  plugins: [
    react(),
    babel({ presets: [reactCompilerPreset()] }),
  ],
  preview: { port: 4173, strictPort: true },
});
```

Listing A.8: Modulebenen-Block in App.svelte: TestPlan wird einmalig außerhalb der Komponenteninstanz berechnet.

```
<script module lang="ts">
import { MockEngine } from '@benchmark/shared-logic';
import type { TestPlan } from '@benchmark/shared-logic';
import { GRID_SIZE, TICK_COUNT, MUTATION_RATE, SEED, TICK_INTERVAL_MS } from './config';
const engine = new MockEngine();
const plan: TestPlan = engine.generateTestPlan(
  GRID_SIZE, TICK_COUNT, MUTATION_RATE, SEED
);
</script>
```

Listing A.9: Grid-Initialisierung in App.svelte: Reaktiver Zustand auf Basis einer Shallow-Copy der initialState-Objekte.

```
let grid = $state(plan.initialState.map(item => ({ ...item })));
```

Listing A.10: Prop-Empfang in Grid.svelte und Cell.svelte via \$props().

```
// Grid.svelte
let { items }: { items: GridItem[] } = $props();
// Cell.svelte
let { value }: { value: number } = $props();
```

Listing A.11: Direkte Mutation des reaktiven Grids in App.svelte.

```
const mutations = plan.ticks[tickIndex];
for (const { id, newValue } of mutations) {
  grid[id].value = newValue;
}
```

Listing A.12: Worker-Steuerung in App.svelte via \$effect.

```
$effect(() => {
  const worker = new Worker(
    new URL('./ticker.worker.ts', import.meta.url),
    { type: 'module' }
  );
  // ... onmessage, startWorker, IS_BENCHMARK_MODE-Logik
  return () => {
    worker.terminate();
    window.removeEventListener('benchmark-start', startWorker);
  };
});
```

Listing A.13: Playwright-Konfiguration in playwright.config.ts.

```
export default defineConfig({
  testDir: './src',
  timeout: 3_200_000, // globales Timeout: 53 min
  workers: 1,
  use: {
    browserName: 'chromium',
    channel: 'chromium',
    headless: true,
    launchOptions: {
      args: process.env['NO_GPU'] ? [] : ['--disable-software-rasterizer'],
    },
    viewport: { width: 1280, height: 720 },
  },
  projects: [
    { name: 'memory', testMatch: 'memory.spec.ts' },
    { name: 'performance', testMatch: 'performance.spec.ts' },
    { name: 'inp', testMatch: 'inp.spec.ts' },
  ],
});
```

Listing A.14: Aufbau der Testmatrix in scenarios.ts.

```
export const GRID_SIZES = [100, 400, 900, 1600, 4000, 6000, 8000, 10000];
export const FREQUENCIES = [
  { intervalMs: 100, tickCount: 100 },
  { intervalMs: 50, tickCount: 200 },
  { intervalMs: 33, tickCount: 300 },
  { intervalMs: 16, tickCount: 500 },
];
export const REPETITIONS = 10;
```

```

export const FRAMEWORKS = {
  react: 'http://localhost:4173',
  svelte: 'http://localhost:4174',
};

export function allScenarios(): Scenario[] {
  const result: Scenario[] = [];
  for (const gridSize of GRID_SIZES) {
    for (const { intervalMs, tickCount } of FREQUENCIES) {
      for (const framework of Object.keys(FRAMEWORKS) as Framework[]) {
        result.push({ framework, gridSize, intervalMs, tickCount });
      }
    }
  }
  return result;
}

```

Listing A.15: Berechnung der INP-Phasen in injectINPObserver (inp.ts).

```

{
  duration:          e.duration,
  inputDelay:        e.processingStart - e.startTime,
  processingTime:    e.processingEnd - e.processingStart,
  presentationDelay: e.duration - (e.processingEnd - e.startTime),
}

```

Listing A.16: INP-Berechnung nach W3C-Spezifikation in calcINP (inp.ts).

```

const sorted      = [...allEntries].sort((a, b) => b.duration - a.duration);
const ignoreCount = Math.floor(allEntries.length / 50);
const worst       = sorted[ignoreCount] ?? sorted[0!];
return {
  inp:                worst.duration,
  inputDelay:         worst.inputDelay,
  processingTime:     worst.processingTime,
  presentationDelay:  worst.presentationDelay,
  pooledEntries:      allEntries.length,
};

```

Listing A.17: Klassifizierung von Trace-Ereignissen in parseTrace (cdp.ts).

```

// ph: 'X' = complete event (has both start timestamp and duration)
// dur field is in microseconds -> divide by 1000 for milliseconds
const SCRIPTING_NAMES = new Set([
  'EvaluateScript', 'FunctionCall', 'EventDispatch',
  'v8.execute', 'RunMicrotasks',
]);
const RENDERING_NAMES = new Set([
  'RecalculateStyles', 'Layout',
  'UpdateLayerTree', 'ParseAuthorStyleSheet',
]);

```

B. Vollständige Messergebnisse

Die folgende Tabelle dokumentiert die aggregierten Kennwerte aller 64 Benchmark-Konfigurationen (8 n -Stufen $\times$ 4 Frequenzstufen $\times$ 2 Frameworks, je 10 Wiederholungen). Alle Werte sind Mediane über die 10 Wiederholungen; der Interquartilsabstand steht jeweils in Klammern. Abkürzungen: FW = Framework (R = React 19, S = Svelte 5); n = Zellanzahl; f = Updatefrequenz in Hz; INP = Interaction to Next Paint in ms; iD = *inputDelay* in ms; pT = *processingTime* in ms; pD = *presentationDelay* in ms; Scr. Md. = *scriptingMsPerTick*-Median in ms/Tick; Ren. Md. = *renderingMsPerTick*-Median in ms/Tick; LT = *longTaskCount*-Median; Heap Δ = JS-Heap-Delta in MB.

Tab. B.1.: Vollständige Messergebnisse aller 64 Benchmark-Konfigurationen.

FW	n	f (Hz)	INP (ms)	iD (ms)	pT (ms)	pD (ms)	Scr. Md. (ms/Tick)	(IQR)	Ren. Md. (ms/Tick)	(IQR)	LT	Heap Δ (MB)
<i>React 19</i>												
R	100	10	32	0,4	0,1	31,5	0,27	(0,01)	0,15	(0,01)	0	0,42
R	100	20	32	0,3	0,1	31,6	0,23	(0,00)	0,14	(0,00)	0	0,47
R	100	≈ 30	32	0,4	0,0	31,6	0,22	(0,01)	0,14	(0,00)	0	0,51
R	100	≈ 62	48	0,2	0,0	47,8	0,20	(0,01)	0,14	(0,01)	0	0,53
R	400	10	40	0,4	0,2	39,4	0,60	(0,02)	0,45	(0,01)	0	0,56
R	400	20	32	1,8	0,0	30,2	0,52	(0,03)	0,44	(0,01)	0	0,60
R	400	≈ 30	32	0,4	0,2	31,4	0,50	(0,03)	0,43	(0,00)	0	0,60
R	400	≈ 62	40	1,0	0,0	39,0	0,49	(0,02)	0,41	(0,01)	0	0,65
R	900	10	32	3,1	0,0	28,9	1,11	(0,03)	1,06	(0,01)	0	0,72
R	900	20	32	1,2	0,0	30,8	1,02	(0,05)	1,05	(0,01)	0	0,78
R	900	≈ 30	32	3,6	0,0	28,4	0,97	(0,02)	1,05	(0,01)	0	0,80
R	900	≈ 62	48	2,3	0,1	45,6	0,98	(0,05)	1,00	(0,02)	0	0,85
R	1 600	10	32	0,6	0,0	31,4	1,69	(0,03)	1,89	(0,02)	0	1,06
R	1 600	20	32	0,5	0,0	31,5	1,65	(0,10)	1,89	(0,01)	0	1,10
R	1 600	≈ 30	32	6,7	0,2	25,1	1,61	(0,10)	1,91	(0,01)	0	1,12
R	1 600	≈ 62	32	5,5	0,1	26,4	1,57	(0,02)	1,75	(0,02)	0	1,11
R	4 000	10	48	11,8	0,1	36,1	4,22	(0,14)	4,74	(0,08)	0	2,13
R	4 000	20	56	11,9	1,0	43,1	4,00	(0,28)	4,61	(0,10)	0	2,04
R	4 000	≈ 30	56	12,6	0,1	43,3	4,21	(0,12)	4,60	(0,03)	0	2,15
R	4 000	≈ 62	56	16,0	0,1	39,9	2,88	(0,05)	2,83	(0,03)	0	2,22
R	6 000	10	64	18,3	0,1	45,6	6,53	(0,47)	7,38	(0,08)	0	2,93
R	6 000	20	88	19,3	0,1	68,6	6,35	(0,35)	7,22	(0,26)	0	2,96
R	6 000	≈ 30	88	24,9	0,0	63,1	6,40	(0,17)	5,82	(0,06)	0	3,00
R	6 000	≈ 62	80	25,4	0,0	54,6	3,27	(0,17)	3,51	(0,06)	0	2,95
R	8 000	10	72	4,8	0,1	67,1	9,13	(0,83)	10,46	(0,15)	0	3,62
R	8 000	20	96	19,1	0,1	76,8	9,30	(0,57)	10,05	(0,18)	0	3,69
R	8 000	≈ 30	96	24,5	0,0	71,5	7,24	(0,40)	7,36	(0,08)	0,5	3,77
R	8 000	≈ 62	104	69,6	0,0	34,4	3,53	(0,20)	4,32	(0,07)	58	3,73
R	10 000	10	88	36,1	0,0	51,9	11,24	(0,66)	13,31	(0,48)	0	4,45
R	10 000	20	96	20,9	0,0	75,1	11,17	(0,51)	11,87	(0,18)	15,5	4,51
R	10 000	≈ 30	104	5,0	0,0	99,0	7,43	(0,22)	8,40	(0,07)	45	4,56
R	10 000	≈ 62	144	56,9	0,0	87,1	3,42	(0,13)	4,95	(0,05)	86	4,83
<i>Svelte 5</i>												

(wird fortgesetzt)

(Fortsetzung von vorheriger Seite)

FW	<i>n</i>	<i>f</i> (Hz)	INP (ms)	iD (ms)	pT (ms)	pD (ms)	Scr. Md. (ms/Tick)	(IQR)	Ren. Md. (ms/Tick)	(IQR)	LT	Heap Δ (MB)
S	100	10	32	0,3	0,0	31,7	0,15	(0,00)	0,14	(0,00)	0	0,15
S	100	20	32	0,4	0,0	31,6	0,13	(0,00)	0,14	(0,00)	0	0,16
S	100	≈30	32	0,3	0,0	31,7	0,13	(0,00)	0,13	(0,01)	0	0,16
S	100	≈62	32	0,4	0,0	31,6	0,12	(0,00)	0,14	(0,02)	0	0,18
S	400	10	32	0,2	0,0	31,8	0,35	(0,01)	0,44	(0,00)	0	0,16
S	400	20	32	1,3	0,0	30,7	0,29	(0,02)	0,42	(0,01)	0	0,17
S	400	≈30	32	0,3	0,0	31,7	0,26	(0,01)	0,42	(0,00)	0	0,18
S	400	≈62	32	1,1	0,0	30,9	0,25	(0,01)	0,41	(0,00)	0	0,17
S	900	10	32	0,6	0,1	31,3	0,59	(0,01)	1,03	(0,00)	0	0,19
S	900	20	32	0,4	0,0	31,6	0,53	(0,02)	1,03	(0,01)	0	0,17
S	900	≈30	32	0,4	0,0	31,6	0,48	(0,01)	1,03	(0,01)	0	0,18
S	900	≈62	48	1,8	0,0	46,2	0,47	(0,01)	0,98	(0,01)	0	0,20
S	1 600	10	32	3,0	0,0	29,0	0,87	(0,01)	1,86	(0,01)	0	0,21
S	1 600	20	32	5,8	0,0	26,2	0,80	(0,03)	1,87	(0,02)	0	0,22
S	1 600	≈30	40	4,5	0,0	35,5	0,76	(0,01)	1,87	(0,02)	0	0,22
S	1 600	≈62	48	4,6	0,0	43,4	0,75	(0,02)	1,73	(0,02)	0	0,24
S	4 000	10	48	0,9	0,1	47,0	1,81	(0,11)	4,74	(0,08)	0	0,36
S	4 000	20	48	8,8	0,0	39,2	1,77	(0,04)	4,75	(0,09)	0	0,37
S	4 000	≈30	48	8,6	0,2	39,2	1,78	(0,15)	4,61	(0,04)	0	0,39
S	4 000	≈62	56	20,6	0,2	35,2	1,69	(0,04)	3,30	(0,03)	0	0,38
S	6 000	10	56	15,4	0,0	40,6	2,64	(0,07)	7,33	(0,14)	0	0,46
S	6 000	20	64	15,0	0,0	49,0	2,53	(0,05)	7,13	(0,06)	0	0,51
S	6 000	≈30	64	23,6	0,2	40,2	2,65	(0,04)	7,50	(0,08)	0	0,54
S	6 000	≈62	64	22,3	0,0	41,7	2,42	(0,04)	3,85	(0,03)	0	0,54
S	8 000	10	64	1,5	0,0	62,5	3,49	(0,08)	10,12	(0,12)	0	0,58
S	8 000	20	80	18,2	0,0	61,8	3,41	(0,10)	10,18	(0,19)	0	0,61
S	8 000	≈30	80	22,8	0,0	57,2	3,30	(0,19)	8,59	(0,07)	0	0,58
S	8 000	≈62	80	32,1	0,0	47,9	3,08	(0,07)	4,45	(0,06)	29	0,66
S	10 000	10	80	1,9	0,0	78,1	4,12	(0,10)	13,55	(0,13)	0	0,72
S	10 000	20	104	18,0	0,0	86,0	4,29	(0,10)	13,71	(0,30)	1,5	0,72
S	10 000	≈30	104	51,8	0,0	52,2	4,09	(0,11)	9,45	(0,04)	22,5	0,76
S	10 000	≈62	96	34,1	0,7	61,2	3,66	(0,10)	4,92	(0,07)	91	0,75